Modicon M580

BMECXM CANopen Modules
User Manual

09/2017

EIO0000002129.02

www.schneider-electric.com

Schneider Electric

The information provided in this documentation contains general descriptions and/or technical characteristics of the performance of the products contained herein. This documentation is not intended as a substitute for and is not to be used for determining suitability or reliability of these products for specific user applications. It is the duty of any such user or integrator to perform the appropriate and complete risk analysis, evaluation and testing of the products with respect to the relevant specific application or use thereof. Neither Schneider Electric nor any of its affiliates or subsidiaries shall be responsible or liable for misuse of the information contained herein. If you have any suggestions for improvements or amendments or have found errors in this publication, please notify us.

You agree not to reproduce, other than for your own personal, noncommercial use, all or part of this document on any medium whatsoever without permission of Schneider Electric, given in writing. You also agree not to establish any hypertext links to this document or its content. Schneider Electric does not grant any right or license for the personal and noncommercial use of the document or its content, except for a non-exclusive license to consult it on an "as is" basis, at your own risk. All other rights are reserved.

All pertinent state, regional, and local safety regulations must be observed when installing and using this product. For reasons of safety and to help ensure compliance with documented system data, only the manufacturer should perform repairs to components.

When devices are used for applications with technical safety requirements, the relevant instructions must be followed.

Failure to use Schneider Electric software or approved software with our hardware products may result in injury, harm, or improper operating results.

Failure to observe this information can result in injury or equipment damage.

© 2017 Schneider Electric. All Rights Reserved.

Table of Contents

Safety Information 7
About the Book 11
Part I CANopen Hardware Implementation 15
Chapter 1 Generalities 17
Module Description 18
Communication Profile 23
Key Features of BMECXM Modules 25
BMECXM0100H (Hardened) Module 28
Chapter 2 BMECXM Module Installation and Replacement 29
BMECXM Module Installation 30
BMECXM Module Replacement 33
Part II CANopen Software Implementation 35
Chapter 3 Generalities 37
Implementation Presentation 38
Maximum Configuration 40
Device PDO Allocation 41
Performance 43
Operating Modes 45
Fallback Strategy 50
Chapter 4 CANopen Configuration 53
4.1 Overview 54
Overview 54
4.2 Adding a CANopen X80 Master BMECXM module 56
Adding a CANopen X80 Master BMECXM Module 56
4.3 Bus Configuration 58
Access the CANopen Bus Editor 59
Adding Slave Devices on the CANopen Bus 61
Delete/Move/Duplicate a Device on the CANopen Bus 65
View CANopen Bus in the Project Browser 67
4.4 Device Configuration 68
Presentation of CANopen Devices 69
Slave Functions 70
Configuration Using Unity Pro 74
Configuration Using an External Tool 85

4.5 Master Configuration 87
CANopen Master Module Configuration Window 88
CANopen Master Port Configuration Screen 90
Chapter 5 Ethernet Services Configuration 95
DTM Browser 96
DTM User Interface 99
Ethernet IO Tab 104
Security Tab 107
SNMP Tab 109
NTP Tab 111
Chapter 6 Language Objects 113
Implicit Process Data Exchange 114
Device DDT Variables 115
Chapter 7 Programming 117
Network Management Services 118
Exchanges Using SDOs 121
READ_SDO: Reading Service Data Object 122
WRITE_SDO: Writing Service Data Object 125
Function Block Examples 128
Chapter 8 Diagnostics 129
LED Diagnostic 130
Device DDT for BMECXM Modules 135
Device DDT for CANopen Slave Devices 138
BMECXM DTM Diagnosis 139
Sending Explicit Messages to BMECXM module 142
Embedded Web Pages 145
Emergency Objects 151
Chapter 9 Firmware Upgrade 155
Firmware Update 155
Appendices 157
Appendix A CANopen Master Local Object Dictionary Entry 159
Object Dictionary Entries According to Profile DS301 160
Object Dictionary Entries According to Profile DS302 163
BMECXM Manufacturer Specific Object Dictionary Entries 165

Appendix B CANopen Commands 167
CANopen SDO Commands 168
CANopen SDO Abort Code 170
CANopen Start Command 172
CANopen Slave Enabling Command 173
Appendix C CIP Objects 175
DIAG_FXM_Diagnostic Object 176
DIAG_CXM Object 181
EIP Interface Diagnostic Object 183
I/O Connection Diagnostics Object 187
EtherNet/IP Explicit Connection Diagnostic Object 190
Glossary 193
Index 201

Safety Information

Important Information

NOTICE

Read these instructions carefully, and look at the equipment to become familiar with the device before trying to install, operate, service, or maintain it. The following special messages may appear throughout this documentation or on the equipment to warn of potential hazards or to call attention to information that clarifies or simplifies a procedure.



The addition of this symbol to a “Danger” or “Warning” safety label indicates that an electrical hazard exists which will result in personal injury if the instructions are not followed.



This is the safety alert symbol. It is used to alert you to potential personal injury hazards. Obey all safety messages that follow this symbol to avoid possible injury or death.

⚠ DANGER

DANGER indicates a hazardous situation which, if not avoided, **will result in** death or serious injury.

⚠ WARNING

WARNING indicates a hazardous situation which, if not avoided, **could result in** death or serious injury.

⚠ CAUTION

CAUTION indicates a hazardous situation which, if not avoided, **could result** in minor or moderate injury.

NOTICE

NOTICE is used to address practices not related to physical injury.

PLEASE NOTE

Electrical equipment should be installed, operated, serviced, and maintained only by qualified personnel. No responsibility is assumed by Schneider Electric for any consequences arising out of the use of this material.

A qualified person is one who has skills and knowledge related to the construction and operation of electrical equipment and its installation, and has received safety training to recognize and avoid the hazards involved.

BEFORE YOU BEGIN

Do not use this product on machinery lacking effective point-of-operation guarding. Lack of effective point-of-operation guarding on a machine can result in serious injury to the operator of that machine.

⚠ WARNING

UNGUARDED EQUIPMENT

- Do not use this software and related automation equipment on equipment which does not have point-of-operation protection.
- Do not reach into machinery during operation.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

This automation equipment and related software is used to control a variety of industrial processes. The type or model of automation equipment suitable for each application will vary depending on factors such as the control function required, degree of protection required, production methods, unusual conditions, government regulations, etc. In some applications, more than one processor may be required, as when backup redundancy is needed.

Only you, the user, machine builder or system integrator can be aware of all the conditions and factors present during setup, operation, and maintenance of the machine and, therefore, can determine the automation equipment and the related safeties and interlocks which can be properly used. When selecting automation and control equipment and related software for a particular application, you should refer to the applicable local and national standards and regulations. The National Safety Council's Accident Prevention Manual (nationally recognized in the United States of America) also provides much useful information.

In some applications, such as packaging machinery, additional operator protection such as point-of-operation guarding must be provided. This is necessary if the operator's hands and other parts of the body are free to enter the pinch points or other hazardous areas and serious injury can occur. Software products alone cannot protect an operator from injury. For this reason the software cannot be substituted for or take the place of point-of-operation protection.

Ensure that appropriate safeties and mechanical/electrical interlocks related to point-of-operation protection have been installed and are operational before placing the equipment into service. All interlocks and safeties related to point-of-operation protection must be coordinated with the related automation equipment and software programming.

NOTE: Coordination of safeties and mechanical/electrical interlocks for point-of-operation protection is outside the scope of the Function Block Library, System User Guide, or other implementation referenced in this documentation.

START-UP AND TEST

Before using electrical control and automation equipment for regular operation after installation, the system should be given a start-up test by qualified personnel to verify correct operation of the equipment. It is important that arrangements for such a check be made and that enough time is allowed to perform complete and satisfactory testing.

⚠ WARNING

EQUIPMENT OPERATION HAZARD

- Verify that all installation and set up procedures have been completed.
- Before operational tests are performed, remove all blocks or other temporary holding means used for shipment from all component devices.
- Remove tools, meters, and debris from equipment.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Follow all start-up tests recommended in the equipment documentation. Store all equipment documentation for future references.

Software testing must be done in both simulated and real environments.

Verify that the completed system is free from all short circuits and temporary grounds that are not installed according to local regulations (according to the National Electrical Code in the U.S.A, for instance). If high-potential voltage testing is necessary, follow recommendations in equipment documentation to prevent accidental equipment damage.

Before energizing equipment:

- Remove tools, meters, and debris from equipment.
- Close the equipment enclosure door.
- Remove all temporary grounds from incoming power lines.
- Perform all start-up tests recommended by the manufacturer.

OPERATION AND ADJUSTMENTS

The following precautions are from the NEMA Standards Publication ICS 7.1-1995 (English version prevails):

- Regardless of the care exercised in the design and manufacture of equipment or in the selection and ratings of components, there are hazards that can be encountered if such equipment is improperly operated.
- It is sometimes possible to misadjust the equipment and thus produce unsatisfactory or unsafe operation. Always use the manufacturer's instructions as a guide for functional adjustments. Personnel who have access to these adjustments should be familiar with the equipment manufacturer's instructions and the machinery used with the electrical equipment.
- Only those operational adjustments actually required by the operator should be accessible to the operator. Access to other controls should be restricted to prevent unauthorized changes in operating characteristics.

About the Book

At a Glance

Document Scope

This manual describes the implementation of a CANopen fieldbus on the Modicon M580 range.

NOTE: Regarding Safety considerations, “Emergency objects” and “Fatal error” are mentioned in this manual in conformance with the definition from the DS301 document of the CiA (CAN in Automation).

Validity Note

This documentation is valid for Unity Pro 13.0 or later.

The technical characteristics of the devices described in this document also appear online. To access this information online:

Step	Action
1	Go to the Schneider Electric home page *www.schneider-electric.com*.
2	In the **Search** box type the reference of a product or the name of a product range. ● Do not include blank spaces in the reference or product range. ● To get information on grouping similar modules, use asterisks (*).
3	If you entered a reference, go to the **Product Datasheets** search results and click on the reference that interests you. If you entered the name of a product range, go to the **Product Ranges** search results and click on the product range that interests you.
4	If more than one reference appears in the **Products** search results, click on the reference that interests you.
5	Depending on the size of your screen, you may need to scroll down to see the data sheet.
6	To save or print a data sheet as a .pdf file, click **Download XXX product datasheet**.

The characteristics that are presented in this manual should be the same as those characteristics that appear online. In line with our policy of constant improvement, we may revise content over time to improve clarity and accuracy. If you see a difference between the manual and online information, use the online information as your reference.

Related Documents

Title of Documentation	Reference Number
Modicon M580, Hardware, Reference Manual	EIO0000001578 (English), EIO0000001579 (French), EIO0000001580 (German), EIO0000001582 (Italian), EIO0000001581 (Spanish), EIO0000001583 (Chinese)
Modicon M580 Standalone, System Planning Guide for Frequently Used Architectures	HRB62666 (English), HRB65318 (French), HRB65319 (German), HRB65320 (Italian), HRB65321 (Spanish), HRB65322 (Chinese)
Hardware Catalog Manager, Software, Operation Guide	EIO0000002141 (English), EIO0000002142 (French), EIO0000002143 (German), EIO0000002144 (Italian), EIO0000002145 (Spanish), EIO0000002146 (Chinese)
Modicon Controllers Platform Cyber Security, Reference Manual	EIO0000001999 (English), EIO0000002001 (French), EIO0000002000 (German), EIO0000002002 (Italian), EIO0000002003 (Spanish), EIO0000002004 (Chinese)
CANopen, Hardware Setup Manual	35010857 (English), 35010859 (French), 35010858 (German), 35010861 (Italian), 35010860 (Spanish), 33004206 (Chinese)
Grounding and Electromagnetic Compatibility of PLC Systems, Basic Principles and Measures, User Manual	33002439 (English), 33002440 (French), 33002441 (German), 33003702 (Italian), 33002442 (Spanish), 33003703 (Chinese)

Title of Documentation	Reference Number
Unity Pro, Communication, Block Library	33002527 (English), 33002528 (French), 33002529 (German), 33003682 (Italian), 33002530 (Spanish), 33003683 (Chinese)
Unity Loader, A SoCollaborative, User Manual	33003805 (English), 33003806 (French), 33003807 (German), 33003809 (Italian), 33003808 (Spanish), 33003810 (Chinese)
Unity Pro, Program Languages and Structure, Reference Manual	35006144 (English), 35006145 (French), 35006146 (German), 35013361 (Italian), 35006147 (Spanish), 35013362 (Chinese)

You can download these technical publications and other technical information from our website at http://www.schneider-electric.com/en/download

Product Related Information

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

The application of this product requires expertise in the design and programming of control systems. Only persons with such expertise should be allowed to program, install, alter, and apply this product.

Follow all local and national safety codes and standards.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Part I
CANopen Hardware Implementation

Subject of This Part

This part describes the various hardware configuration possibilities of a CANopen bus architecture.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
1	Generalities	17
2	BMECXM Module Installation and Replacement	29

Chapter 1
Generalities

Introduction

This chapter presents the BMECXM modules equipped with a CANopen port.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Module Description	18
Communication Profile	23
Key Features of BMECXM Modules	25
BMECXM0100H (Hardened) Module	28

Module Description

Overview

The CANopen X80 master modules are declined into two references:

- BMECXM0100
- BMECXM0100H (hardened)

In Unity Pro, there is only one device part number BME CXM 0100 in the **Hardware Catalog** to declare and configure both CANopen X80 master module references.

The CANopen X80 master modules (BMECXM) provide access to CANopen bus on a M580 PAC.

General Characteristics

The table below describes the CAN characteristics of the module:

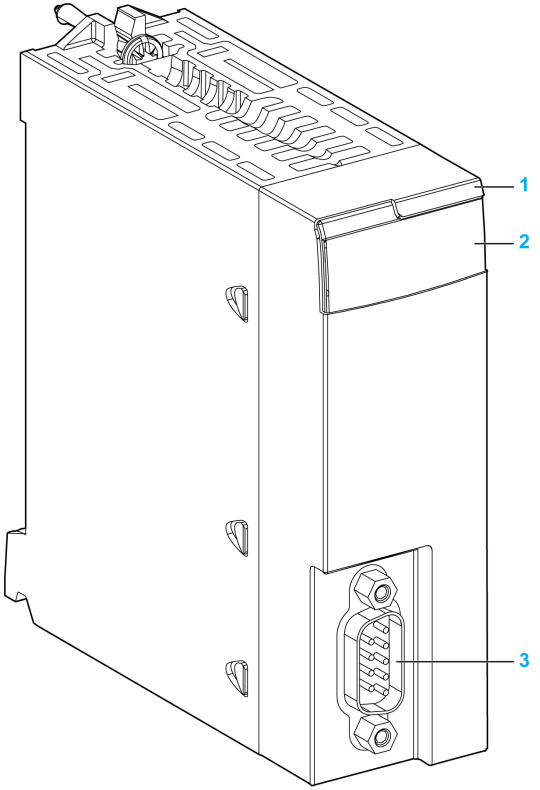
Characteristics		Description
Protocol supported		CANopen
Connection type		Sub-D 9, male
Standard		CANopen CiA 301 V4.2
Maximum cable length		Refer to chapter *Transmission Speed and Cable Length (see CANopen, Hardware Setup Manual)*.
Isolation between CAN bus and ground		500 Vac RMS, 700 Vdc
CAN bus transmission baud rate (kbd)		20, 50, 125, 250, 500, 1000
CANopen slave devices supported		63 maximum
Services	NMT	NMT master according to DS 301
		Boot-up procedure according to DS 302
	SDO	1 SDO client 1 SDO server
	PDO	256 PDOs IN and 256 PDOs OUT
	SYNC	Producer
	Emergency message	Consumer only
	Health	Heartbeat • 1 producer • 63 consumers
		Node Guarding

The BMECXM modules comply with the relevant standards and rules for electrical equipment in an industrial automation environment.

For details, refer to chapter *Standards, Certifications and Conformity Tests (see Modicon M580, Hardware, Reference Manual)*.

Physical Description

This figure shows the external features of the module:



Number	Element	Function
1	Module name	BMECXM0100 or BMECXM0100H
2	LED array	Observe the LED display to diagnose the module.
3	SUB-D 9 connector	CANopen port

LEDs

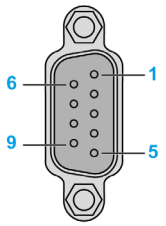
The LED display on the front of the module gives information about module operating status and about CANopen communication status:

LED	Color	Description
RUN	Green	Indicates module operating status.
ERR	Red	Detected error in module operation.
I/O	Red	Indicate exchange status with CANopen devices.
BS (Bus Status)	Red / green	Indicates the EtherNet/IP connection status.
	Yellow	Firmware upgrade in progress
CAN RUN	Green	Indicates the status of the CANopen fieldbus.
CAN ERR	Red	Indicates the status of the CANopen physical layer and indicates detected errors due to missing CAN messages (SYNC, node-guarding or heartbeat)
CAN COM	Yellow	Dedicated to SDO transmission

NOTE: Refer to the section LED diagnostics *(see page 130)* for information describing how to use the LEDs to diagnose the state of the module and the CANopen operations.

CANopen Connector

The figure and table below give the CANopen connector pin assignment:



Pin	Signal	Description
1	–	Reserved
2	CAN_L	CAN_L bus line (Low)
3	CAN_GND	CAN ground
4	–	Reserved
5	CAN_SHLD	CAN shield
6	CAN_GND	CAN ground

Pin	Signal	Description
7	CAN_H	CAN_H bus line (High)
8	–	Reserved
9	Reserved	CAN external power supply that is dedicated to the optocouplers power and transmitters-receivers (optional).

Backplane Connector Connection

The Ethernet bus interface at the back of the BMECXM module connects to the Ethernet backplane connector when you mount the module in the rack *(see page 30)*.

The module is powered by the backplane. It is hot swappable, that is, it may be installed and uninstalled without turning off the power supply to the rack.

The X Bus connector of the backplane is not used by the module as it is an Ethernet only module.

The module uses the Ethernet bus on the Ethernet backplane to manage the connectivity to the Ethernet I/O scanner.

The module can be managed by:

- The Ethernet RIO scanner of the CPU.
- The Ethernet DIO scanner of the CPU.

The module communicates with a PC that is connected to the Ethernet network using an asset management, a network manager, or a web browser.

Communication Profile

Overview

Two different M580 CPUs exist:

- BMEP58••40 including one RIO scanner and one DIO scanner.
- BMEP58••20 including one DIO scanner

Depending on the level performance requested by the process, the BMECXM module can be scanned by the RIO or the DIO scanner of the M580 CPU. Both scanners use EtherNet/IP to scan the module.

In the same M580 PAC, several BMECXM modules can be connected to the same or different I/O scanner.

For detail on EtherNet/IP connection behavior when the module switched to `FALLBACK` state, refer to chapter Fallback Strategy *(see page 50)*.

RIO Scanner

When the BMECXM is scanned by the RIO scanner, the main capabilities are:

- Higher performance constraints
- Highest performance expectation (bandwidth sharing and controlled cycle timing on CANopen and EtherNet/IP).
- Bandwidth controlled (RSTP)
- Timing and cycle synchronized (with either MAST task or FAST task.)
- Support up to 24 kb of IO data.
- RPI is automatically calculated by Unity Pro
- Motion function blocs (MFB) are supported

DIO Scanner

When the BMECXM is scanned by the DIO scanner, the main capabilities are:

- Lower performance constraints
- Lowest performance expectation, no constraint (no bandwidth sharing and no controlled cycle).
- No control and no synchronization.
- Can only support up to 8 kb of IO data.
- No support of motion function blocs (MFB)

Data Consistency

Whatever the BMECXM is scanned by RIO or DIO scanner, all the variables belonging to the same slave are kept consistent, meaning exchanged in the same EtherNet/IP assembly (one for input, one for outputs).

The same EtherNet/IP assembly can contain the data of several devices, and even the data of all the devices (the complete process image).

NOTE: When the BMECXM is scanned by the RIO scanner, all the BMECXM process image is refreshed in the PLC scan. This service is interesting only if the process image has been refreshed on the CANopen fieldbus side during the same period.

Key Features of BMECXM Modules

Communication Profile

The module manages the communication with:

- The CANopen slaves according to the CiA 301 V4.2 standard
- The M580 CPU over Ethernet I/O
  NOTE: The minimum firmware version for the M580 CPU is V2.20.

NOTE: Communication with Ethernet I/O scanner modules is not possible.

Two communication profiles can be used:

- Remote (RIO scanner) linked to AUX, MAST, or FAST tasks.
- Distributed (DIO scanner)

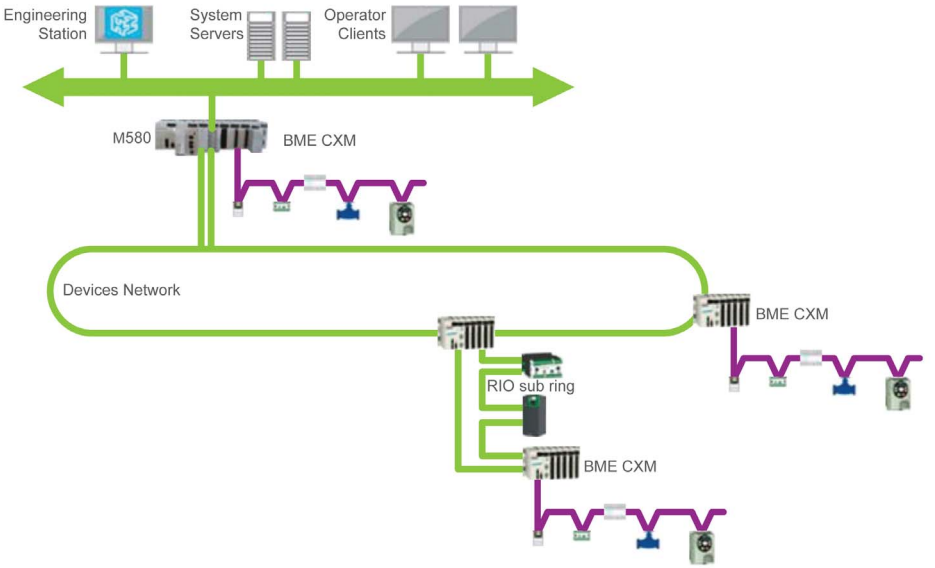
These profiles offer a total flexibility depending on the level of performance required by the process *(see page 43)*.

System Architecture

The BMECXM module can be plugged in any:

- M580 local rack
- Remote drop supporting the M580 Ethernet backplane with a drop end communicator BMECRA31210 module.
  NOTE: The minimum firmware version for the BMECRA31210 is V2.10.

You can use several CANopen X80 master modules on the M580:



Services

Operating modes are based on the FDT/DTM technology and are fully integrated in Unity Pro.

The BMECXM module can provide these services:

Configuration	
CANopen *(see page 53)*	The BMECXM module is natively included in Unity Pro **Hardware Catalog**. Use Unity Pro to select and configure the BMECXM module. **NOTE:** ● You can configure third-party CANopen slaves by using the Unity Pro Hardware Catalog Manager via EDS files. ● You cannot configure a CANopen slave using its own DTM.
Ethernet Services *(see page 95)*	Use Unity Pro DTM to configure the BMECXM module.
Communication	
Language objects *(see page 113)*	Use Unity Pro application to access CANopen slaves via device DDTs that are automatically created for each device (implicit exchanges through PDOs).
Programming *(see page 117)*	Use Unity Pro application to access CANopen slaves via SDO function blocks (explicit exchanges) and device DDT (implicit exchanges).
Diagnostics	
LED Indicators *(see page 130)*	Use LED indicators to check the status of the BMECXM module and its communications with the network.
Device DDT *(see page 135)*	Use the device DDT to diagnose the BMECXM module from the application.
SNMP *(see page 109)*	Use SNMP services to gain easy access from an SNMP network to: ● Diagnostic information for the BMECXM module ● Event notification for some services Configure this service with the BMECXM DTM.
DTM *(see page 139)*	Use Unity Pro DTM to view communication and status information of: ● the BMECXM module, ● the CPU, and ● the CANopen slaves.
Embedded Webpages *(see page 145)*	Use the web browser of your PC to access detailed diagnostics data of the BMECXM module.
Firmware upgrade	
Firmware upgrade *(see page 155)*	Use the Unity Loader tool to upgrade the firmware of the BMECXM module.

Cyber Security

The CANopen X80 master module is in line with the global policy of the Modicon M580 range in term of cyber security.

Access rights between Unity Pro and the BMECXM DTM are synchronized.

Webserver is in read-only.

In addition, you can restrict the BMECXM access to:
- FTP, HTTP, SNMP, NTP, and EIP services
- Authorized IP addresses

For more information, refer to *Modicon Controllers Platform, Cyber Security, Reference Manual*.

Module Limits

When you use a BMECXM module on the Modicon M580, observe these limits:
- Each BMECXM module can manage a maximum of 63 CANopen slaves.
- The size of the configuration file (.prm) of each BMECXM module is limited to 64 Kb maximum.
- Each BMECXM module on the Ethernet bus communicates with the CPU. They are in the same network. Transparency is thus provided from the CPU or any PC connected on the same subnet up to the BMECXM module.

NOTE: No communication is possible between two BMECXM modules.

Module Restrictions

These services are not supported:
- Multi-master on CANopen bus
- Complex manufacturer-specific data types
- Scanning from a BMENOC03•1
- Access to CAN layer 2
- Copy/paste a BMECXM

NOTE: The motion function blocs (MFB) are only supported when the BMECXM module is scanned by the RIO scanner.

NOTE: The motion function blocs (MFB) are not supported when the BMECXM module is configured on AUX tasks.

BMECXM0100H (Hardened) Module

Overview

Hardened equipment is the ruggedized version of standard equipment that can operate in extended temperature ranges and in dirty, or corrosive environments. It can also operate in potentially explosive atmosphere. There are hardened versions of several of the CPUs, backplanes, modules, and power supplies, as well as other components, in the M580 system.

Extended Temperature Considerations

The standard temperature range for M580 equipment is 0...60 °C (32...140 °F). Hardened equipment can operate at extended temperature range: -25...70 °C (-13...158 °F).

When used in the standard temperature range, hardened equipment has the same performance characteristics as the standard equipment. However, at the higher and lower ends of the extended temperature range (lower than 0 °C (32 °F) or higher than 60 °C (140 °F)), the hardened power supplies can have reduced power ratings that affect power calculations.

If hardened equipment is operated above or below the extended temperature limits, that is lower than -25 °C (-13 °F) or higher than 70 °C (158 °F), the equipment can operate abnormally.

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

Do not operate M580 equipment outside of its specified temperature range.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Operating in Harsh Environments

Hardened equipment has a conformal coating applied to its electronic boards. When associated with appropriate installation and maintenance, this treatment allows it to be more robust in harsh environments.

Conformal coating increases the isolation capability of the circuit boards and their resistance to:

- Condensation
- Dusty atmospheres (conducting foreign particles)
- Chemical corrosion, in sulphurous atmospheres (for example, in oil refineries or purification plants) or in atmospheres that contain halogens such as chlorine.

Chapter 2
BMECXM Module Installation and Replacement

Introduction

This chapter presents the instructions to install and replace BMECXM modules.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
BMECXM Module Installation	30
BMECXM Module Replacement	33

BMECXM Module Installation

Introduction

In a M580 architecture, you can mount the BMECXM module on a local rack or a remote drop.

The BMECXM module will only power up on a BMEXBP•••• Ethernet backplane.

NOTE: The minimum firmware version for the backplane is V1.0.

For more information on backplane, refer to *Modicon X80 Racks and Power Supplies, Hardware, Reference Manual*.

Selecting a Backplane

On the BMEXBP•••• backplanes, the BMECXM module can be mounted in any open slot, except the following restrictions:

- When mounted on a local rack, slots 0 and 1 are reserved for the M580 CPU.
- When mounted on a remote drop, slot 0 is reserved for the drop end communicator module.

In addition to the above restrictions on BMEXBP•••• backplanes, some open slots are not allowed for the BMECXM module when mounted on:
BMEXBP1002 backplane: slots 2 and 8 are not allowed.
BMEXBP1200 backplane: slots 2, 8, 10, and 11 are not allowed.

Grounding Considerations

The BMECXM module is equipped with ground connection contacts at the rear for grounding purposes. These contacts connect the grounding bus of the module to the grounding bus of the rack.

To ground the rack, refer to the chapter ***Grounding the Rack and Power Supply Module*** *(see Modicon X80 Racks and Power Supplies, Hardware, Reference Manual)*.

Installing the Module on the Rack

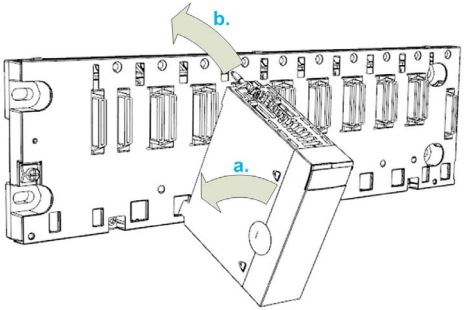
⚠⚠ DANGER

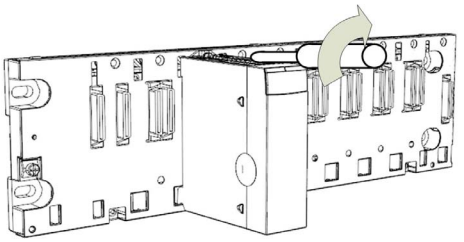
ELECTRICAL SHOCK HAZARD

- Switch off the power supply at both ends of the PAC connection, and lock out and tag out both the power sources.
- In case lock out and tag out are not available, ensure that the power sources cannot be inadvertently switched on.
- Use suitable insulation equipment when inserting or removing all or part of this equipment.

Failure to follow these instructions will result in death or serious injury.

To mount the module on the backplane, follow these steps:

Step	Action
1	Turn off the power supply to the rack.
2	Remove the protective cover from the module interface on the rack.
3	 **a.** Insert the locating pins on the bottom of the module into the corresponding slots in th rack. **b.** Use the locating pins as a hinge and pivot the module until it is flush with the rack. (The twin connector on the back of the module inserts the connectors on the rack).

Step	Action
4	Tighten the retaining screw to hold the module in place on the rack:  **NOTE:** The maximum tightening torque is 1.5 N•m (1.11 lb-ft).
5	Connect the CANopen cable to the CANopen connector of the BMECXM module. **NOTE:** For details on CANopen network, refer to *CANopen Hardware Setup Manual*.

A bad module connection can lead to an unexpected behavior of the system.

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

- Tighten the retaining screw of the module.
- Tighten the retaining screws of the SUB-D 9 CANopen connector.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

BMECXM Module Replacement

Overview

At any time and without turning off the power to the rack, any BMECXM module on the rack can be replaced by another module with compatible firmware.

⚠⚠ DANGER

EXPLOSION OR ELECTRIC SHOCK

- Only perform a hot swap operation in locations known and confirmed to be non-hazardous
- Use only your hands and suitable insulation equipment.
- Do not use any metal tools.

Failure to follow these instructions will result in death or serious injury.

⚠ WARNING

HAZARD OF ELECTRIC SHOCK

Before attempting to hot swap a BMECXM module:

- Positively confirm that the rack is connected to the protective earth ground.
- Positively confirm that you have an equipotential grounding system in place.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

The replacement module obtains its operating parameters over the Ethernet. Assuming that the FDR server is enabled *(see Modicon M580, Hardware, Reference Manual)*, the transfer occurs to the BMECXM module.

NOTE: You have to understand and plan for the consequences of hot-swapping a module. Disconnecting a module will interrupt communication to the connected CANopen slave devices.

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

- Thoroughly identify and understand all implications and consequences of operating mode modifications before attempting them with a new device.
- Take all necessary preventive measures to ensure safe conditions before making operating mode modifications.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Hot Swapping Procedure

To hot-swap the module, follow these steps:

Step	Action
1	Remove the CANopen cable from the module.
2	Remove the module from the backplane.
3	Install the new module to the free slot of backplane. Tighten the retaining screw to hold the module in place on the rack: **NOTE:** The maximum tightening torque is 1.5 N•m (1.11 lb-ft).
4	Reconnect the CANopen cable to the CANopen connector of the BMECXM module.

A bad module connection can lead to an unexpected behavior of the system.

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

- Tighten the retaining screw of the module.
- Tighten the retaining screws of the SUB-D 9 CANopen connector.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Replacing a CANopen Slave

NOTE: For CANopen slave device replacement procedure refer to respective device instructions.

After a CANopen slave device replacement, the BMECXM module sends the initial parameters automatically to the new device that restarts automatically. If you have changed the initial parameters, you need to retransfer them explicitly from the application.

Part II
CANopen Software Implementation

Subject of This Part

This part describes the various possibilities for software configuration, programming, and diagnostics in a CANopen application.

What Is in This Part?

This part contains the following chapters:

Chapter	Chapter Name	Page
3	Generalities	37
4	CANopen Configuration	53
5	Ethernet Services Configuration	95
6	Language Objects	113
7	Programming	117
8	Diagnostics	129
9	Firmware Upgrade	155

Chapter 3
Generalities

Introduction

This chapter describes CANopen software implementation principles on the Modicon M580 bus.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Implementation Presentation	38
Maximum Configuration	40
Device PDO Allocation	41
Performance	43
Operating Modes	45
Fallback Strategy	50

Implementation Presentation

Overview

To implement a CANopen bus, you have to define the physical context of the application in which the bus is integrated: rack, supply, processor, and modules. Then you have to ensure that the necessary software is implemented.

Implementation Principle

This table shows the different implementation phases:

Phase	Description	Mode
Configuration	Setting of configuration parameters.	Offline
Programming	Programming these specific functions: ● Implicit bit objects or associated words via the device DDT ● Explicit bit objects or associated words via read/write SDO	Offline or online
Transfer	Transferring the application to the PLC.	Online
Debugging and diagnostics	Debugging the application, controlling inputs/outputs, and accessing diagnostic messages with: ● LED indicators ● Device DDTs ● DTM diagnosis ● Embedded webpages	Online
Documentation	Printing the various information relating to the configuration of the CANopen master and devices.	Offline or online

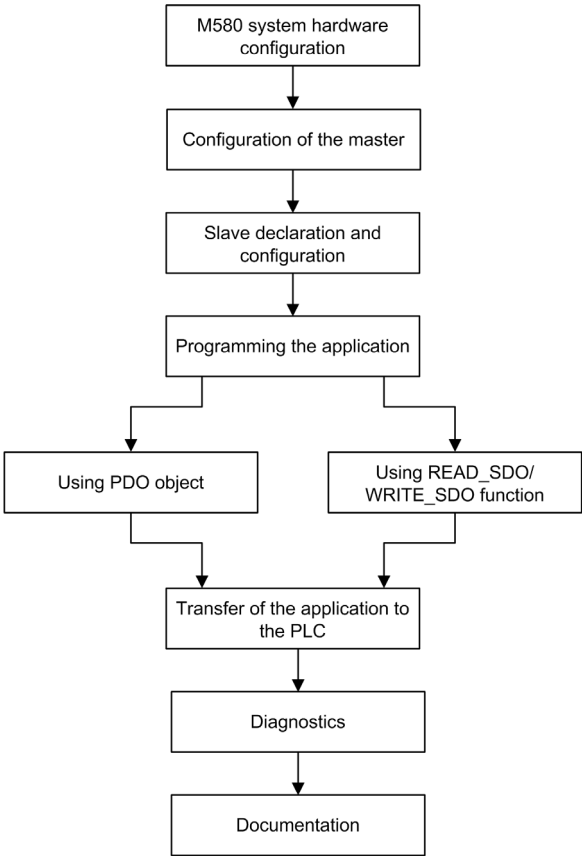
NOTE:

- For more information on modes (Offline or Online), refer to chapter Project management *(see Unity Pro, Operating Modes)*.
- The above order is given for your information. Unity Pro software enables you to use editors in the desired order of interactive manner.

NOTICE
DIAGNOSTIC DELAY Use diagnosis system information and monitor the response time of the communication. In case of disturbed communication, the response time can be 1 to 2s. **Failure to follow these instructions can result in equipment damage.**

Implementation Method

This flowchart shows the CANopen port implementation method for BMECXM modules:



Maximum Configuration

Overview

The maximum configuration is determined by:

- BMECXM limits
- M580 limits

BMECXM Limits

If one of these limits is reached, the maximum capacity of the configuration is reached. In that case, you can add another BMECXM module in the architecture.

This table shows the maximum configuration of the BMECXM module:

Parameter	Maximum value
Number of devices supported	63
Number of PDO	• 256 IN • 256 OUT **NOTE:** For more information, refer to device PDO Allocation *(see page 41)*.
Process image size	4 Kb IN and 4 Kb OUT
BMECXM configuration file (prm file on FDR server)	64 Kb

M580 Limits

This table shows how the maximum configuration can depend on other limits:

Parameter	Depends on
Number of Ethernet slots	Rack type
Number of racks	CPU
Scanner capacity	CPU type for: • Number of RIO and DIO equipment supported • IN and OUT memory capacity **NOTE:** All the resources are shared with all the other equipment that is configured on Ethernet I/O.

If these limits are reached, you can distribute the equipment between the RIO or DIO scanner of the CPU or add DIO scanners to the architecture.

Device PDO Allocation

Overview

Depending on your configuration, you can check if the maximum number of devices, Tx PDO, or Rx PDO is reached.

You can do the same calculation with third-party devices integrated via the Hardware Catalog Manager.

Maximum Number of PDOs

This table indicates the maximum number of PDOs used by each device present by default in the Unity Pro Catalog Manager:

Family	Device	Tx PDO	Rx PDO
Motor control	APP_1CC00	5	5
	APP_1CC02	5	5
	TeSysT_MMC_L	4	4
	TeSysT_MMC_L_EV40	4	4
	TeSysT_MMC_R	4	4
	TeSysT_MMC_R_EV40	4	4
	TeSysU_C_Ad	4	4
	TeSysU_C_Mu_L	4	4
	TeSysU_C_Mu_R	4	4
	TeSysU_Sc_Ad	4	4
	TeSysU_Sc_Mu_L	4	4
	TeSysU_Sc_Mu_R	4	4
	TeSysU_Sc_St	4	4
Sensors	OsiCoder	2	0

Family	Device	Tx PDO	Rx PDO
Distributed I/Os	FTB_1CN08E08CM0	2	2
	FTB_1CN08E08SP0	2	2
	FTB_1CN12E04SP0	2	2
	FTB_1CN16CM0	2	2
	FTB_1CN16CP0	2	2
	FTB_1CN16EM0	2	2
	FTB_1CN16EP0	2	2
	FTM_1CN10	5	5
	OTB_ISLAND	8	8
	OTB_1C0_DM9LP	8	8
	STB_NCO_1010	32	32
	STB_NCO_2212	32	32
Motion & Drive	ATV312_V5_1	2	2
	ATV31_V1_1	2	2
	ATV31_V1_2	2	2
	ATV31_V1_7	2	2
	ATV31T_V1_3	2	2
	ATV32_MFB	3	3
	ATV61_V1_1	3	3
	ATV71_V1_1	3	3
	IclA_IFA	1	1
	IclA_IFE	1	1
	IclA_IFS	1	1
	LXM05_MFB	4	4
	LXM05_V1_12	4	4
	LXM15LP_V1_45	4	4
	LXM15MH_V6_64	4	4
	SD3_28	4	4
Safety	XPSMC16ZC	4	0
	XPSMC32ZC	4	0
Third party products	CPV_C02	1	1
	CPX_FB14	4	4
	P2M2HBVC11600	1	1

Performance

Impact on Task Cycle Time

The impact of the PDO broadcasting on the task cycle time is as follows:

Task	Typical
CANopen inputs	xx µs/PDO
CANopen outputs	xx µs + xx µs/PDO
Diagnostics	xx µs

For more information, refer to the Modicon M580 performance characteristics *(see Modicon M580, Hardware, Reference Manual)*.

RPI Performance

According to your configuration, you can get a message from Unity Pro.

In that case, you can check that:

- The data amount and RPI are compatible with the performance objective of the BMECXM modules: 2000 packets/s for all the IN and OUT connections of the BMECXM modules.
- The RPI is compatible with the fieldbus period.

If not, the RPI can be increased by the DTM. If you need determinism, you can either reduce the CANopen configuration, increase the baudrate, or increase the task period.

For more information, refer to RPI values *(see page 105)*.

SDO Performance

There is one SDO at a time for each slave. Up to 63 SDOs can be managed in parallel according to the CPU capacity.

SDO are sent and received in synchronisation with the PLC Mast task. Therefore, the SDO response time is linked to the period of the MAST task, to the CANopen baudrate and to the response time of the slave.

Bus Start

The CANopen bus start time depends on the number of devices.

The minimum time to start a CANopen bus is 27 seconds.

The time to configure one device is about 0.8 second.

The start time of a CANopen bus with 63 devices is about 1 minute.

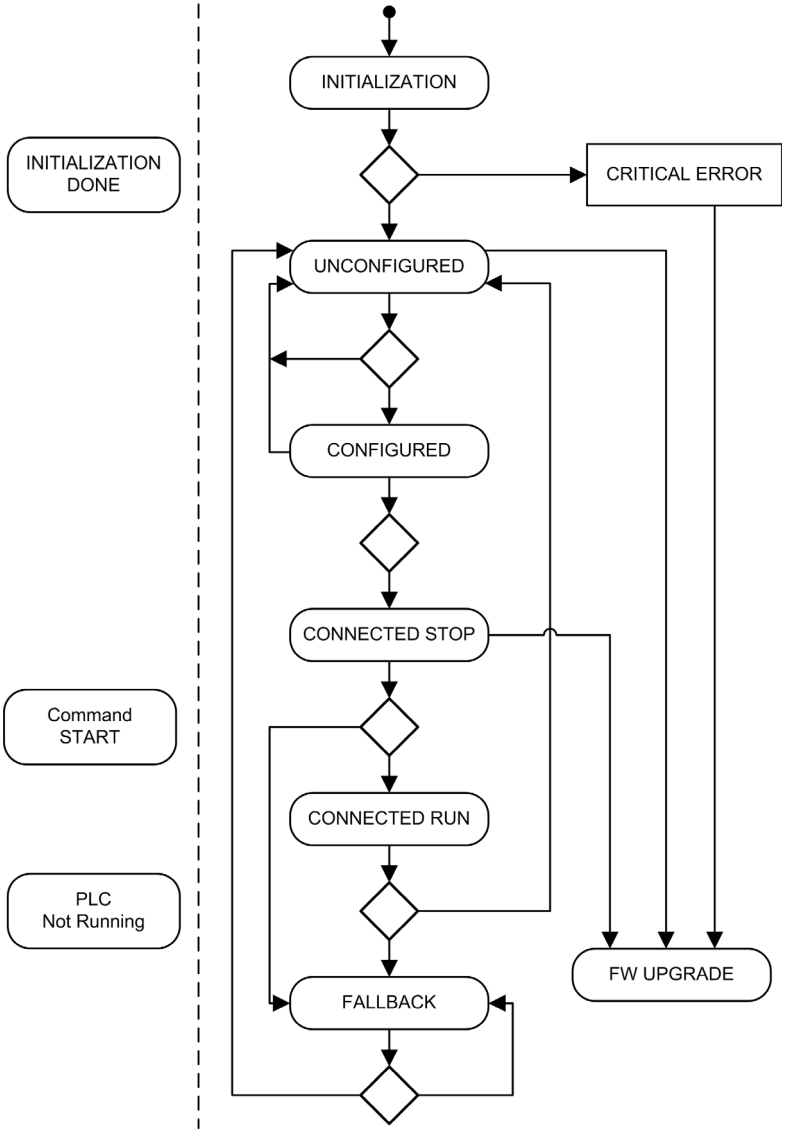
Device Disconnection

The time to detect the disconnection of a device depends on the protocol configuration *(see page 79)*:

Protocol	Description
Node Guarding	The time to detect the disconnection is **Guard Time * Life Time Factor**
Heartbeat	The time to detect the disconnection is **Node Heartbeat Producer Time + Node Heartbeat consumer Time**

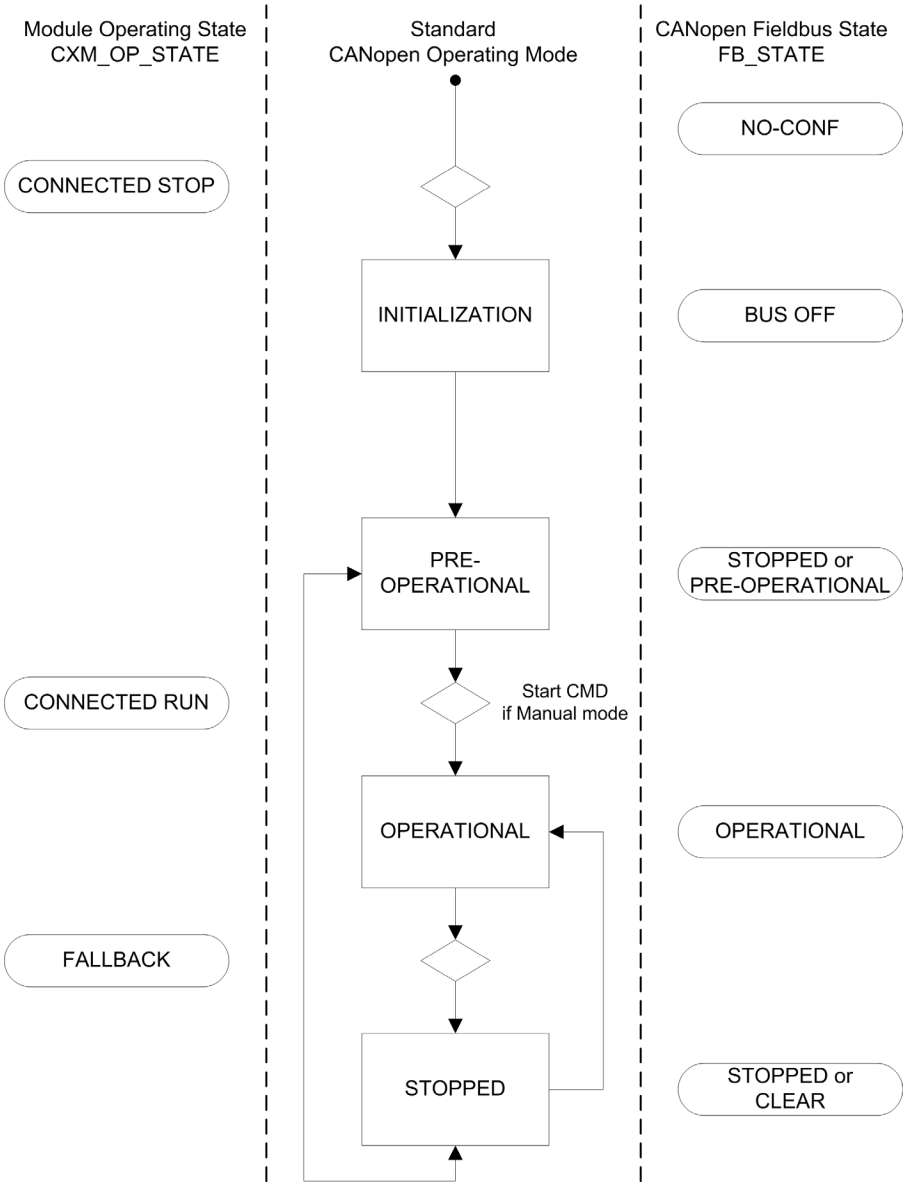
Operating Modes

BMECXM States



BMECXM State	Description
INITIALIZATION	The module enters in this mode on power on or following up a reboot. The power on self-test (POST) is performed during this phase. **NOTE:** When the power on self-test failed, the module switch to CRITICAL ERROR state. Refer to LEDs *(see page 132)* description to diagnose this state.
UNCONFIGURED	The power on self-test is completed and the module proceed to: ● Backplane initialization, ● Get IP address (from DHCP), ● Get configuration file (from FDR server).
CONFIGURED	The module has received its configuration file and is waiting for all expected EtherNet/IP connection with the PLC, as defined in the configuration file (*.prm* file). After 5 s, if not all expected connections are opened, the BMECXM module restarts.
CONNECTED STOP	The boot slave procedure is started to initialize all CANopen slave devices configured in the *.prm* file. The status of each device is progressively updated in the **Slave Live List** tab. **NOTE:** If the BMECXM module is configured in manual mode, the EM_start command is mandatory to reach the CONNECTED RUN state.
CONNECTED RUN	This state is reached when: ● all EtherNet/IP connection are connected and in RUN state. ● The CANopen fieldbus is in OPERATIONAL state. In CONNECTED RUN state, the BMECXM module exchanges I/O with the CANopen slave devices. From this state, the PLC can drive the BMECXM module state by sending NMT requests. The PLC can also send NMT request *(see page 118)* via explicit message to CANopen slave devices to drive their state individually.
FALLBACK	The BMECXM module switches in FALLBACK state and follow the Fallback startegy *(see page 50)*.
FW UPGRADE	The firmware upgrade is in progress. Only Unity Loader can communicate with the BMECXM module and EtherNet/IP connections with the PLC are stopped.

CANopen States



CANopen States	Description
INITIALIZATION	The BMECXM module executes the initialization of the CANopen bus according to the Boot-up procedure. It is done when the BMECXM module is in CONNECTED STOP state. During the CANopen bus initialization, only read accesses to the object dictionary of the CANopen master and slave devices are possible via the SDO command interface.
PRE-OPERATIONAL	The boot-up sequence is now completed. In this state, no command has been received to enter in OPERATIONAL state. The CANopen slave devices respond to SDO and NMT messages but not to PDOs.
OPERATIONAL	This is the main state of the CANopen bus. In this state: ● The error control service is active. ● The detection of the CANopen slave devices is started, according to the assignment in the configuration file. ● Emergency and boot-up messages are received. **NOTE:** If the BMECXM module is configured in manual mode, the EM_start command is mandatory to reach the OPERATIONAL state.
STOPPED	In this state, only the slave monitoring is active. No service is available to read or write SDO.

EtherNet/IP Connections States

BMECXM Operating State	Fieldbus Operating State	EtherNet/IP State
IDLE	IDLE	There is no connection
UNCONFIGURED	NO-CONF	
CONFIGURED	BUS OFF	
CONNECTED STOP	PRE OPERATIONAL	The connection is running
CONNECTED RUN	● OPERATIONAL(1) ● STOPPED(1) ● PRE OPERATIONAL(1)	
FALLBACK	Fallback strategy: ● OPERATIONAL(2) ● STOPPED(3) ● CLEAR(4)	Depending on the I/O scanner type: ● STOP ● IDLE ● CLOSE
FW UPGRADE	Not applicable	There is no connection. The connection is aborted if existing.

(1) Switching form one state to an other one is done via an NMT command.
(2) The CANopen fieldbus state stay in OPERATIONAL but with the last data received from the PLC.
(3) CANopen master and slaves switch to STOPPED state.
(4) The CANopen fieldbus state stay in OPERATIONAL but with data set to zero.

Communication Objects

The following table presents which communication objects are allowed depending on the CANopen states:

Object	INITIALIZATION	PRE-OPERATIONAL	STOPPED	OPERATIONAL
PDO	–	–	–	Yes
SDO	–	Yes	–	Yes
SYNC	–	Yes	–	Yes
EMCY	–	Yes	–	Yes
Bootup	Yes	–	–	–
NMT	–	Yes	Yes	Yes
– Communication object not allowed.				

Fallback Strategy

Overview

The CANopen X80 master module switches in FALLBACK state:

- as soon as it detects that the PLC is stopped, or
- after the holdup time, if at least one EtherNet/IP connection of the PLC is closed.

The following table gives an overview of the module behavior following a PLC STOP:

		RIO Scanner	DIO Scanner		
		BMEP58•040	BMEP58•040	BMEP58•020	
		Default configuration	Default configuration	Default configuration	User configuration
EtherNet/IP	Connection(1)	Open	Closed	Open	Closed
	Run/Idle flag	Run	–	Idle	–
Data (I/O Exchange)		Hold previous	No exchange. Device DDT is cleared	Hold previous	No exchange. Device DDT is cleared
(1) CIP connection between the CPU and the CANopen X80 master module.					

Output Fallback Strategy

In FALLBACK state the module applies the fallback strategy configured in the **Ethernet IO** tab of the DTM:

- **Operational, outputs maintained**: The CANopen fieldbus stays in OPERATIONAL state and slaves outputs are maintained with the last values.
- **Operational, outputs set to 0**: The CANopen fieldbus stays in OPERATIONAL state and slaves outputs are set to 0 (zero).
- **Stop**: The CANopen fieldbus switches to STOPPED state and all devices on the network switched to STOPPED state (broadcast NMT).

Input Fallback Strategy

In case of EtherNet/IP connection is lost between the PLC and the BMECXM module, all inputs values of the device DDT are set to 0 (zero), including the Health bits.

In case of PLC stopped event, the behavior of inputs differs according to the EtherNet/IP connection state:

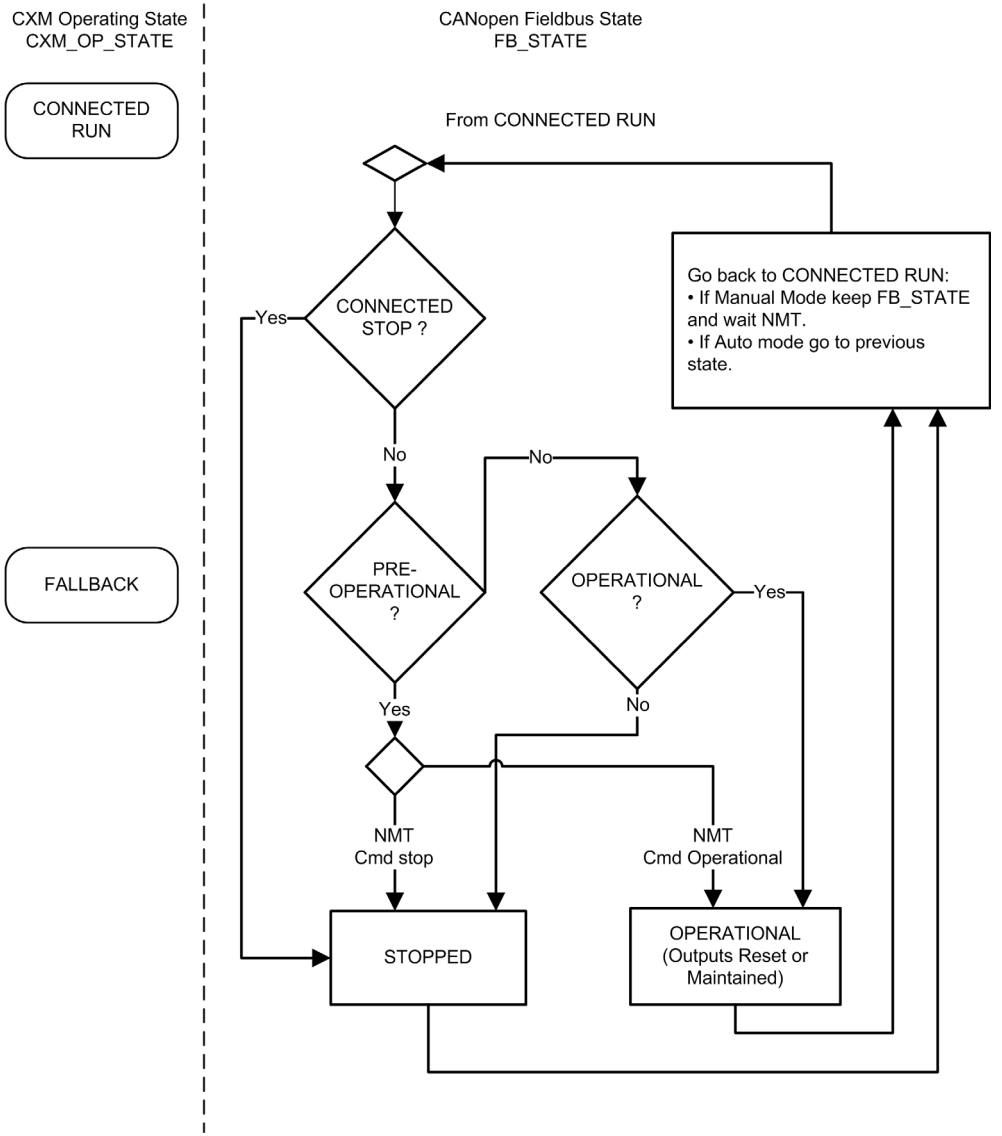
EtherNet/IP connections are closed: input data values are set to 0 (zero).

EtherNet/IP connections are not closed, Run/Idle flag to Idle: input data are refreshed

EtherNet/IP connections are not closed, Run/Idle flag to Run, output data contains the CPU state (STOP): input data are refreshed

States in Fallback Mode Strategy

The following diagram gives the CANopen fieldbus states in fallback mode:



Chapter 4
CANopen Configuration

Introduction

This chapter presents the CANopen configuration. It shows how to use Unity Pro programming software to select and configure BMECXM modules and CANopen slave devices.

What Is in This Chapter?

This chapter contains the following sections:

Section	Topic	Page
4.1	Overview	54
4.2	Adding a CANopen X80 Master BMECXM module	56
4.3	Bus Configuration	58
4.4	Device Configuration	68
4.5	Master Configuration	87

Section 4.1
Overview

Overview

Introduction

The CANopen configuration consists in the configuration of the CANopen fieldbus and of the bus master and slaves.

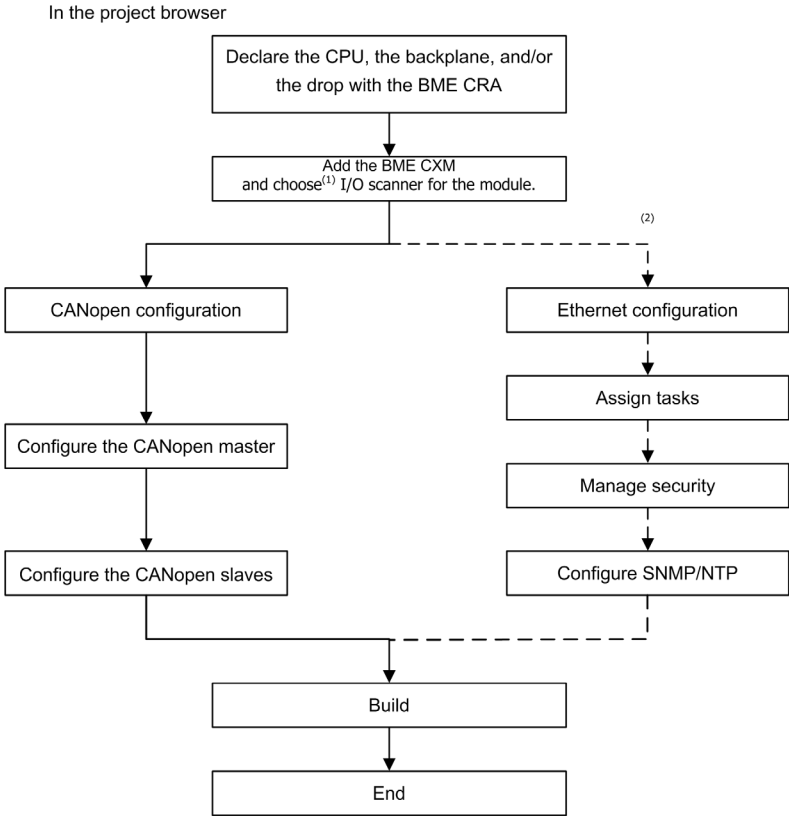
The configuration of CANopen architecture is integrated into Unity Pro.

In the same M580 PAC, you can configure several CANopen master BMECXM modules with associated CANopen slave devices.

⚠ CAUTION
UNEXPECTED EQUIPMENT BEHAVIOR ● Configure each CANopen slave device on the correct CANopen master BMECXM module. ● Always verify that the Unity Pro configuration is consistent with the hardware installation. **Failure to follow these instructions can result in injury or equipment damage.**

Configuration Steps

This chart represents the steps for configuration of the CANopen architecture:



(1) After validation of module insertion in the project, if you want to change the scanner association, you have to delete the device from the configuration and then recreate the device with the new scanner association.

(2) Optional steps.

NOTE: For more information on optional steps, refer to chapter Ethernet Services Configuration *(see page 95)*.

Section 4.2
Adding a CANopen X80 Master BMECXM module

Adding a CANopen X80 Master BMECXM Module

Prerequisite

Before adding the module you have to declare the M580 CPU and if required, the remote drop(s).

Procedure

To add a BMECXM module to the Unity Pro project, follow these steps:

Step	Action
1	From the **Project Browser**, expand (+) the **Configuration** directory.
2	Depending on your hardware architecture, double-click: ● The **PLC bus** subdirectory for a local rack ● The **EIO Bus** subdirectory for a remote drop
3	Right-click the open slot in the rack and click **New Device...**. **Result**: The **New Device** window is displayed. Expand (+) **Communication** to select the BMECXM module and click **OK**. **NOTE:** You can also click **Tools → Hardware Catalog → Modicon M580 local drop → Communication** and drag the BMECXM module to an open slot in the rack.
4	In the popup window, select the appropriate combination of **I/O-scanner**, **Protocol**, and **Profile** and click **OK**. **Result**: The **Properties of device** window is displayed. **NOTE:** All the tabs contain read-only information except the **General** tab.
5	In the **General** tab, you can change the name(1) of the module in the **Name** field. In that case, the fields in the **Default I/O vision management** box are automatically modified to match the new name. By default, the prefix of the name corresponds to the topological address of the module in the configuration. For a BMECXM module on bus EIO, on drop 3, on rack 1, and slot 4, the default name will be `EIO2_d3_r1_s4_ECXM0100`. **NOTE:** The default naming rule in Unity Pro prevent from getting modules of the same type mixed up.
(1) In Unity Pro, this name is also used as: ● The module name in the **DTM Browser** under the **Host PC** ● The device DDT name	

Step	Action
6	Click **OK**. **Result**: The new device is added.
7	Save the project by clicking **File** → **Save**.
(1) In Unity Pro, this name is also used as: ● The module name in the **DTM Browser** under the **Host PC** ● The device DDT name	

⚠ WARNING

UNEXPECTED EQUIPMENT BEHAVIOR

- Always verify that the Unity Pro configuration is consistent with the hardware installation.
- If you attempt to change the default names of your devices, manage the naming to prevent from addressing the wrong device.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Unity Pro Commands for the Module

In the Unity Pro bus editor (local or remote), right-click the BMECXM module to access these commands:

Name	Description
Delete Module(1)	Delete the selected module from the rack.
Open Module(1)	See a description of the selected communications module.
Move Module(1)	Move the selected module to the rack slot that you designate.
Go to DTM	Show the DTM of the selected module in the **DTM Browser**.
Power Supply Budget...(2)	Access the **Power supply** tab and view: ● The total power ● The power discharged in the module for each voltage it uses **NOTE:** Close this window before performing any command in Unity Pro.
(1) This command also appears in the **Edit** menu. (2) This command also appears in the **Services** menu.	

Section 4.3
Bus Configuration

Subject of this Section

This section presents the configuration of the CANopen bus.

What Is in This Section?

This section contains the following topics:

Topic	Page
Access the CANopen Bus Editor	59
Adding Slave Devices on the CANopen Bus	61
Delete/Move/Duplicate a Device on the CANopen Bus	65
View CANopen Bus in the Project Browser	67

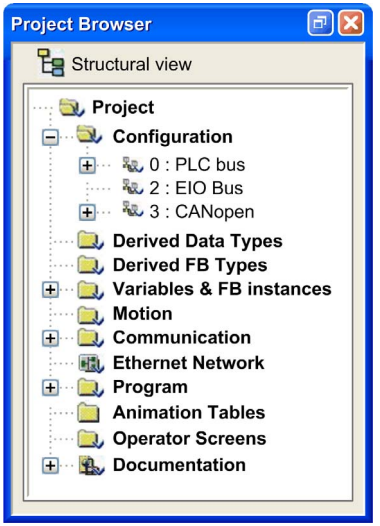
Access the CANopen Bus Editor

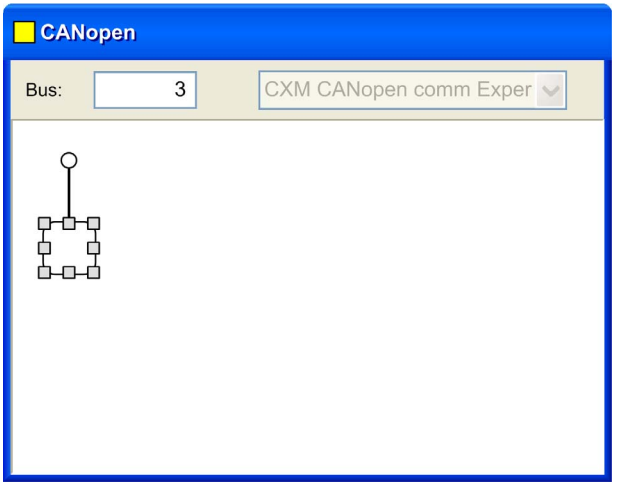
Overview

As soon as the BMECXM module is configured in the Unity Pro project, a node is automatically created in the **Project Browser**.

Procedure

To access the CANopen bus editor, follow these steps:

Step	Action
1	From the **Project Browser**, expand (+) the **Configuration** directory: 

Step	Action
2	Double-click the CANopen directory. **Result**: The **CANopen** window is displayed:  **NOTE:** You can also select the CANopen subdirectory and click **Open** in the contextual menu.

Adding Slave Devices on the CANopen Bus

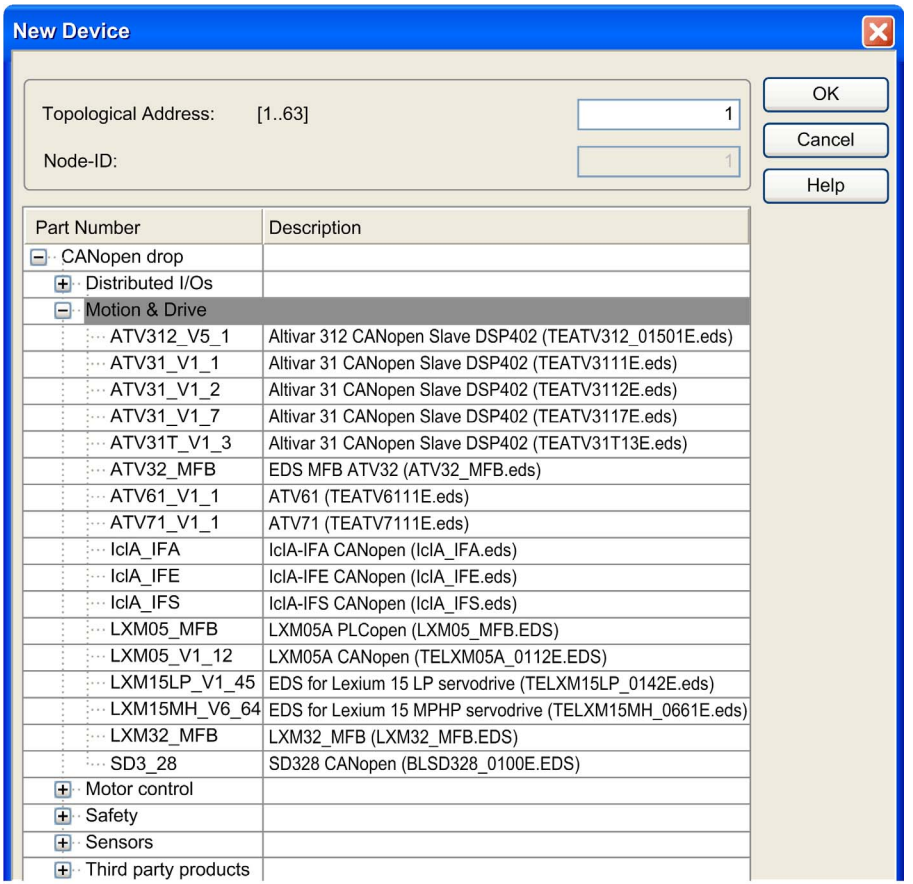
Overview

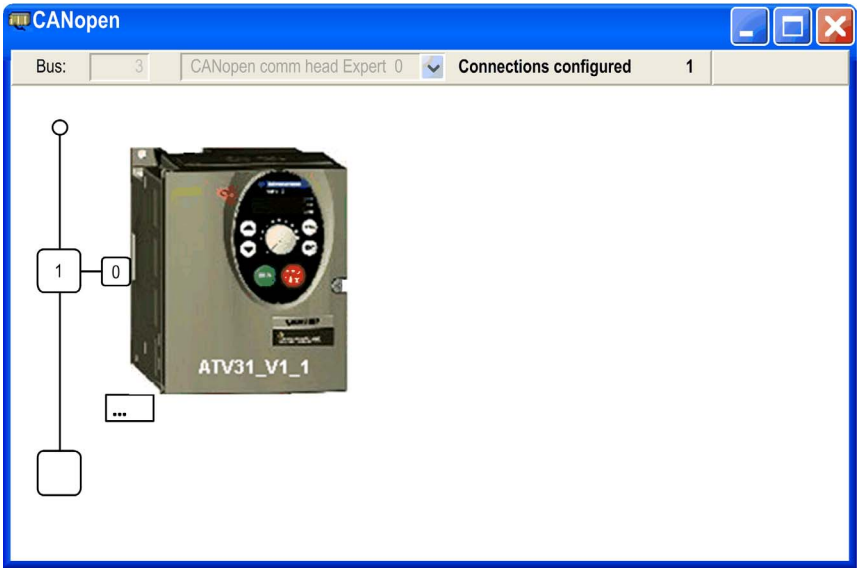
You can launch the bus editor from this node to define the topology of the bus and configure the CANopen elements.

NOTE: You cannot modify the configuration of the CANopen bus in connected mode.

Procedure

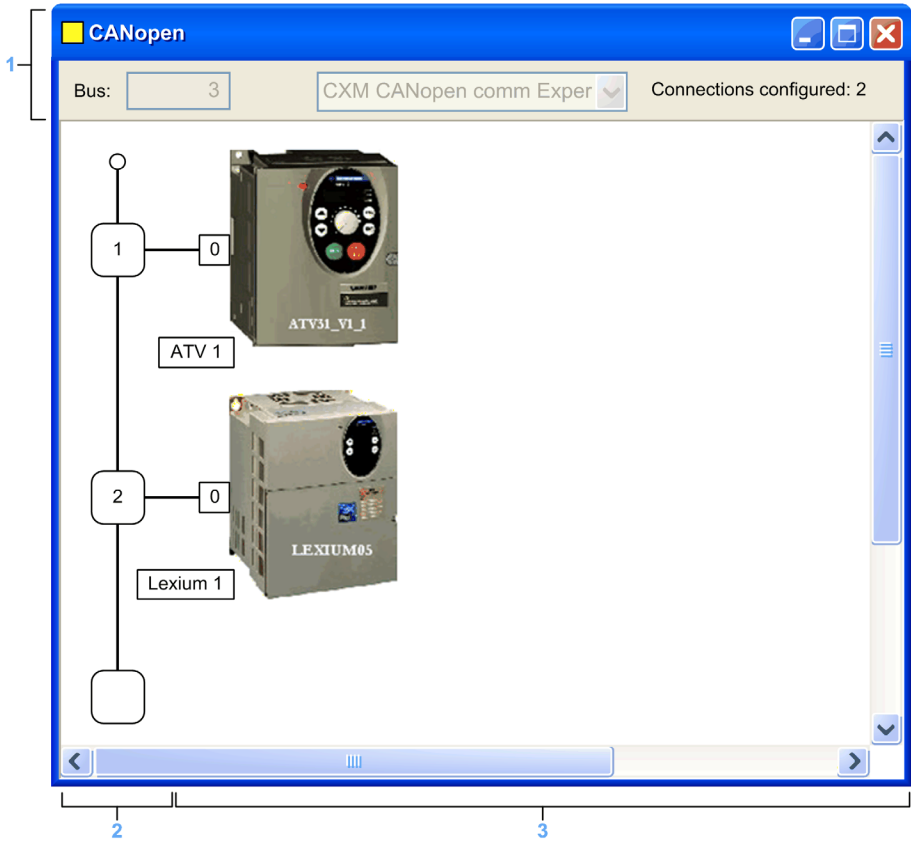
To add a slave device follow these steps:

Step	Action
1	Access the CANopen bus editor *(see page 59)*.
2	Double-click the connection point, where the module should be connected. **Result**: the **New Device** window is displayed. 
3	In the **Topological Address** field, type the number of the connection point corresponding to the address. **NOTE:** By default, the Unity Pro software offers the first free consecutive address.
4	From the **CANopen drop**, expand (+) the family to select your CANopen device.

Step	Action
5	Click **OK**. **Result**: the module is declared. 

CANopen Bus Editor

The following graphic describes the different areas of the **CANopen** bus editor:



Number	Element	Function
1	Bus	Bus number
	Connections configured	Indicates the number of connection points configured.
2	Logical address area	Includes the addresses of the devices connected to the bus.
3	Module area	Includes the devices that are connected to the bus.

NOTE: Available connection points are indicated by an empty white square.

Delete/Move/Duplicate a Device on the CANopen Bus

Deleting a Device

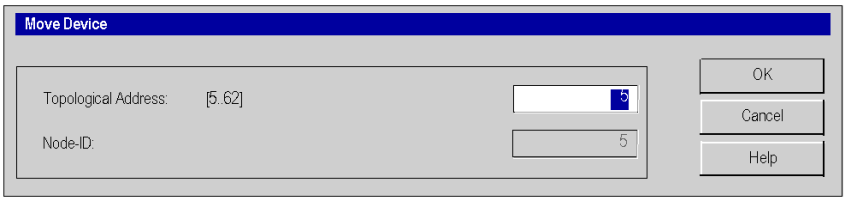
To delete a device follow these steps:

Step	Action
1	Access the CANopen bus editor *(see page 59)*.
2	Right-click the connection point of the device to be deleted and click **Delete Drop**.

Moving a Device

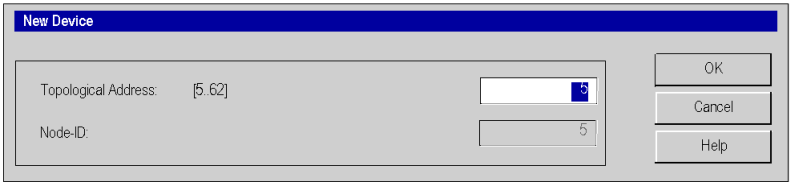
Moving a device does not involve a physical move on the bus, but rather a change in the device topological address. The name of the device DDT instance change except if you have manually modified the default name of your device in the CANopen X80 master module DTM *(see page 101)*.

To move a device, follow these steps:

Step	Action
1	Access the CANopen bus editor *(see page 59)*.
2	Select the connection point to be moved: a frame surrounds the selected connection point.
3	Drag and drop the connection point to be moved to an empty connection point. **Result**: the **Move Device** window is displayed: 
4	Type the number of the destination connection point.
5	Click **OK** to confirm the new connection point.

Duplicating a Device

To duplicate a device, follow these steps:

Step	Action
1	Access the CANopen bus editor *(see page 59)*.
2	Right-click the device to be copied and click **Copy**.
3	Right-click the destination connection point and click **Paste**. **Result**: the **New Device** window is displayed: 
4	Type the number of the destination connection point.
5	Click **OK** to confirm the new connection point.

View CANopen Bus in the Project Browser

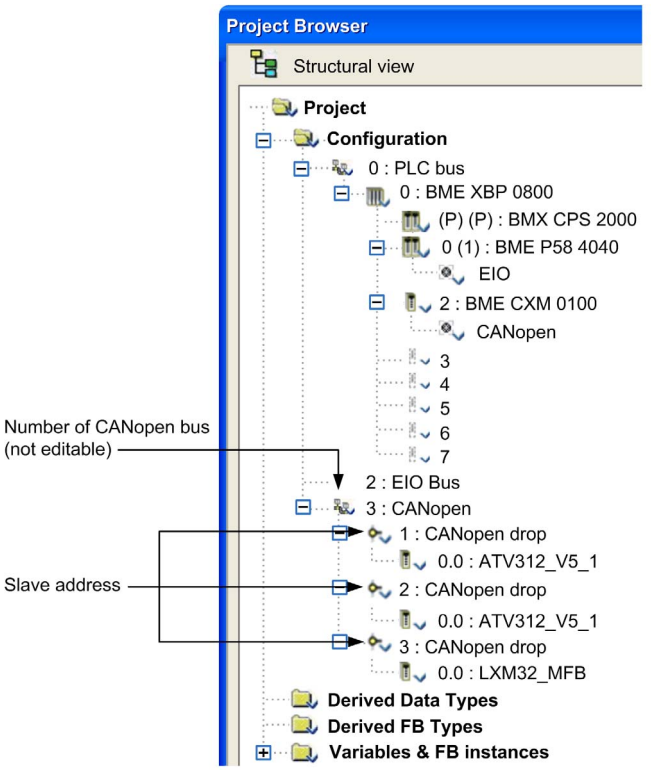
Overview

The CANopen bus is shown in the configuration directory of the **Project Browser**. The bus number is automatically calculated by Unity Pro.

NOTE: The value of the bus number cannot be modified.

Illustration

This illustration shows the CANopen bus and slaves in the **Project Browser**:



NOTE: If you right-click **3 : CANopen drop → Go to Bus Master**, the BMECXM module node that corresponds to the bus is automatically selected.

Section 4.4
Device Configuration

Subject of this Section

This section presents the configuration of the initial parameters of the CANopen devices.

What Is in This Section?

This section contains the following topics:

Topic	Page
Presentation of CANopen Devices	69
Slave Functions	70
Configuration Using Unity Pro	74
Configuration Using an External Tool	85

Presentation of CANopen Devices

Overview

Devices that you can connect to a CANopen bus and that can be configured in Unity Pro are grouped according to their family:

- **Distributed I/Os**
- **Motion & Drive**
- **Motor control**
- **Safety**
- **Sensors**
- **Third-party products**

CANopen Device Import

Only devices from the **Hardware Catalog** can be used with Unity Pro.

To add a CANopen device in the **Hardware Catalog**, you have to:

- Import it into the Hardware Catalog Manager *(see Hardware Catalog Manager, Software, Operation Guide)*.
- Update the **Hardware Catalog** of Unity Pro.

NOTE: The **Hardware Catalog** in Unity Pro, is an overview in read mode only of the Hardware Catalog Manager.

CANopen Catalog Compatibility Rules

The following elements in the catalog are supported on Modicon M580:

- All existing Schneider Electric CANopen devices
- Modules created with Unity Pro ≥ V11.1
- Modules created with Unity Pro < V11.1
- Specific functions created on old device with Unity Pro < V11.1
- Specific functions created on existing Schneider Electric devices with Unity Pro < V11.1

The **Hardware Catalog** can contain devices created with Unity Pro < V11.1. To update the catalog, you have to:

- Import the *cpx file created with the previous Unity Pro version in the Hardware Catalog Manager
- Open a *.sta file created with the previous Unity Pro version.

CANopen Device Configuration

To configure the initial parameters of CANopen devices, you can use:

- Unity Pro
- An external tool

Configuration depends on the type of CANopen devices.

NOTE: Before configuring a device, select the function when available *(see page 70)*.

Slave Functions

Overview

To facilitate their configuration, some CANopen devices are represented through functions.

Each function defines:

- Premapped PDOs
- Debugging variables that can be mapped. For more information, refer to the **PDO** tab *(see page 77)*.

NOTE: The function should be selected before configuring the slave.

Available Functions

The available functions are as follows:

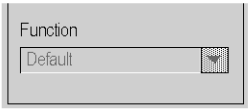
Devices Involved	Function	Description
Altivar	Basic	This function allows a simple control of the speed.
	MFB	This function allows control of the device through PLCOpen Motion function block library.
	Standard	This function allows control of the speed and/or torque. All the parameters that can be mapped are mapped in the supplemental PDOs for: ● An adjustment of the operating parameters (length of acceleration,...), ● Additional surveillance (current value,...), ● Additional control (PID, outputs command,...).
	Advanced	This function allows control of the speed and/or torque. Some parameters can be configured and can also be mapped in the PDOs to allow: ● An adjustment of the operating parameters (length of acceleration,...), ● Additional surveillance (current value,...), ● Additional control (PID, outputs command,...).

Devices Involved	Function	Description
STB NCO1010 & NCO2212	Simple	Use this profile if the island does not contain high-resolution analog I/O module or the TeSys U STB modules. This profile contains: ● NIM diagnostic information (index 4000-index 4006), ● 8-bit discrete input information (index 6000), ● 16-bit discrete information (index 6100), ● 8-bit discrete output information (index 6200), ● 16-bit discrete output information (index 6300), ● Low-resolution analog input information (index 6401), ● Low-resolution analog output information (index 6411). This profile limits the number of index or subindex entries for any of the above objects to 32. If the island configuration exceeds this limitation, use the large profile.
	Extended	Use this profile if the island contains high-resolution analog I/O module or the TeSys U STB modules. This profile contains: ● NIM diagnostic information (index 4000-index 4006), ● 8-bit discrete input information (index 6000), ● 16-bit discrete information (index 6100), ● 8-bit discrete output information (index 6200), ● 16-bit discrete output information (index 6300), ● Low-resolution analog input information (index 6401), ● Low-resolution analog output information (index 6411), ● High-resolution analog input information or HMI words (index 2200-221F), ● High-resolution analog output information or HMI words (index 3200-321F), ● TeSys U input information (index 2600-261F), ● TeSys U output information (index 3600-361F). This profile limits the number of index or subindex entries for any of the above objects to 32. If the island configuration exceeds this limitation, use the large profile.

Devices Involved	Function	Description
STB NCO2212	Advanced	Use this profile if the island contains enhanced CANopen devices, special features as run-time parameters along with high-resolution analog I/O module or HMI or the TeSys U STB modules. This profile contains: ● NIM diagnostic information (index 4000-index 4006), ● 8-bit discrete input information (index 6000), ● 16-bit discrete information (index 6100), ● 8-bit discrete output information (index 6200), ● 16-bit discrete output information (index 6300), ● Low-resolution analog input information (index 6401), ● Low-resolution analog output information (index 6411), ● High-resolution analog input information or HMI words (index 2200-221F), ● High-resolution analog output information or HMI words (index 3200-321F), ● TeSys U input information (index 2600-261F), ● TeSys U output information (index 3600-361F), ● Third-party CANopen devices (index 2000-201F), ● RTP information (index 4100 & index 4101). This profile limits the number of index or subindex entries for any of the above objects to 32. If the island configuration exceeds this limitation, use the large profile.
	Large	Use this profile if the island configuration does not fit any of the above profiles. This profile contains all the objects available for the STB island and hence consumes more memory address location in the CANopen master.
Altivar 61/71	Controller	This function is especially created for CANopen communications with the built-in controller card and all the application cards (pump control,...).
Festo CPV	Advanced	The advanced level is designed to configure the maximum I/Os and the complete parameters set.
	Basic	The basic level is designed to configure the valve terminal without CP extension.
	CP_Extension	This level is designed to configure I/Os including the CP extension.
Festo CPX	Basic_DIO_only	The basic level is designed to configure the CPX with pneumatic valves and digital I/O only.
	Generic_DIO_AIO	The generic DS401 level is designed to configure CPX valves and I/Os, including analog I/O modules.
	Advanced	The advanced level is designed to configure the maximum I/Os and the complete parameters set.

Devices Involved	Function	Description
All slaves except ATV and LXM05/32 _MFB and LXM15LP _V1_45	Default	This feature is the default function for certain devices. It may not be modified.

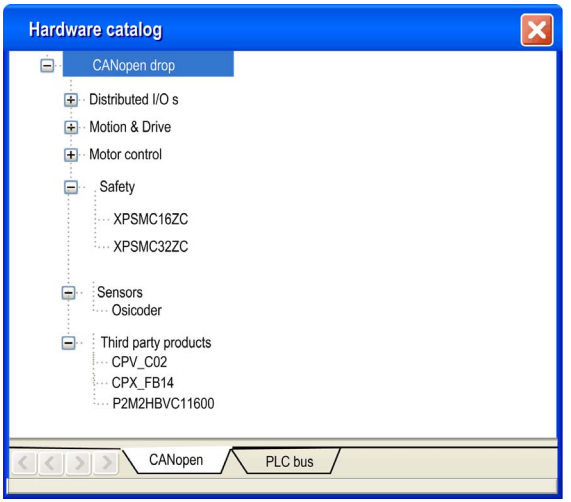
NOTE: Some devices can only handle one function. In this case, the function appears grayed out and cannot be modified.



Configuration Using Unity Pro

Overview

Devices that can be configured using Unity Pro are shown in the **Hardware Catalog**:



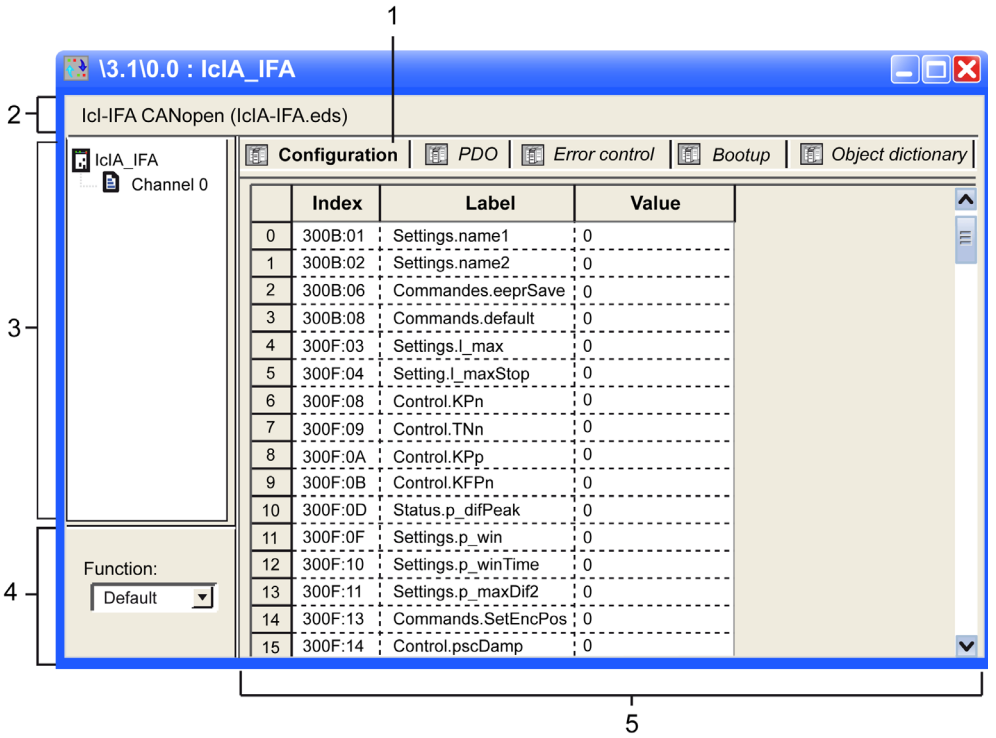
Procedure

To configure a slave, follow these steps:

Step	Action
1	Access the CANopen bus configuration screen *(see page 59)*.
2	Double-click the slave to be configured.
3	Configure the usage function using the **Configuration** tab.
4	Configure the PDOs using the **PDO** tab.
5	Select the control of error detected using the **Error control** tab.
6	Configure the bootup procedure using the **Bootup** tab.
7	Integrate a third-party product using the **Object Dictionary** tab.

Configuration Tab

This figure shows an example of the configuration window of a slave:



This table shows the elements of the configuration window and their functions:

Number	Element	Function
1	Tabs	Indicates the type of window displayed. In this case, it is the configuration window.
2	Module	Indicates the device shortened name.
3	CANopen communication	Allows you to select the device and display the **Overview** tab. It gives the characteristics of the device.
		Allows you to select the channel and display these tabs: ● **Configuration** ● **PDO** (input/output objects) ● **Error control** ● **Bootup** ● **Object dictionary**
4	General parameters	Allows you to select the slave functions *(see page 70)*.
5	Configuration	Allows you to set up the channels of the devices. **NOTE:** Some devices can be configured with an external tool. In this case, the configuration is stored in the device and you cannot enter configuration parameters because this field is empty.

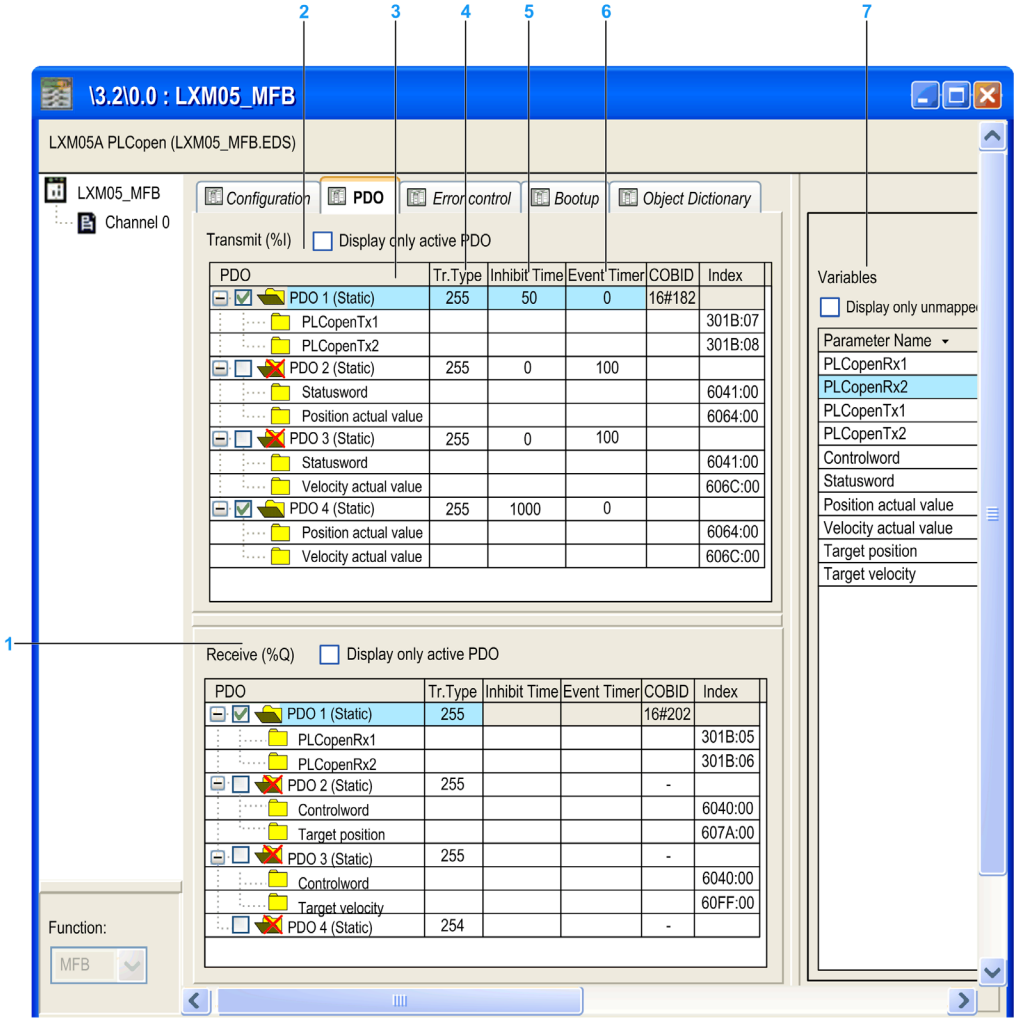
NOTE: All parameters are not sent when the device takes its configuration. The CPU sends only parameters that are different from the default values.

For more information on general, configuration, adjustment, and debugging parameters, refer to the documentation of each device.

PDO Tab

PDOs make it possible to manage the communication flow between the CANopen master and the slaves. A PDO can be enabled or disabled. The **PDO** tab allows you to configure a PDO.

This figure shows an example of the **PDO** tab:



This table shows the elements of the **PDO** tab and their functions:

Number	Element	Function
1	**Receive (%Q)**	Information received by the slave from the master.
2	**Transmit (%I)**	Information transmitted by the slave to the master.
3	**PDO**	According to the EDS file, some PDOs are already mapped. Otherwise, variables can be mapped to the PDOs.
4	**Tr.Type**	The transmission type can be: ● **Synchronous acyclic (0)**: the message is transmitted synchronously with the SYNC message but not periodically according to the value. ● **Synchronous cyclic (1-240)**: the PDO is transmitted synchronously and cyclically. This value indicates the number of SYNC messages between 2 PDO transmissions. ● **Asynchronous (Manuf. Event) (254)**: the PDO is transmitted asynchronously. It depends on the implementation in the device. Used for digital I/O. ● **Asynchronous (Profile Event) (255)**: the PDO is transmitted asynchronously when the value changes. **NOTE:** Verify that the configured transmission type is supported by the selected device.
5	**InhibitTime**	Mask the communication during this time.
6	**Event Timer**	Time to manage an event in order to start a PDO.
7	**Variables**	Variables can be mapped to the PDOs. To assign a variable to a PDO, drag and drop the variable into the desired PDO. A variable cannot be assigned with a static PDO.

NOTE: Double-click the element to edit the value of parameters.

To configure the STB NCO 1010, it is necessary to determine all the objects that are valid for this device and to configure them manually in the PDOs. For more information about the list of the associated objects, refer to the *STB user manual*.

PDO Multi-Mapping

The BMECXM master allows the PDO multi-mapping. It is possible to configure the same CANopen object in two different PDOs:

If an object is mapped...	Then ...
In more than one RPDO of a node	It is not supported.
In an RPDO and in a TPDO of a node	It is represented by two network variables in both process images.
In more than one TPDO of a node	Each instance is linked to the same network variable.

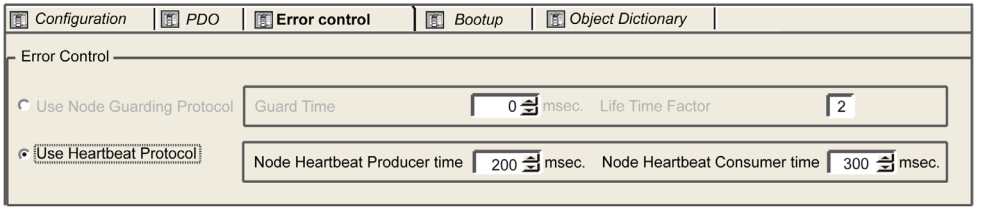
The PDO configuration is checked at build time. If there is an error detected:

- A log is displayed in the **Rebuild All Project** window at the bottom of the screen
- A dialog box with an error detected message appears when validating

Error Control Tab

Some CANopen slave devices only support either Heartbeat or Node Guarding. But for devices that support both Heartbeat and Node Guarding protocols, the only choice in Unity Pro is the Heartbeat mechanism.

The **Error control** tab for CANopen slave modules allows you to configure monitoring:



If the **Node Heartbeat Producer time** value or **Guard Time** value is set to 0 (zero), then according to the CANopen expected behavior, a CANopen slave device disconnected or not present in the configuration will not be diagnosed.

⚠ WARNING

UNEXPECTED DIAGNOSTIC BEHAVIOR

- Do not set the **Guard Time** value to 0 (zero), when using the Node Guarding protocol.
- Do not set the **Node Heartbeat Producer time** value to 0 (zero), when using the Heartbeat protocol.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

This table shows the elements of the **Error control** tab and their functions:

Protocol Used	Function
Node Guarding	Monitoring of network nodes: • **Guard Time**: period when the NMT (Network Management) master sends an RTR (Remote Transmission Request) at regular intervals **NOTE:** A value set to 0 deactivates the monitoring of the node. • **Life Time Factor** (read-only): 2. The concerned node answers in a given lapse time defined as below: lifetime = **Guard Time** * **Life Time Factor** **NOTE:** If there is no connection monitoring during the lifetime interval, the CANopen slave device signals an error.
Heartbeat	Mechanism that consists on sending cyclical presence messages generated by a heartbeat producer (CANopen slave) and the Heartbeat consumer (BMECXM) surveys the Heartbeat message reception. • **Node Heartbeat Producer time**: value corresponding to the sending time **NOTE:** A value set to 0 deactivates monitoring by a consumer. • **Node Heartbeat Consumer time** (read-only): value corresponding to the reception time (default value set to 300 ms and cannot be modified). **NOTE:** • The values for the consumer must not be less than the values for the producer. By default, **Node Heartbeat Consumer time** = 1.5 * **Node Heartbeat Producer time**. • If the BMECXM does not receive a signal within the time set for **Node Heartbeat Consumer time**, it generates a heartbeat event.

Bootup Tab

⚠ WARNING

UNEXPECTED EQUIPMENT OPERATION

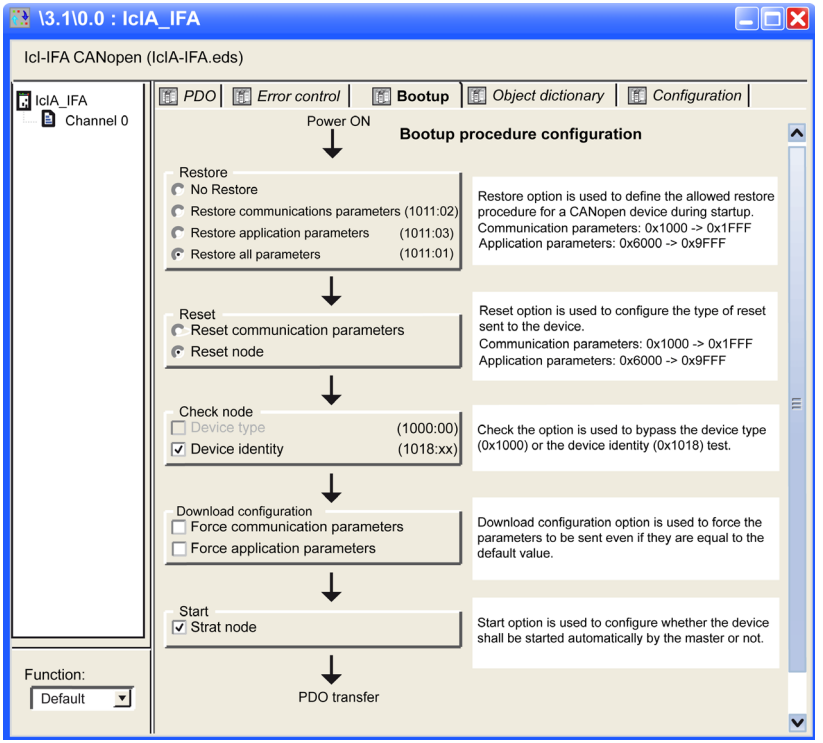
Manually verify all deactivated standard checks on the device before operating the system.

Changing the default parameters of the **Bootup** tab bypasses standard system checks.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

The goal of bootup procedure tab is to bypass the standard bootup procedure for devices that do not comply with CANopen standards.

The **Bootup** tab allows you to configure the bootup procedure:



This table defines the different functionalities of the **Bootup procedure configuration**:

Type		Functionality
Restore	**No Restore**	–
	Restore communication parameters	Enabled option according to the object 0x1011sub02. If checked, all parameters between 0x1000 to 0x1FFF are restored.
	Restore application parameters	Enabled option according to the object 0x1011sub03. If checked and if the device correctly implements the service, all application parameters are restored.
	Restore all parameters	Enabled option according to the object 0x1011sub01. If checked, all parameters are restored to default value.
Reset	**Reset communication parameters**	Option always enabled. If checked, all communication parameters are reset.
	Reset node	Option always enabled. If checked, all parameters are reset.
Check node	**Device type**	If the device type identification value for the slave in object dictionary 0x1F84 is not 0x0000 ("don't care"), compares it to the actual value. **NOTE:** If unchecked, this option forces the object dictionary 0x1F84 to 0x0000.
	Device identity	If the configured vendor ID in object dictionary 0x1F85 is not 0x0000 ("don't care"), read slave index 0x1018, sub-Index 1 and compare it to the actual value. The same comparison is done for **ProductCode**, **RevisionNumber**, and **SerialNumber** according to object 0x1F86-0x1F88. **NOTE:** If unchecked, this option forces the object dictionary 0x1F86-0x1F88 (sub device node ID) to 0x0000.
Download Configuration	**Force communication parameters**	Forces the download of communication or configuration parameters (unchecked by default). If the option is: ● Checked, it forces all the corresponding objects to be downloaded. ● Unchecked, you have to follow these standard rules: parameters are downloaded if they are different from the default values or if they are forced in the object dictionary. In other cases, parameters are not downloaded.
	Force application parameters	
Start	**Start node**	If the option is: ● Checked (default value), the CANopen master starts the device automatically after the bootup procedure. ● Unchecked, the device stays in pre-operational state after bootup procedure. In this case, the device has to be started by the application program.

Object Dictionary Tab

⚠ WARNING

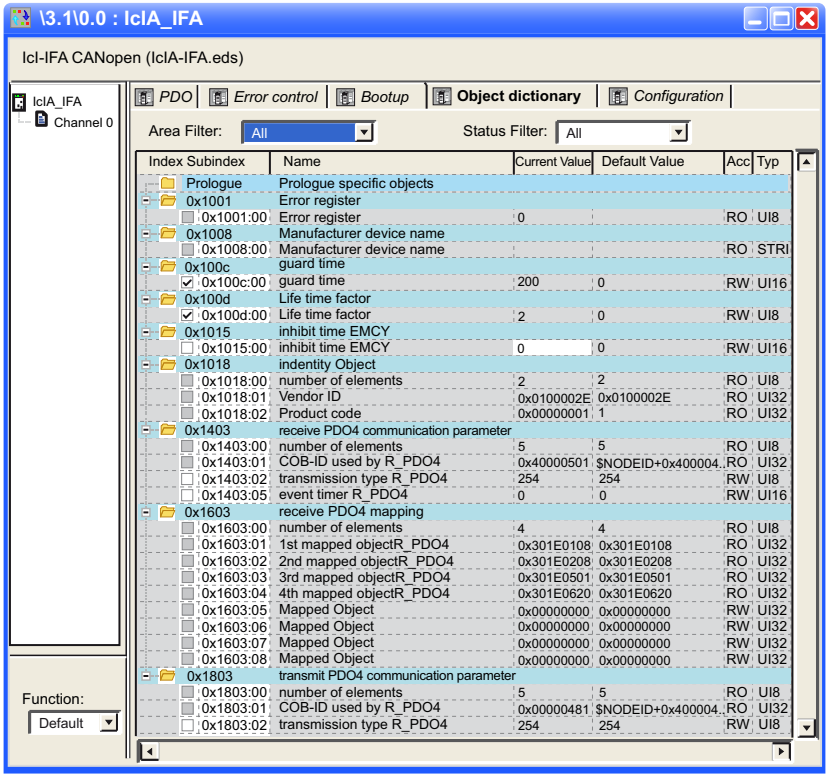
UNEXPECTED EQUIPMENT OPERATION

Manually verify all Object Dictionary values and mapping.

Changing the default values and mapping of the Object Dictionary table generates non-standard behavior of the equipment.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

The **Object Dictionary** tab allows you to configure and integrate third-party products:



This table shows the elements of the **Object Dictionary** tab and their functions:

Element	Function
Parameter	If the check box associated to each parameter is: ● Activated: force parameters to be transmitted even if they are unchanged. ● Deactivated: block parameters that do not need to be sent to the device. **NOTE:** To prevent programming redundancies or conflicts, parameters that can be modified in **Configuration**, **PDO**, or **Error Control** tabs are grayed out.
Current Value	Modify the current value of an object (except read-only objects) by typing a value in the box. By default, the object is sent if the current value is modified. You can block object sending by deactivating the check box.
Default Value	Set objects to a specific value just before (prolog) or just after (epilog) the standard bootup procedure.
Area Filter	● All: Show all area. ● Prolog / Epilog: Show only prolog and epilog projects ● XXXX...YYYY: Show only objects between XXXX to YYYY.
Status Filter	● All: Show all objects. ● Configured: Show only transmitted objects to the device during bootup. ● Not configured: Show only not transmitted objects to the device. ● Modified: Show only objects from which values are different from default values

You can drag and drop available objects from the index folder to the prolog or epilog section. If not, for example PDOs or read-only, a pop-up appears. Some functions are only available in prolog and epilog section.

NOTE: An object that has been put in the prolog or epilog section is always sent.

Configuration Using an External Tool

Overview

To configure a Lexium 32/32i, a Lexium ILA, ILE, ILS, a TeSys U, or an ATV device, it is necessary to use an external tool:

- Advantys Configuration Software for the STB
- SoMove software for the ATV32, ATV312, ATV12, ATV61, ATV71, ATV process, and the TeSys U
- SoMove software for the Lexium 32/32i,
- Lexium CT for Lexium ILA, ILE, ILS.
- EasyIclA V1.104 for ICLA_IFA, ICLA_IFE, ICLA_IFS.

NOTE: To facilitate the programming of the motion and drives devices, it is highly recommended to use the software with the Unity Pro MFBs.

File Access

If external tools generate an EDS or DCF file, you can integrate the products via the Hardware Catalog Manager *(see Hardware Catalog Manager, Software, Operation Guide)*.

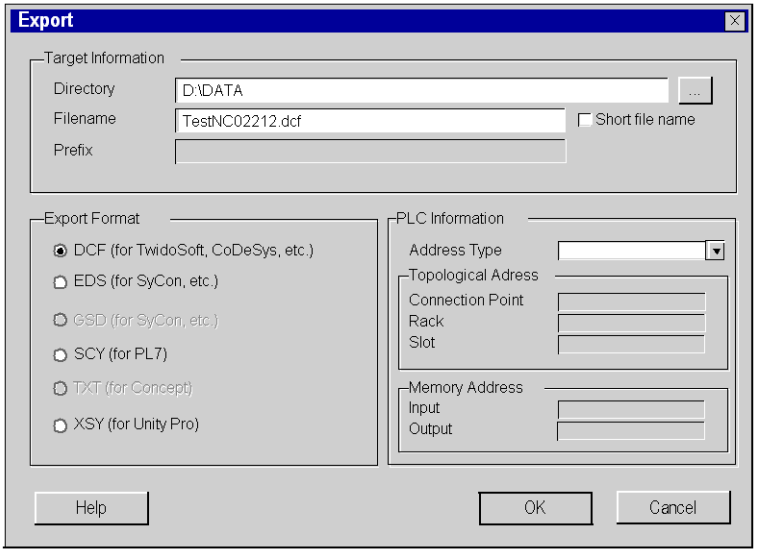
For Advantys STB distributed I/O modules, you can directly access the DCF file from Unity Pro as shown below.

Advantys Configuration Software

Advantys Configuration Software (Version 2.5 or higher) has to be used to configure an STB NCO 2212. The Advantys Configuration Software validates the configuration and creates a DCF file that contains all the objects used in the configuration ordered in the proper sequence. The DCF file can be imported from Unity Pro.

NOTE: The creation of the DCF file is only possible from the full version of Advantys.

To add an island to a CANopen bus, follow these steps:

Step	Action
1	In Advantys Configuration Software (Version 2.2 or above), create a new Island.
2	Select the STB NCO 2212 network interface module.
3	Select the modules that are used in the application.
4	Configure the island.
5	When the configuration is over, click **File/Export** to export the island in DCF format. **Result**: The **Export** window is displayed: 
6	Click **OK** to confirm.
7	Once the file is exported, launch Unity Pro and open the project in which the island is used.
8	Add an STB device to the Bus Editor *(see page 61)*.
9	Right-click the STB device and click **Open the module**.
10	In the **PDO** tab, click the **Import DCF** button.
11	Click **OK** to confirm. **Result**: The PDOs are configured automatically.

NOTE: Repeat this procedure to modify the topology of an island.
For more information about the STB configuration, refer to the STB user manual.

Section 4.5
Master Configuration

Subject of this Section

This section presents the master configuration.

What Is in This Section?

This section contains the following topics:

Topic	Page
CANopen Master Module Configuration Window	88
CANopen Master Port Configuration Screen	90

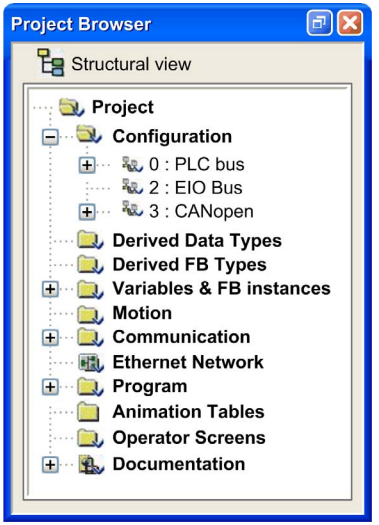
CANopen Master Module Configuration Window

Overview

The module configuration editor is used to configure the master of the CANopen bus.

Procedure

To access the module configuration editor follow these steps:

Step	Action
1	From the **Structural view** of the **Project Browser**, expand (+) the **Configuration** directory: 
2	Expand (+) the bus (**PLC bus** or **EIO Bus**) where is declared the CANopen X80 master module you want to configure. **NOTE:** You can also right-click the bus CANopen you want to configure the master in the **Configuration** directory, and click **Go to Bus Master**.
3	Right-click the BMECXM module, and click **Open**. **Result**: The module configuration window for the CANopen X80 master module is displayed.

Description

The module configuration window has three tabs:

- **Overview** (not editable)
- **Web : Main IP** (in online mode only)
- **Ethernet Configuration**

Ethernet Configuration Tab

This table shows the elements of the **Ethernet Configuration** tab and their functions:

Element	Function
Scanner configuration	Provide the scanner, protocol, and profile of the scanner chosen at module insertion in Unity Pro project. This is not editable. **NOTE:** To change scanner association (assuming that the choice is available), the module needs to be deleted and inserted again with the new scanner association.
IP/DHCP configuration	Click **Update IP/DHCP configuration** hyperlink to access an **Ethernet Network** window where you can edit: ● The **IP Address**, **Subnet Mask**, and **Gateway Address**. ● The **Device Name** in the **Identifier** column.
Other configuration	Click **Go to DTM configuration** for a direct access to the DTM user interface *(see page 99)*.

Web : Main IP Tab

When online, this tab displays the web pages directly for diagnostics purpose *(see page 145)*.

NOTE: To go online, refer to DTM connections *(see page 96)*.

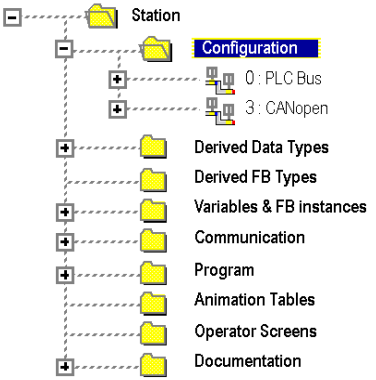
CANopen Master Port Configuration Screen

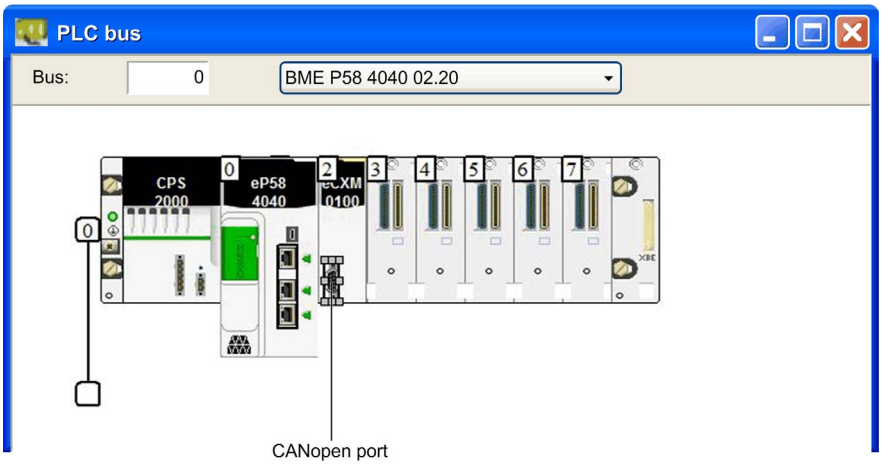
Overview

The CANopen master port configuration screen is used to declare and configure the master of the CANopen network from a Modicon M580 PLC station.

Procedure

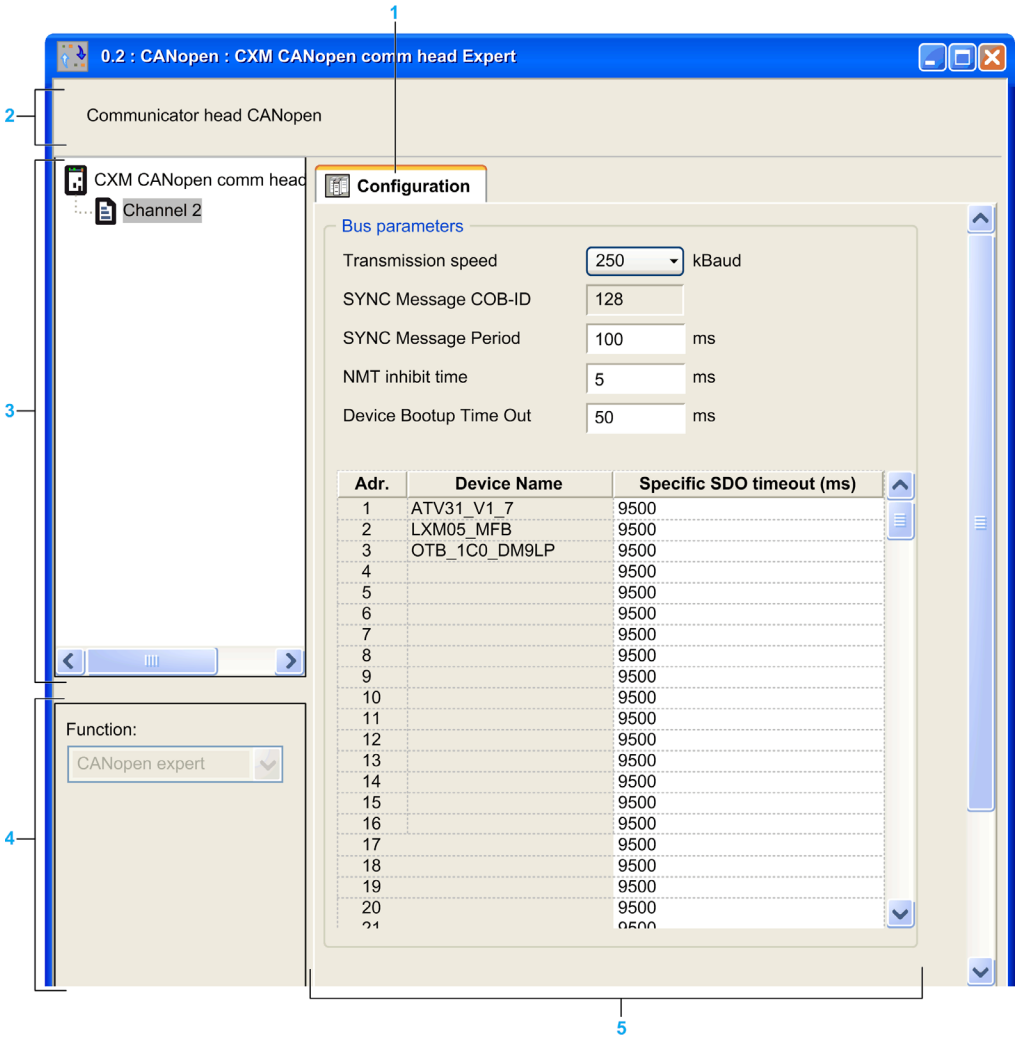
To access the CANopen master port configuration screen of a M580, follow these steps:

Step	Action
1	From the **Project Browser**, expand (+) the **Configuration** directory: 
2	Double-click the **PLC Bus** subdirectory. **Result**: The **PLC Bus** window is displayed.

Step	Action
3	Double-click the CANopen port of the BMECXM module:  **Result**: The CANopen master configuration window is displayed.

Description

This figure shows an example of the CANopen master configuration window:



This table shows the elements of the CANopen master configuration window and their functions:

Number	Element	Function
1	Tab	Indicates the type of window displayed. In this case, it is the configuration window.
2	Module	Indicates the abbreviated heading of the BMECXM.
3	CANopen communication	Allows you to select: ● The device and display the **Overview** tab that gives the characteristics of the BMECXM. ● The channel and display the **Configuration** tab that enables you to declare and configure the CANopen master.
4	General parameters	Indicates the slave functions *(see page 70)*.
5	Configuration	Allows you to configure the parameters of the CANopen bus.

Bus Parameters

The CANopen master port configuration screen allows the configuration of these bus parameters:

Bus parameters	Default value	Comments
Transmission speed	250 kBaud	Enables you to select in a dropdown list: ● 1000 ● 500 ● 250 ● 125 ● 50 ● 20
SYNC Message COB-ID	128	The BMECXM send SYNC message using COB-ID 0000 0080 hex.
SYNC Message Period	100 ms	Define the interval period between two SYNC messages.
NMT inhibit time	5 ms	During bootup, the CANopen master implements a delay between each NMT message to avoid slave overload. The value has to be given in multiple of 100 µs. **NOTE:** The value 0 disables the inhibit time.
Device Bootup Time Out	50 ms	The global SDO timeout for the master is related to the scanning of the network. During this time, the BMECXM reads the object 1000 hex of each slaves devices to analyze the configuration of the CANopen fieldbus.
Specific SDO timeout (ms)	9500 ms	An individual SDO timeout is necessary for slave devices with long response times that are for accesses to the objects 1010 hex,1011 hex,1F50 hex. The grid displays the node ID, the name, and the SDO timeout value for each present CANopen slave device.

Chapter 5
Ethernet Services Configuration

Introduction

The configuration of the Ethernet services is done in the BMECXM module DTM.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
DTM Browser	96
DTM User Interface	99
Ethernet IO Tab	104
Security Tab	107
SNMP Tab	109
NTP Tab	111

DTM Browser

Overview

Unity Pro incorporates the Field Device Tool (FDT)/Device Type Manager (DTM) approach to integrate distributed devices with your process control application.

For details, refer to chapter *FDT Container (see Unity Pro, Operating Modes)*.

Procedure

To access the DTM configuration for the BMECXM module in the Unity Pro **DTM Browser**, follow these steps:

Step	Action
1	Click **Tools → DTM Browser** to open the **DTM Browser** .
2	In the **DTM Browser**, double-click the name that you assigned to the BMECXM module. **Result**: The DTM **Configuration** window is displayed.
3	In the **Configuration** window, select: • **General Settings** ❍ IP/DHCP ❍ Ethernet IO *(see page 104)* ❍ Security *(see page 107)* ❍ SNMP *(see page 109)* ❍ NTP *(see page 111)* • CANopen **Devices** *(see page 101)*

DTM Connections

To connect or disconnect a DTM and the BMECXM module in the **DTM Browser**, follow these steps:

Step	Action
1	Select the DTM that you want to connect to or disconnect from.
2	Right-click and select: • **Connect**(1) to monitor and diagnose the real-time operation of the device, or • **Disconnect**(1) **NOTE:** A connected DTM appears in **bold** text. The **Connect** command is available only for disconnected DTMs.
(1) You can also access the **Connect** and **Disconnect** commands in the Unity Pro **Edit** menu.	

DTM Types

The **DTM Browser** displays a hierarchical list of DTM nodes on a connectivity tree. The DTM nodes that appear in the list have been added to your Unity Pro project. Each node represents an actual module or device in your Ethernet network.

In the **DTM Browser**, two kinds of DTMs are instantiated automatically:

- Master (communication) DTM: This DTM is both a device DTM and a communication DTM. The master DTM is a pre-installed component of Unity Pro.
- BMECXM DTM: This is a device DTM that allows the configuration of the CANopen bus.

DTM Names

Each DTM has a default name when it is inserted into the browser. The name is limited to 26 characters.

This table describes the components of the default name:

Element	Description
Channel	This is the name of the channel communication medium into which the device is plugged. This name is read from the DTM and is set by the device vendor. For example: EtherNet/IP
Address	This is the bus address of the device that defines the connection point on its parent gateway network. For example: the device IP address
DTM name	The default name is determined by the vendor. You can edit the name in the **DTM Browser** by clicking the DTM node or in the **Ethernet Configuration** tab of the CANopen master module screen *(see page 88)*.

NOTE: The DTM name is different from the **Device Name** that is used to get the IP address of the BMECXM0100 module in the **IP/DHCP** tab.

The default name for gateway and device DTMs is in the format *<BusName><BusNumber>_d<DropNumber>_r<RackNumber>_s<SlotNumber>_<PartNumber>_[SubSetName]*. The name of the subset is optional.

As an example on a local rack, a BMECXM0100 module on PLC bus, drop 0, rack 0, and slot 2, has this name: *PLC0_d0_r0_s2_ECXM0100*.

As an example on a remote drop, a BMECXM0100 module on EIO bus, drop 3, rack 1, and slot 4, has this name: *EIO2_d3_r1_s4_ECXM0100*.

DTM Status

The **DTM Browser** contains graphics to indicate the status of each DTM in the connectivity tree:

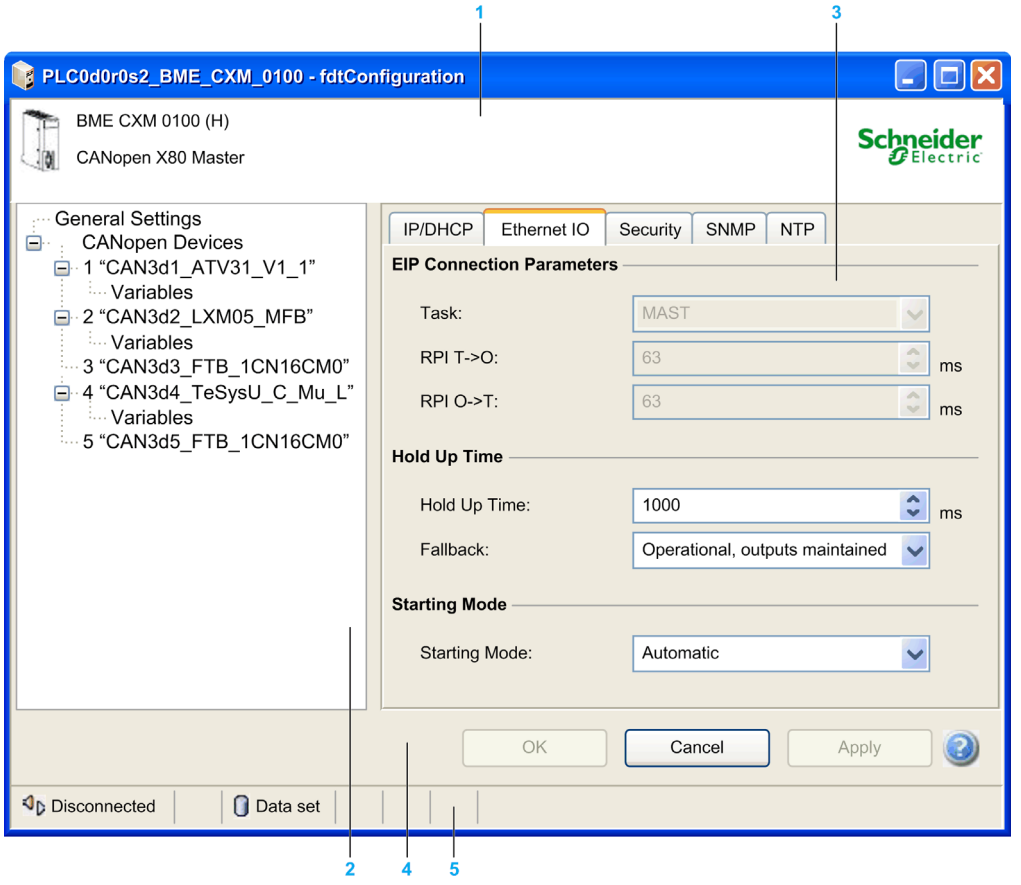
Status	Description
Built / Not-built	A blue check mark is superimposed on a device icon to indicate that the node/subnode is not built. This means that some property of the node has changed. Information stored in the physical device is thus no longer consistent with the local project.
Connected / Disconnected	A connected DTM appears in **bold** text. • Connecting a DTM to its physical device automatically connects all higher-level parent nodes up to the root node. • Disconnecting a DTM from its physical device automatically disconnects all its lower-level child nodes. **NOTE:** Connecting or disconnecting a DTM to or from its device does not also connect or disconnect Unity Pro to or from the device. DTMs can be connected/disconnected while Unity Pro is either offline or online.
Installed / Not-installed	A red **X** is superimposed on a device icon to indicate that the DTM for that device is not installed on the PC.
Ethernet ready	A  is superimposed on a device icon to indicate that the device is an Ethernet ready equipment. This means that the equipment provides additional services compared to standard EIP or Modbus equipment. For more information, refer to Ethernet Ready Equipment *(see Unity Pro, Operating Modes)*.

DTM User Interface

General Layout

You can configure some optional parameters for the BMECXM module through the DTM user interface in Unity Pro.

This figure shows the DTM user interface:



1 Identification area
2 Navigation area
3 Application area
4 Action area
5 Status bar

Description

This table describes the different areas that make up the DTM user interface:

Element	Function
Identification area	Shows the device type name and product name.
Navigation area	The device tree contains: ● **General Settings** node *(see page 100)* ● **CANopen Devices** node *(see page 101)* with a device list that includes a subnode for each device
Application area *(see page 102)*	Contains parameters, most of them being editable.
Action area	Contains these buttons: ● **OK**: Save your changes and close the page. ● **Cancel**: Cancel changes. ● **Apply**: Save your changes and keep the page open. ●  : Open an online help page. **NOTE:** Your changes take effect only when they are successfully downloaded from your PC to the CPU and from the CPU to the BMECXM modules and network devices.
Status bar *(see page 103)*	Shows status information with: ● Connection state icons ● Additional icons

Navigation Area: General Settings Node

The **General Settings** node contains 5 tabs:

Tab	Function
IP/DHCP	These read-only parameters are displayed: ● **IP Settings**: IP address, subnet mask, and default gateway ● **Rack and Slot Information**: rack ID, slot number, and device name **NOTE:** These parameters can be changed in Unity Pro.
Ethernet IO *(see page 104)*	These parameters are displayed: ● **EIP Connection Parameters**: ❍ Task: MAST (by default), FAST, or AUX. ❍ RPI T->O: requested packet interval of the consumption connection(1). ❍ RPI O->T: requested packet interval of the production connection(1). ● **Hold Up Time**: hold-up time and fallback ● **Starting Mode**: automatic (by default) or manual
(1) The refresh period for this connection is in ms (2...2550).	

Tab	Function
Security *(see page 107)*	These parameters are displayed: • **Global Policy**: to enforce or unlock security. • **Services**: FTP, HTTP, SNMP and EIP to be enabled/disabled. • **Access Control**: to enable/disable Ethernet access to the EtherNet/IP server.
SNMP *(see page 109)*	These parameters are displayed: • **IP Address Managers**: IP addresses. • **Agent**: location and contact, and SNMP manager to be enabled/disabled. • **Community Names**: set, get, and trap. • **Security**: authentication failure trap to be enabled/disabled.
NTP *(see page 111)*	The **NTP Client Configuration** parameter is displayed: NTP, IP addresses, and polling period.
(1) The refresh period for this connection is in ms (2...2550).	

Navigation Area: CANopen Devices Node

When clicking the **CANopen Devices** node, a **Device List** tab shows all slave devices with read-only parameters: **Address**, **Device Name**, **Vendor**, **Type**, and **Version**.

When expanding (+) the **CANopen Devices** node and clicking a slave device, the **I/O** tab is displayed with 2 fields:

Field	Function
IO Structure Name	These parameters are displayed: • **Structure Name** (read-only): name assigned to the device input structure from the imported CANopen configuration • **Variable Name** (editable) **NOTE:** The variable name is reinitialized when clicking the **Default Name** button or if the structure name has been changed.
Variables Management	The **Import Mode** (read-only): displays the automatic option only.

Each slave device contains a **Variables** subnode.

When expanding (+) a slave device and clicking the **Variables** subnode, these tabs are displayed, when relevant, with read-only parameters:

Tab	Function
Input	Displays **Type**, **Offset**, **Name**, and **Comment** columns.
Input(bit)	Displays **Offset**, **Position**, **Name**, and **Comment** columns.
Output	Displays **Type**, **Offset**, **Name**, and **Comment** columns.
Output(bit)	Displays **Offset**, **Position**, **Name**, and **Comment** columns.

NOTE: You cannot add, delete, or edit variables.

In these tabs, all the bits or bytes from the I/O modules of the CANopen device are displayed.

At the beginning of each row, the data type is visually indicated:

Data types	BOOL	BYTE	INT, UINT, WORD	DINT, UDINT, DWORD	REAL	String
Visual representation						

NOTE: Icons are left blank if bits and bytes are unused.

Application Area

When you edit a parameter, Unity Pro displays an icon next to the field you are editing and in the navigation tree.

These icons refer to the value of the parameter that is being edited:

Icon	Description
	The entered value is not known. The **Apply** button does not work until a correct value is entered.
	The entered value is not valid. The **Apply** button does not work until a valid value is entered.
	This parameter has changed. The **Apply** button does not work until the value is corrected.

NOTE: You can move the mouse over the icons to display tooltips.

Status Bar

This table shows the connection state icons of the status bar and their functions:

Icon	Element	Function
	Disconnected	The DTM is offline.
	Connected	The DTM is online.
	Connecting	The DTM is connecting.
	Disconnecting	The DTM is disconnecting.
	Communication problem	There are communication errors detected, for example wrong IP address.

This table shows the additional icons of the status bar and their functions:

Icon	Element	Function
	Communication in progress	The DTM communicates with the device.
	Data set	Displays the offline data stored in the DTM.
	Device	Displays the offline data stored in the device itself.
	Summary	Displays the summary state of the data when at least one parameter value has been modified (none by default).

Ethernet IO Tab

Overview

The **Ethernet IO** tab allows you to configure these parameters in Unity Pro:

- **Task**
- **RPI**
- **Hold up time**
- **Fallback**
- **Starting Mode**

Task Applications

An M580 CPU can execute single-task and multi-task applications. A single-task application only executes the MAST task. A multi-task application defines the priorities of each task.

There are four tasks available:

- MAST
- FAST
- AUX0
- AUX1

Each BMECXM module, is linked to a unique PLC task.

Tasks depend on the profile that is set by Unity Pro. For the BMECXM module, there are two profiles:

- Remote (RIO scanner) where you can edit all the tasks
- Distributed (DIO scanner) where you can edit only MAST tasks

For more information on tasks, refer to the chapter on Application Program Structure *(see Unity Pro, Program Languages and Structure, Reference Manual)*.

Modicon M580 Task Characteristics

The time model and task period are defined as follows:

Task	Time model	Task period (ms)	
		Range	Default value
MAST(1)	Cyclic(2) or periodic	1...255	20
FAST	Periodic	1...255	5
AUX0	Periodic	10...2550 by 10	100
AUX1	Periodic	10...2550 by 10	200

(1) MAST task is mandatory.
(2) When set to cyclic mode, the minimum cycle time is 8 ms if there is a RIO network and 1 ms if there is no RIO network in the system.

RPI Values

The Requested Packet Interval (RPI) depends on the profile that is set by Unity Pro at instantiation time. For the BMECXM module, there are two profiles:

- Remote (RIO scanner) where you cannot change the RPIs
- Distributed (DIO scanner) where you can change the RPIs

There are two RPI values:

- T->O for process inputs
- O->T for process outputs

When the BMECXM module is the scanned by the RIO scanner, the RPI values are as follows:

Task Configuration (Period)	RPI			
	Process	Calculated Value...	Minimum(2)	Maximum(2)
Periodic (>0)	T->O	1/2 of PLC task period (rounded down ms)	2 ms	255 ms
	O->T	1.1 of the PLC task period (rounded up ms)	5 ms	
Cyclic(1) (=0)	T->O	1/4 of PLC task watchdog	3 ms	
	O->T	1/4 of PLC task watchdog	5 ms	
(1) For the MAST task only (2) Linked to the task configuration.				

When the BMECXM module is the scanned by the DIO scanner a distributed, the RPI values are as follows:

Mast Task Configuration (Period)	RPI			
	Process	Default Value...	Minimum(1)	Maximum(1)
Periodic (>0)	T->O	2 times of PLC task period	1/2 of PLC task period (rounded down ms)	1500 ms
	O->T	2 times of PLC task period	1.1 of the PLC task period (rounded up ms)	
Cyclic (=0)	T->O	1/4 of PLC task watchdog	2 ms	255 ms
	O->T	1/4 of PLC task watchdog	5 ms	
(1) Linked to the mast task configuration.				

NOTE: Outputs that have changed are published at the end of each PLC scan. For the other ones, the default value applies.

Hold Up Time

The hold up time represents the timeout of input reception to switch to FALLBACK state. By default the **Hold Up Time** = 4 x **Watchdog**.

You can modify the **Hold Up Time** value if you set a value greater than the default value and lower than 5 seconds.

NOTE: For the application, the maximum value is 5 s even if the default or set value is greater than 5 s.

Fallback

The fallback information defines the behaviors of the device:

- **Operational, outputs maintained**: it maintains outputs. The values are kept.
- **Operational, outputs set to 0**: it resets outputs. The values are set to 0.
- **Stop**: the CANopen bus is in STOP state.
  NOTE: For more information, refer to Fallback Strategy *(see page 50)*.

Starting Mode

You can select:

- **Automatic**. The scan is started by the application.
- **Manual**. The BMECXM needs the `EM_Start` command *(see page 172)* from the PLC to go to the `CONNECTED RUN` state and to set the CANopen field bus to `OPERATIONAL`.
  NOTE: For more information, refer to the chapter Operating Modes *(see page 45)*.

⚠ WARNING

UNEXPECTED EQUIPMENT BEHAVIOR

Do not use `RIO_CTRL` and `DIO_CTRL` control bits of the M580 CPU device DDT to start or stop the scan of the BMECXM.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

Security Tab

Overview

The **Security** tab allows you to configure the security level of the services.

The default settings represent the maximum level of security level. The increased security reduces the communication capabilities and the access to communication ports.

NOTE: For general information about security, refer to the **Modicon Controllers Platform, Cyber Security, Reference Manual**.

Properties

This table describes the properties of the **Security** tab:

Parameter		Description
Global Policy	**Enforce Security**	Reset all the services to the default settings and implement the highest level of security.
	Unlock Security	Use the lowest level security settings (opposite of default settings).
Services	FTP	Enable or disable (default) firmware upgrade.
	HTTP	Enable or disable (default) the Web access service.
	SNMP	Enable or disable (default) the diagnostic access service.
	EIP	Enable or disable (default) the diagnostic access service for exchanging I/Os and diagnostic information with the CPU.
Access Control	**Enabled** (by default)	Ethernet services access authorized for the listed addresses.
	Disabled	There is no restriction on which network devices can access EtherNet services. The BMECXM module accepts EtherNet/IP requests from any device.

Services

For security reasons, all the communication ports of the BMECXM module are disabled by default.

If EIP service is disabled, exchanging with the CPU is not possible. Therefore, you have to enable at least the EIP service in the **Security** tab so that the scanner can access the BMECXM module.

If FTP service is disabled, an FTP upgrade is not possible.

Set the **Security** tab parameters before downloading the application to the CPU.

NOTE: Schneider Electric recommends disabling services that are not being used.

Enabling Access Control

When the **Access Control** is enabled, it restricts access to the BMECXM module services declared in the list.

In the box, you can add the IP addresses of:

- The BMECXM module with **Subnet** set to **Yes** that allows any device in the subnet to communicate with the module using EtherNet/IP.
- Any client device that may send a request to the BMECXM module, which in this case acts as an EtherNet/IP server.
- Your maintenance PC that communicates with the BMECXM module via Unity Pro to configure and diagnose your application and view the module webpages.

You must ensure that the corresponding CPU scanner address is filled in the list of authorized addresses.

NOTE: Using the BMECXM module in RIO/DIO requires to add the corresponding RIO/DIO scanner IP address in the Access Control list (ACL).

Adding Devices to the Access Control List

NOTE: Before declaring a new address in the list, you must enable the corresponding service in the section Services *(see page 107)*.

To add devices, follow these steps:

Step	Action
1	Enable the **Access Control**.
2	Click **Add**.
3	Enter the address of the device to access the BMECXM module with either of these methods: ● *Add a single IP address*: Enter the IP address of the device and select **No** in the **Subnet** column. ● *Add a subnet*: Enter a subnet address in the **IP Address** column. Select **Yes** in the **Subnet** column. Enter a subnet mask in the **Subnet Mask** column. **NOTE:** The subnet in the **IP Address** column can be the subnet itself or any IP address in the subnet. If you enter a subnet without a subnet mask, an on-screen message states that the modification cannot be validated. **NOTE:** A red exclamation point (!) indicates a detected error in the entry. You can save the configuration only after the detected error is addressed.
4	Select one or more of the following methods of access you are granting the device or subnet: FTP, HTTP, SNMP, EIP.
5	Repeat these steps 2 to 4 for each additional device or subnet to which you want to grant access to the BMECXM module. **NOTE:** You can enter up to 128 authorized IP addresses or subnets.
6	Click **Apply**.

NOTE: You can remove a device by selecting its IP address and clicking **Delete**.

SNMP Tab

Overview

The **SNMP** tab allows you to configure the SNMP agent in Unity Pro.

SNMP Agent

The BMECXM module includes an SNMP agent. An SNMP agent is a software component running on the BMECXM module.

It allows access to the diagnostic and management information of the module via the SNMP service. The SNMP agent can communicate with up to 2 SNMP managers as part of an SNMP service. SNMP browsers, network management software, and other tools typically use SNMP to access this data.

In addition, the SNMP agent can be configured with the IP address of up to two devices (typically PCs running network management software) to be the target of event driven trap messages. These trap messages inform the management device of events such as cold start and unauthorized access.

Properties

This table describes the properties of the **SNMP** tab:

Group/Parameter		Description
IP Address Managers	**IP Address Manager 1**	The IP address of the first SNMP manager to which the SNMP agent sends trap messages.
	IP Address Manager 2	The IP address of the second SNMP manager to which the SNMP agent sends trap messages.
Agent	**Location (SysLocation)**	The device location (32 characters maximum)
	Contact (SysContact)	Information describing the person to contact for device maintenance (32 characters maximum).
	Enable SNMP Manager	• Check the box: you cannot edit the **Location** and **Contact** settings on this page. Those settings are managed by the SNMP Manager. • Uncheck the box: you can edit the **Location** and **Contact** settings on this page.
(1) private by default **(2) public** by default **(3) alert** by default		

Group/Parameter		Description
Community Names	**Set**(1)	Password required by the SNMP agent before executing write commands from an SNMP manager.
	Get(2)	Password required by the SNMP agent before executing read commands from an SNMP manager.
	Trap(3)	Password required by the SNMP agent before the manager accepts trap messages from the agent.
Security	**Enable "Authentication Failure" Trap**	Check the box: the SNMP agent sends a trap message to the SNMP manager if an unauthorized manager sends a **Get** or **Set** command to the agent.
(1) private by default **(2) public** by default **(3) alert** by default		

NTP Tab

Overview

The **NTP** tab allows you to configure the NTP Unity Pro.

Properties

This table describes the properties of the **NTP** tab:

Parameter		Description
NTP Client Configuration	**NTP**	To be configured when syslog event logging is configured in Unity Pro via **Tools → Project Settings → General → PLC diagnostics**.
	Primary IP address	The IP address of the first NTP server
	Secondary IP address	The IP address of the second NTP server
	Polling Period	The polling period is number of seconds (1...120, default = 20) between updates from the NTP server. A smaller polling period results in better accuracy.

Chapter 6
Language Objects

Introduction

This chapter describes implicit messages, which are mapped in the device DDT, and emergency messages that are associated to the BMECXM modules.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Implicit Process Data Exchange	114
Device DDT Variables	115

Implicit Process Data Exchange

Overview

Use implicit messaging to create a communication link between the BMECXM module and the CPU.

Description

The BMECXM module supports implicit exchange with:

- The CPU through the EtherNet/IP protocol
- The CANopen slaves through PDOs

Implicit EtherNet/IP messages are automatically exchanged at each cycle of the task associated with the module.

Implicit messages concern the status of the slave and BMECXM modules, and the slave process data. Those implicit messages are mapped to the device DDT.

Device DDT Variables

Overview

Device derived data type (device DDT) are used to access slave process data and to read/write the data of the BMECXM modules.

There is:

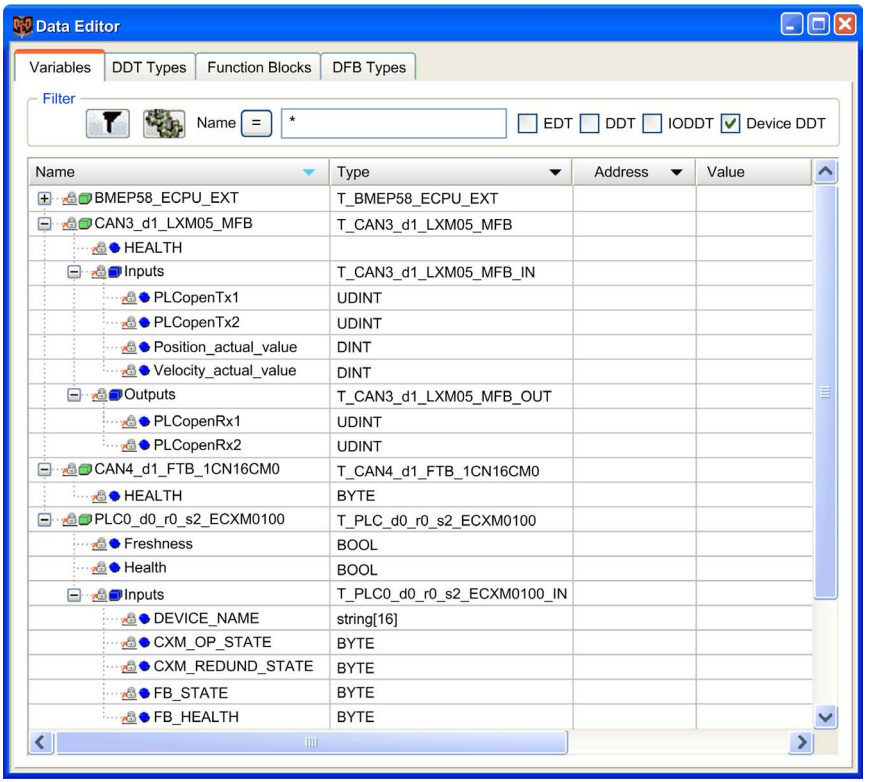
- One device DDT for each CANopen slave with input and output data.
  CANopen slave device DDTs are automatically created when building the application. The device DDTs are built with the list of variables exchanged by the PDOs and with a `HEALTH` BYTE showing the slave status.
  For more information, refer to the **PDO** tab *(see page 77)*.
- One device DDT for each BMECXM module. It is automatically created at insertion of the module in the project.

The **Data Editor** displays these variables *(see page 135)*.

Accessing Device DDT

You can access the device DDTs and the corresponding variables in Unity Pro. You can add this variable to a user-defined Animation Table *(see Unity Pro, Operating Modes)* to monitor read-only variables and edit read/write variables.

To access the device DDTs, follow these steps:

Step	Action
1	Click **Tools → Project Browser** to open the Unity Pro **Project Browser**.
2	Expand (+) **Variables & FB instances**.
3	Double-click **Device DDT Variables**. **Result**: The **Data Editor** window is displayed: 
4	In the **Variables** tab, expand (+) the name to display inputs, outputs, and other parameters.

NOTE: The red arrow and lock icons in the **Device DDT** table indicate that the variable name was auto-generated by Unity Pro based on the configuration of the BMECXM module and CANopen slave. The variable name cannot be edited.

Chapter 7
Programming

Introduction

This chapter describes the programming of a CANopen architecture.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Network Management Services	118
Exchanges Using SDOs	121
READ_SDO: Reading Service Data Object	122
WRITE_SDO: Writing Service Data Object	125
Function Block Examples	128

Network Management Services

Overview

The network management (NMT) is used to start, stop, reset and initialize CANopen nodes. It process boot-up messages and error control events of the individual CANopen slave devices.

The error control mechanism monitors CANopen slave devices by **Heartbeat** or **Node Guarding**.

The behavior of the CANopen slave devices depends on the configuration of the following objects:

- `NMT startup` object (1F80 hex)
  This object specify the start-up state (`OPERATIONAL`, `PRE-OPERATIONAL`, or `STOPPED`) for every CANopen slave devices on the network.
- `Slave assignment` object (1F81 hex)

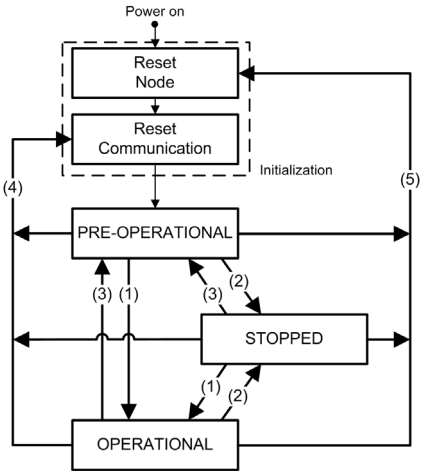
The start-up states of the CANopen slave devices can be modified once the BMECXMCXM is in `CONNECTED RUN` state by sending NMT commands using explicit messages.

The NMT commands are accessible through a `WRITE_SDO` or `READ_SDO` in object 1F82 hex *(see page 163)*.

NOTE: NMT commands are accepted in both automatic and manual modes.

NMT State Machine

The following figure gives the NMT service commands to controlling the operating behavior of a CANopen node on the network:



(1) `Start remote node`
(2) `Stop remote node`
(3) `Enter pre-operational`
(4 `Reset communication`
(5) `Reset node`

NMT Commands Values

The following table gives the NMT commands available through CANopen SDO commands *(see page 168)*:

Value (hex)	READ_SDO command	WRITE_SDO command
00	NMT state unknown	Reserved
01	CANopen device missing	Reserved
02	Reserved	
03	Reserved	
04	NMT state `STOPPED`	NMT service `Stop remote node`
05	NMT state `OPERATIONAL`	NMT service `Start remote node`
06	Reserved	NMT service `Reset node`
07	Reserved	NMT service `Reset communication`
08	Reserved	
–	–	
7E	Reserved	
7F	NMT state `PRE-OPERATIONAL`	NMT service `Enter pre-operational`
80	Reserved	
–	–	
83	Reserved	
84	NMT state `STOPPED`	NMT service `Stop remote node` (excluding NMT master and requesting CANopen device)
85	NMT state `OPERATIONAL`	NMT service `Start remote node` (excluding NMT master and requesting CANopen device)
86	Reserved	NMT service `Reset node` (excluding NMT master and requesting CANopen device)
87	Reserved	NMT service `Reset communication` (excluding NMT master and requesting CANopen device)
88	Reserved	
–	–	
8E	Reserved	
8F	NMT state `PRE-OPERATIONAL`	NMT service `Enter pre-operational` (excluding NMT master and requesting CANopen device)
90	Reserved	
–	–	
FF	Reserved	

NMT Command Data Examples:

This table shows the command data for a `Start remote node`:

Value	Size	Parameter
0: PLC 1...5: DTM	SINT	Connect ID
127	SINT	Node ID
1F82 hex	INT	Index
Node ID: 1...7F hex: Node ID targeted 80 hex: all nodes (broadcast)	SINT	Subindex
[1]	INT	Length
05 hex 85 hex (excluding master)	SINT	Data

This table shows the command data for a `Stop remote node`:

Value	Size	Parameter
0: PLC 1...5: DTM	SINT	Connect ID
127	SINT	Node ID
1F82 hex	INT	Index
Node ID: 1...7F hex: Node ID targeted 80 hex: all nodes (broadcast)	SINT	Subindex
[1]	INT	Length
04 hex 84 hex (excluding master)	SINT	Data

Exchanges Using SDOs

Overview

SDO commands are used to access (read/write) the parameters of CANopen entries and devices in the object dictionary.

For SDO objects sent by the PLC application, the explicit exchanges of messages on a CANopen bus is done by read/write functions.

⚠ WARNING

UNINTENDED EQUIPMENT OPERATION

When modifying a variable, check the consequences of the SDO command in the documentation of the specific target CANopen device.

Failure to follow these instructions can result in death, serious injury, or equipment damage.

SDO Access

It is possible to access SDOs using the `READ_SDO` *(see page 122)* and `WRITE_SDO` *(see page 125)* function blocks.

For examples of SDO function blocks, refer to function block example *(see page 128)*.

SDO Timeouts

The following SDO timeouts are configurable in the CANopen bus parameter window *(see page 93)*:

Global SDO timeout: By default the value is set to 50 ms and defined in the object `5FF0 hex`. This is the time for the BMECXM module to read object `1000 hex` of each CANopen slave devices of the CANopen fieldbus at bootup.

Slave specific SDO timeout: By default the value is set to 9500 ms for all CANopen slave devices and defined in the object `5FF1 hex`.

This is the time for the BMECXM module to read objects `1010 hex`, `1011 hex`, and `1F50 hex` of the CANopen slave device at bootup.

NOTE: This slave specific SDO timeout is necessary for devices with long response time.

In addition to these configurable SDO timeouts, a `READ_SDO` has a timeout of 1 s and a `WRITE_SDO` has a timeout of 2 s.

READ_SDO: Reading Service Data Object

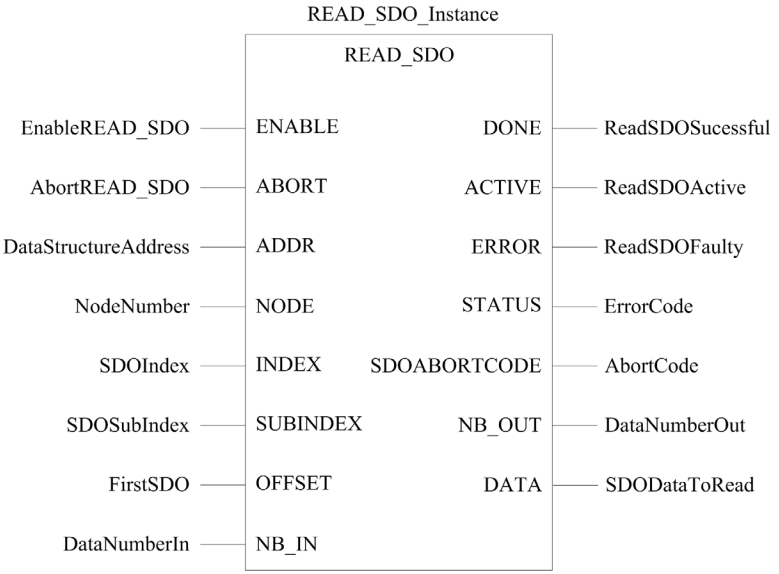
Function Description

The `READ_SDO` function block reads (explicit exchange) from the PLC application up to the device (SDO).

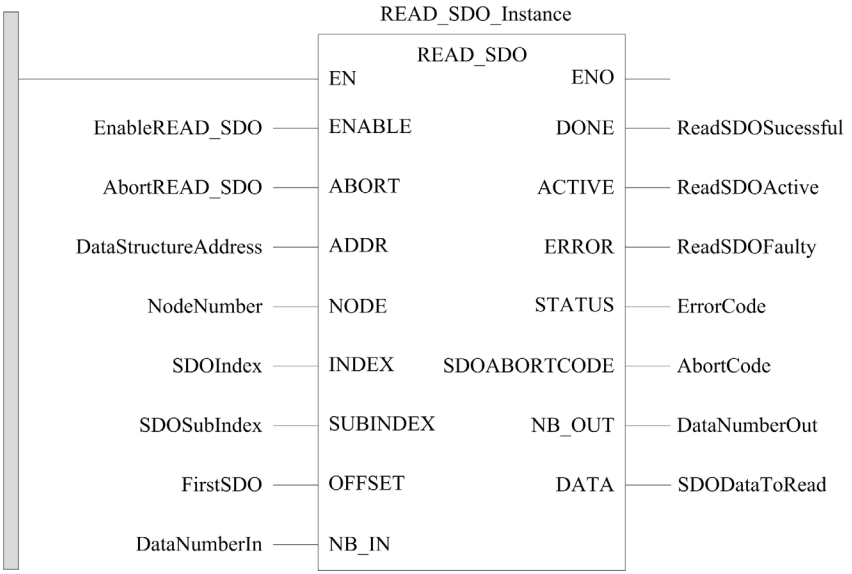
This function block provides access to the abort code when the SDO command is not successful (only if the fieldbus is in RUN mode, and only towards the configured devices).

FBD Representation

Representation:



LD Representation



IL Representation

Representation:

```
CAL READ_SDO_Instance (ENABLE := EnableREAD_SDO,
ABORT := AbortREAD_SDO, ADDR := DataStructureAddress,
NODE := NodeNumber, INDEX := SDOIndex, SUBINDEX := SDOSubIndex,
OFFSET := FirstSDO, NB_IN := DataNumberIn, DONE => ReadSDOSuccessful,
ACTIVE => ReadSDOActive, ERROR => ReadSDOFaulty, STATUS => ErrorCode,
SDOABORTCODE => AbortCode, NB_OUT => DataNumberOut,
DATA => SDODataToRead)
```

ST Representation

Representation:

```
READ_SDO_Instance (ENABLE := EnableREAD_SDO, ABORT := AbortREAD_SDO,
ADDR := DataStructureAddress, NODE := NodeNumber, INDEX := SDOIndex,
SUBINDEX := SDOSubIndex, OFFSET := FirstSDO, NB_IN := DataNumberIn,
DONE => ReadSDOSuccessful, ACTIVE => ReadSDOActive,
ERROR => ReadSDOFaulty, STATUS => ErrorCode, SDOABORTCODE => AbortCode,
NB_OUT => DataNumberOut, DATA => SDODataToRead)
```

Parameter Description

The following table describes the input parameters:

Input parameter	Data type	Description
ENABLE	BOOL	ON: the operation is enabled.
ABORT	BOOL	ON: the currently active operation is aborted.
ADDR	ANY_ARRAY_INT	Array containing the address of the destination entity of the read operation, result of ADDMX function.
NODE	BYTE	Byte used to select a particular NMT slave device on the CANopen network (16#01 to 16#7F).
INDEX	INT	Two bytes used to access a particular object in a CANopen SDO server device.
SUBINDEX	BYTE	Byte used to access a particular subobject in a CANopen SDO server device.
OFFSET	INT	Two bytes indicating the starting offset into the selected object. It can be non-zero when performing segmented SDO transfers. **NOTE:** Not used when addressing an EtherNet/IP module (address with CIP suffix).
NB_IN	INT	Two bytes providing a count of the desired number of data values to read (in bytes). **NOTE:** • If set to 0, the number of data to read is set to the size of the variable associated to the output parameter DATA. • When used with the BMECXM0100 module, this input parameter equal 0 whatever the value you set.

The following table describes the output parameters:

Output parameter	Data type	Description
DONE	BOOL	ON: the operation concludes successfully.
ACTIVE	BOOL	ON: the operation is active.
ERROR	BOOL	ON: the operation is aborted without success.
STATUS	WORD	Provides the error code *(see Unity Pro, Communication, Block Library)* if an error is detected by the function block.
SDOABORTCODE	DWORD	SDO abort code *(see page 170)* when STATUS = 16#4007
NB_OUT	INT	Size of data (in BYTES) actually returned in the output parameter DATA.
DATA	ANY_ARRAY_BYTE	Read data.

WRITE_SDO: Writing Service Data Object

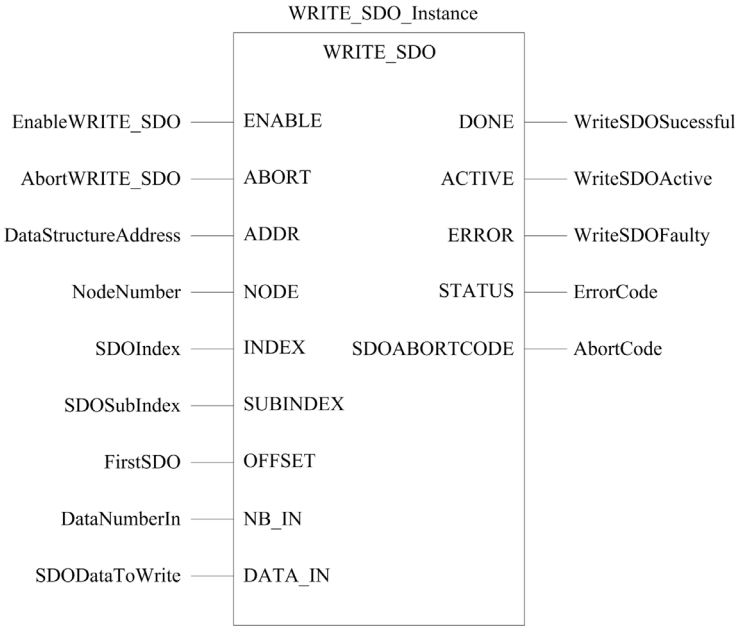
Function Description

The `WRITE_SDO` function block writes (explicit exchanges) from the PLC application up to the device (SDO).

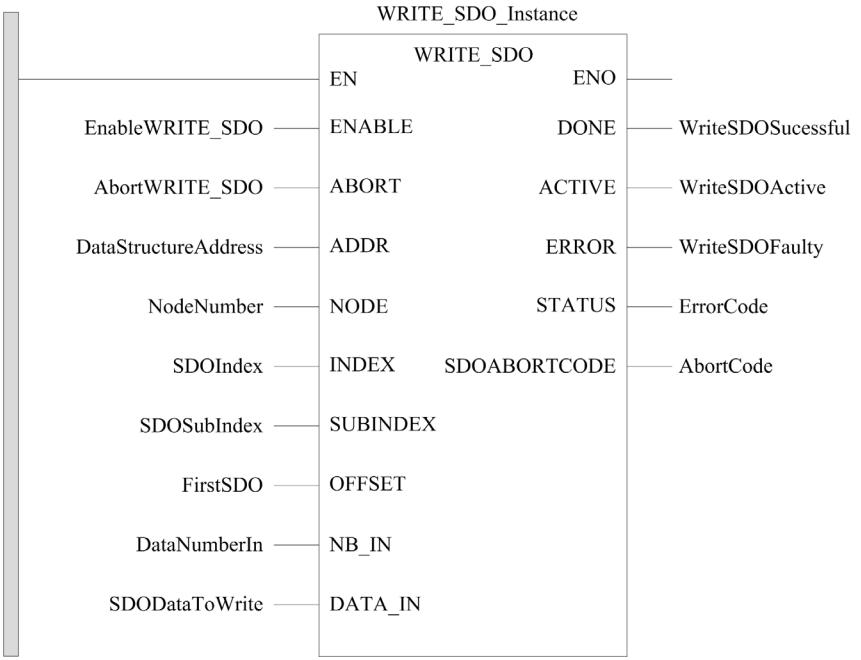
This function block provides access to the abort code when the SDO command is not successful (only if the fieldbus is in RUN mode, and only towards the configured devices).

FBD Representation

Representation:



LD Representation



IL Representation

Representation:

```
CAL WRITE_SDO_Instance (ENABLE := EnableWRITE_SDO,
ABORT := AbortWRITE_SDO, ADDR := DataStructureAddress,
NODE := NodeNumber, INDEX := SDOIndex, SUBINDEX := SDOSubIndex,
OFFSET := FirstSDO, NB_IN := DataNumberIn, DATA_IN := SDODataToWrite,
DONE => WriteSDOSuccessful, ACTIVE => WriteSDOActive,
ERROR => WriteSDOFaulty, STATUS => ErrorCode,
SDOABORTCODE => AbortCode)
```

ST Representation

Representation:

```
WRITE_SDO_Instance (ENABLE := EnableWRITE_SDO, ABORT := AbortWRITE_SDO,
ADDR := DataStructureAddress, NODE := NodeNumber, INDEX := SDOIndex,
SUBINDEX := SDOSubIndex, OFFSET := FirstSDO, NB_IN := DataNumberIn,
DATA_IN := SDODataToWrite, DONE => WriteSDOSuccessful,
ACTIVE => WriteSDOActive, ERROR => WriteSDOFaulty, STATUS => ErrorCode,
SDOABORTCODE => AbortCode)
```

Parameter Description

The following table describes the input parameters:

Input parameter	Data type	Description
ENABLE	BOOL	ON: the operation is enabled.
ABORT	BOOL	ON: the currently active operation is aborted.
ADDR	ANY_ARRAY_INT	Array containing the address of the destination entity of the read operation, result of ADDMX function.
NODE	BYTE	Byte used to select a particular NMT slave device on the CANopen network (16#01 to 16#7F).
INDEX	INT	Two bytes used to access a particular object in a CANopen SDO server device.
SUBINDEX	BYTE	Byte used to access a particular subobject in a CANopen SDO server device.
OFFSET	INT	Two bytes indicating the starting offset into the selected object. It can be non-zero when performing segmented SDO transfers. **NOTE:** Not used when addressing an EtherNet/IP module (address with CIP suffix).
NB_IN	INT	Two bytes providing a count of the desired number of data values to be written (in bytes).
DATA_IN	ANY_ARRAY_BYTE	Data to write.

The following table describes the output parameters:

Output parameter	Data type	Description
DONE	BOOL	ON: the operation concludes successfully.
ACTIVE	BOOL	ON: the operation is active.
ERROR	BOOL	ON: the operation is aborted without success.
STATUS	WORD	Provides the error code *(see Unity Pro, Communication, Block Library)* if an error is detected by the function block.
SDOABORTCODE	DWORD	SDO abort code *(see page 170)* when STATUS = 16#4007.

Function Block Examples

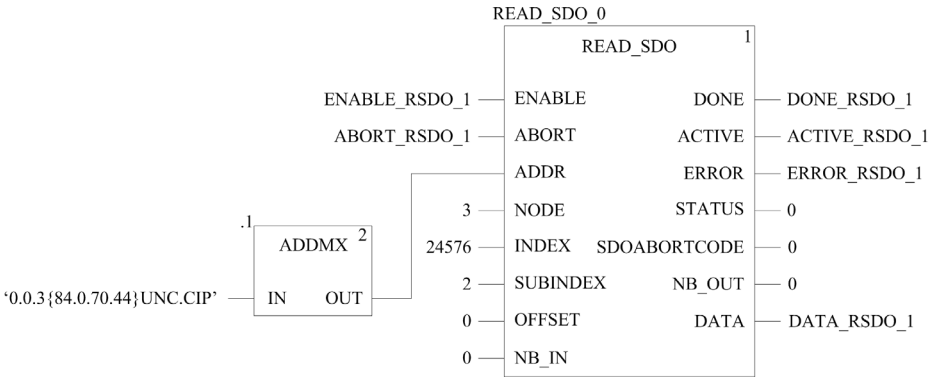
Overview

In these examples:

- The IP address of the BMECXM module is `84.0.70.44`
- The CANopen slave device is an FTB_1CN16CM0 located on the bus at **Node-ID** 3.

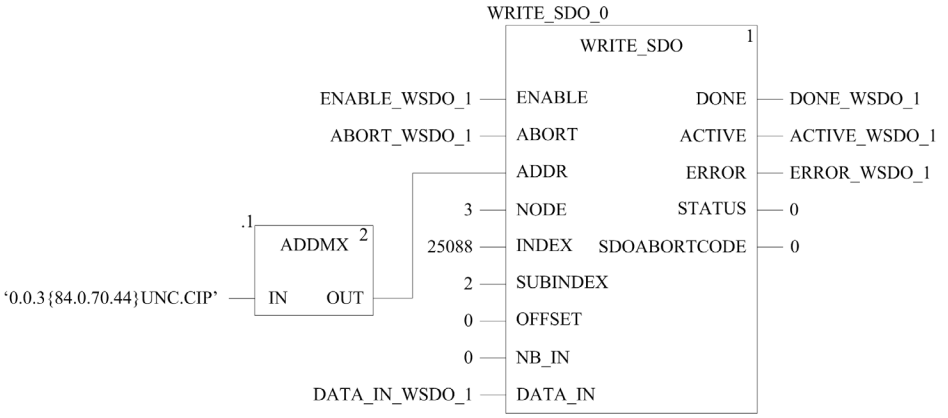
READ_SDO Example in FBD

This example is executing a `READ_SDO` of the object defined at Index 6000 hex and SubIndex 02 hex (Digital Input 8 bits Pin 2):



WRITE_SDO Example in FBD

This example is executing a `WRITE_SDO` of the object defined at Index 6200 hex and SubIndex 02 hex (Write Outputs 9 to 16):



Chapter 8
Diagnostics

Introduction

This chapter describes the diagnostic means for the BMECXM modules:

- LED indicators on the BMECXM module.
- Unity Pro device DDTs:
  - With the device DDT of the M580 CPU, you can perform a first diagnostic.
    If the BMECXM is managed by the RIO scanner, the `RIO_HEALTH` is activated.
    If the BMECXM is managed by the DIO scanner, the `DIO_HEALTH` is activated.
    NOTE: For more details on M580 CPU device DDT, refer to chapter *Standalone DDT Data Structure for M580 CPUs (see Modicon M580, Hardware, Reference Manual)*.
  - The BMECXM device DDT.
  - The CANopen slave device DDT.
- BMECXM DTM diagnosis
- Explicit messages for extended diagnostics:
  - via programming *(see page 117)*.
  - via the M580 CPU DTM graphic user interface, using the `DATA_EXCH`, `READ_SDO` and CIP objects.
- Embedded web pages with emergency messages.

NOTE: The most detailed diagnostic information can be found in the web pages.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
LED Diagnostic	130
Device DDT for BMECXM Modules	135
Device DDT for CANopen Slave Devices	138
BMECXM DTM Diagnosis	139
Sending Explicit Messages to BMECXM module.	142
Embedded Web Pages	145
Emergency Objects	151

LED Diagnostic

Overview

LED indicators report the behavior of the module and its communications with the network. LED indicators appear as words or abbreviations at the top of the module.

LED Display

This is the LED display located on the front panel of the BMECXM module:

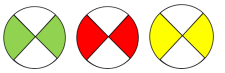


Each LED of the BMECXM module is located with a corresponding positioning letter used in the input parameter `CXM_DISPLAY` of the module device DDT *(see page 135)*:

A = **RUN**	B = **ERR**	C = **I/O**	D = **BS**
E = **CAN RUN**	–	G = **CAN ERR**	–
F = **CAN COM**	–	–	–

LED State and Flash Rates

This table describes the LED state used in the following tables for diagnosing the module:

LED State	Flash Rate	State Symbol
LED off	Constantly OFF	
LED on	Constantly ON	
LED blinking	Iso-phase: ● 200 ms ON ● 200 ms OFF	
LED flashing	One short single flash: ● 200 ms ON ● 1200 ms OFF	
One of the possible patterns	–	

LED Description

This table describes the **RUN**, **ERR**, **I/O**, and **BS** LED states and colors of the BMECXM module:

LED	Color	State	Description
RUN	Green	On	The module is in `RUN` state.
		Off	• There is no power to the module, or • Module configuration failed (see **ERR** LED for identifying the detected error).
		Blinking	• The power on self-test is running, or • A firmware update is in progress (see **BS** LED to confirm).
ERR	Red	On	• Critical error detected while running the power-on self-test (`INITIALIZATION` phase failed), or • Error detected while getting IP address via DHCP (duplicate address).
		Off	No errors detected.
		Blinking	• The power on self-test is running, or • An error detected while retrieving the FDR file.
I/O	Red	Off	Signification is depending on the module status: • If the module is in not in `RUN` state: ❍ A firmware update is in progress (see **BS** LED to confirm). ❍ Module configuration failed (see **ERR** LED for identifying the detected error). • If the module is in `RUN` state, the combination of **I/O** and **CAN ERR** LEDs provides a CANopen diagnostic *(see page 133)*.
		On	The CANopen fieldbus state is `NO-CONF` or `BUS OFF`.
		Blinking	Signification is depending on the module status: • If the module is in not in `RUN` state, the power-on self-test is running. • If the module is in `RUN` state, the combination of **I/O** and **CAN ERR** LEDs provides a CANopen diagnostic *(see page 133)*.
BS (bus status)	–	Off	• Module is not configured, or • Module is waiting for IP address from DHCP.
	Green	On	All EtherNet/IP connections are established.
		Flashing	The module has an IP address, but there is no EtherNet/IP connection.
		Blinking	The power on self-test is in progress.
	Red	On	There is a duplicate IP address.
		Flashing	At least one EtherNet/IP connection is lost. The LED flashes until the connection is re-established or the module is reset.
	Yellow	Blinking	The loading of the firmware is in progress.

This table describes the **CAN RUN**, **CAN ERR**, and **CAN COM** LED states and colors of the CANopen fieldbus:

LED	Color	State	Description
CAN RUN	Green	Off	There is no power to the module.
		On	The CANopen fieldbus state is `OPERATIONAL`.
		Blinking	The CANopen fieldbus state is `PRE-OPERATIONAL`.
		Flashing	The CANopen fieldbus state is `STOPPED`.
CAN ERR	Red	On	• No CANopen device is configured, or • CANopen fieldbus state is `BUS OFF`
		Off	No CANopen error detected.
		Flashing	At least one of the detected error counters has reached or exceeded the warning level (too many error frames).
CAN COM	Yellow	Flashing	There is an SDO message.

General Diagnostic

A general diagnostic of the module is possible when you observe the four upper LEDs (**RUN**, **ERR**, **I/O**, and **BS**) in combination:

LEDs				Condition
 **RUN**	 **ERR**	 **I/O**	 **BS**	Firmware download.
 **RUN**	 **ERR**	 **I/O**	 **BS**	Power is off. `CXM_OP_STATE=IDLE`
 **RUN**	 **ERR**	 **I/O**	 **BS**	Power-on self-test is in progress. `CXM_OP_STATE=INITIALIZATION`
 **RUN**	 **ERR**	 **I/O**	 **BS**	The power on self-test failed. `CXM_OP_STATE=INITIALIZATION`
 **RUN**	 **ERR**	 **I/O**	 **BS**	The power on self-test is completed and the module proceed to: • Backplane initialization, • Get IP address (from DHCP), • Get configuration file (from FDR server). `CXM_OP_STATE=UNCONFIGURED`

(1) The status of the **I/O** LED in combination with **CAN ERR** LED gives information on I/O exchange with CANopen devices *(see page 133)*.
(2) In case of duplicate IP address, the LED **BS** is blinking at the start-up phase then the module restarts.

LEDs				Condition
 RUN	 ERR	 I/O	 BS	Error detected while getting IP address via DHCP (duplicate address(2)). `CXM_OP_STATE=UNCONFIGURED`
 RUN	 ERR	 I/O	 BS	Valid IP address but no EtherNet/IP connection. `CXM_OP_STATE=CONFIGURED`
 RUN	 ERR	 I/O(1)	 BS	EtherNet/IP connections established. `CXM_OP_STATE=CONNECTED STOP, CONNECTED RUN`, or `FALLBACK`
 RUN	 ERR	 I/O(1)	 BS	EtherNet/IP connections are closed. `CXM_OP_STATE=CONNECTED STOP, CONNECTED RUN`, or `FALLBACK`
 RUN	 ERR	 I/O(1)	 BS	Detected communication failure. `CXM_OP_STATE=CONNECTED STOP, CONNECTED RUN`, or `FALLBACK`
(1) The status of the **I/O** LED in combination with **CAN ERR** LED gives information on I/O exchange with CANopen devices *(see page 133)*. **(2)** In case of duplicate IP address, the LED **BS** is blinking at the start-up phase then the module restarts.				

CANopen LED Diagnostic

The module in `RUN` state is a prerequisite to diagnose the I/O exchange with CANopen devices with the LEDs.

The following tables give diagnostic when you observe the **I/O** and **CAN ERR** LEDs in combination:

LEDs				Condition
 RUN	 ERR	 I/O	 BS	No device configured, or CANopen bus is off (physical wire is bus off).
 CAN RUN		 CAN ERR		
 CAN COM				

LEDs				Condition
 RUN	 ERR	 I/O	 BS	Configuration detected error, communication detected error on CANopen device or CANopen device absent on bus.
 CAN RUN		 CAN ERR		
 CAN COM				

LEDs				Condition
 RUN	 ERR	 I/O	 BS	No error detected.
 CAN RUN		 CAN ERR		
 CAN COM				

Device DDT for BMECXM Modules

Overview

Use the device derived data type (DDDT) for diagnosis. There is one device DDT for each BMECXM module.

The device DDT, contains:

- Input parameters
- Other parameters

NOTE: To access these parameters, refer to device DDT Variables *(see page 116)*.

Input Parameters

Parameter	Type	Bit	Description
DEVICE_NAME	STRING [16]		Device name of the CXM module
CXM_OP_STATE	BYTE		Module state operating mode: **0** = INITIALIZATION **1** = UNCONFIGURED **2** = CONFIGURED **3** = CONNECTED STOP **4** = CONNECTED RUN **5** = FALLBACK
CXM_REDUND_STATE	BYTE		Reserved
FB_STATE	BYTE		Fieldbus operating mode: **0** = IDLE **1** = NO-CONF **2** = BUS OFF **3** = STOPPED **4** = PRE-OPERATIONAL **5** = OPERATIONAL **6** = CLEAR
FB_HEALTH	BYTE		Status information of the network manager for diagnosing fieldbus: **0** = Idle **1** = Fieldbus error detected **2** = Device fault detected **3** = Device error detected **4** = Device fault detected and error detected
SLAVE_PROG_LIST	ARRAY [0...15] OF BYTE		Slave list for programmed state. 1 bit per slave device.
SLAVE_LIVE_LIST	ARRAY [0...15] OF BYTE		Slave list for responding state. 1 bit per slave device.

Parameter	Type	Bit	Description
SLAVE_DIAG_LIST	ARRAY [0...15] OF BYTE		Slave list for error state. 1 bit per slave device.
SLAVE_WAIT_LIST	ARRAY [0...15] OF BYTE		Slave list that indicates if the device is waiting for explicit order to operate or not. 1 bit per slave device.
FB_MAX_SCAN	UDINT		Maximal time period of field devices scanning (by 100 µs resolution).
FB_LAST_SCAN	UDINT		Last time period of field devices scanning (by 100 µs resolution).
FB_MIN_SCAN	UDINT		Minimal time period of field devices scanning (by 100 µs resolution).
CXM_DISPLAY	UINT		8 LEDs A, B, C, D, E, F, G, and H located in 2 bits (high, low): **A = RUN:** bits (1, 0) **B = ERR:** bits (3, 2) **C = I/O:** bits (5, 4) **D = BS:** bits (7, 6) **E = CAN RUN:** bits (9, 8) **F = CAN COM:** bits (11, 10) **G = CAN ERR:** bits (13, 12) **H = Not used:** bits (15, 14) **Off** bit high = 0 and bit low = 0 **Green** bit high = 0 and bit low = 1 **Red** bit high = 1 and bit low = 0 **Yellow** bit high = 1 and bit low = 1
ETH_STATUS	BYTE		Status information for Ethernet:
		0	**PORT1_LINK**: • 0: Link down for Ethernet port 1 • 1: Link up for Ethernet port 1
		4	**RPI_CHANGE**: • 0: EtherNet/IP RPI not in progress • 1: EtherNet/IP RPI in progress
		5	**REDUNDANCY_STATUS**: • 0: Backup path not available • 1: Backup path available
		6	**REDUNDANCY_OWNER**: • 0: Redundant owner not present • 1: Redundant owner present
		7	**GLOBAL_STATUS**: • 0: One or more services not operating normally • 1: All services operating normally

Parameter	Type	Bit	Description
SERVICE_STATUS	BYTE		Status information of Ethernet services:
		1	**SNTP_SERVICE**: ● 0: Service not operating normally ● 1: Service operating normally or disabled
		3	**SNMP_SERVICE**: ● 0: Service not operating normally ● 1: Service operating normally or disabled
		4	**FDR_SERVICE**: ● 0: Unable to download PRM file ● 1: Service operating normally or disabled
		5	**FIRMWARE_UPGRADE** ● 0: Firmware upgrade unauthorized ● 1: Service operating normally
		6	**WEB_PAGE** ● 0: Webpage not available ● 1: Service operating normally or disabled
		7	**EVENT_LOG_STATUS** ● 0: Service not operating normally ● 1: Service operating normally or disabled
ETH_PORT_1_AND_2_ STATUS	BYTE		Not applicable.
ETH_PORT_3_STATUS	BYTE		Not applicable.
SYSLOG_STATUS	BYTE		0: Set to 1 if the syslog client does not receive the acknowledgment of the TCP messages from the syslog server.

Other Parameters

Parameter	Type	Description
Freshness	BOOL	Global freshness
Health	BOOL	Global I/O health reports a default in the update of the device DDT due to connection issues: ● 0: when a detected fault is reported by FB_HEALTH in block 1 or a wrong I/O signature is detected ● 1: OK

Device DDT for CANopen Slave Devices

Overview

Use the device derived data type (DDDT) for diagnosis.

There is one device DDT for each CANopen slave device.

The device DDT, contains:

- `HEALTH` parameter
- `Inputs` parameter
- `Outputs` parameter

NOTE: To access these parameters, refer to device DDT Variables *(see page 116)*.

`HEALTH` Parameter

The `HEALTH` byte provides the status of the CANopen slave device:

Bit 3 Wait	Bit 2 Prog	Bit 1 Live	Bit 0 Diag	General Device Status
0	0	0	0	CANopen node not used
0	1	1	0	`OPERATIONAL`
1	1	1	0	`PRE-OPERATIONAL`
0	1	1	1	`ERR` (Configured with fault)
0	1	0	1	`FAULT` (Inoperative)
0	1	0	0	`DISABLE` (Configured)
1	1	0	0	`STOPPED`

Inputs and `Outputs` Parameters

The parameter `Inputs` contains the variables mapped in active PDOs to transmit to the CANopen master.

The parameter `Outputs` contains the variable mapped in active PDOs to receive from the CANopen master.

NOTE: For detailed information on PDOs, refer to chapter *Device Configuration* *(see page 68)*.

BMECXM DTM Diagnosis

Overview

The Unity Pro DTMs provides communication and status informations that is collected at polling intervals. To diagnose the operation of the CANopen application, use the BMECXM DTM.

NOTE: Unity Pro DTMs are online diagnosis. To go online, refer to DTM Connections *(see page 96)*.

Prerequisites

To be able to connect the DTM to the BMECXM module, you have to:

- Enable the access control of the PC that supports the DTM in the **Security** tab *(see page 107)*.
- Declare its IP address as **Source IP Address** in the **Channel Properties** of the Ethernet master DTM screen.

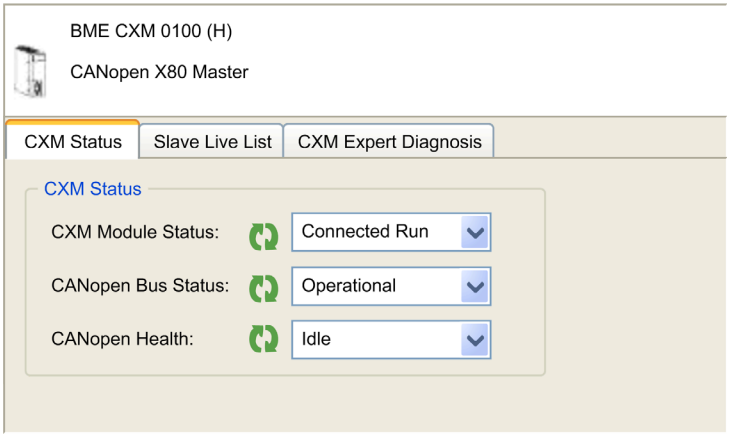
⚠ CAUTION
MISINTERPRETATION OF DIAGNOSIS Make sure that you are connected to the right BMECXM CANopen master module before diagnosing a CANopen slave device. **Failure to follow these instructions can result in injury or equipment damage.**

Connect the DTM

Before you can open the diagnostics page, make the connection between the DTM for the target BMECXM module and the physical module:

Step	Action
1	Open the Unity Pro **DTM Browser** (**Tools → DTM Browser**).
2	Find the name that is assigned to the BMECXM module.
3	Right-click on the module name
4	Scroll to **Connect**.

Accessing the DTM Diagnosis

Step	Action
1	Right-click the name that is assigned to your BMECXM module in the **DTM Browser**.
2	Scroll to **Device Menu → Diagnosis** to view the available diagnostics pages. 

CXM Status Tab

The **CXM Status** tab shows an overview of the current status.

This table describes the status information for the following parameters:

Parameter	Type	Description
CXM Module Status	BYTE	Indicates the status of the module: • **Initialization** • **Unconfigured** • **Configured** • **Connected Stop** • **Connected Run** • **Fallback**
CANopen Bus Status	BYTE	Indicates the status of the CANopen bus: • **Idle** • **No-Conf** • **Bus Off** • **Stopped** • **Pre-Operational** • **Operational** • **Clear**

Parameter	Type	Description
CANopen Health	BYTE	Indicates the status of the fieldbus: • **Idle** • **Fieldbus error detected** • **Device fault detected** • **Device error detected** • **Device fault detected and error detected**

Slave Live List Tab

The **Slave Live List** tab shows the slave diagnosis.

This table describes the color status of the LEDs as shown in the grid of the **Slave Live List** tab:

Color	General Device Status
White	CANopen node not used
Green	`OPERATIONAL`
Orange	`PRE-OPERATIONAL`
Red	`ERR` (Configured with fault)
Half Red/Orange	`FAULT` (Inoperative)
Half Orange/White	`DISABLE` (Configured)
Yellow	`STOPPED`

CXM Expert Diagnosis Tab

The **CXM Expert Diagnosis** tab displays parameters in hierarchical table, grouped by the following sections:

Group	Displays parameters available in ...
Info	DIAG_FXM_Diagnostic (object 301 hex) *(see page 176)*
Status	
EIP Parameters	EIP Interface Diagnostic (object 350 hex) *(see page 183)*
IO Connections	IO Connection Diagnostic (object 352 hex) *(see page 187)*
Explicit Messaging	EtherNet/IP Explicit Connection Diagnostic (object 353 hex) *(see page 190)*
Fieldbus info	DIAG_FXM_Diagnostic (object 301 hex) *(see page 176)*

NOTE: When clicking the **Reset** button, all counter parameters are set to 0.

Sending Explicit Messages to BMECXM module.

Overview

Use the EtherNet/IP Explicit Message window in the Unity Pro DTM to send an explicit message from Unity Pro to the BMECXM module on the network.

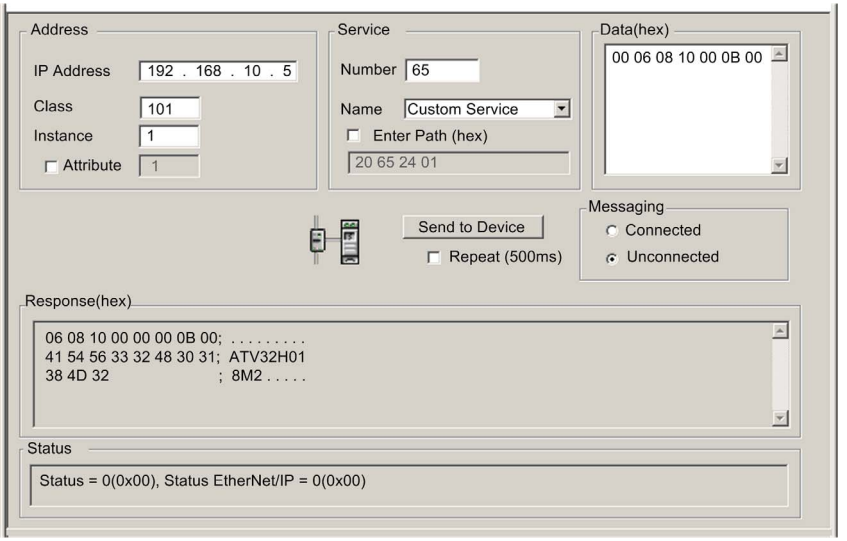
⚠ CAUTION
MISINTERPRETATION OF DIAGNOSIS Make sure that you are connected to the right BMECXM CANopen master module before diagnosing a CANopen slave device. **Failure to follow these instructions can result in injury or equipment damage.**

NOTE: For more details on how to configure EtherNet/IP explicit messages, refer to chapter *Explicit Messaging (see Modicon M580, Hardware, Reference Manual)*.

Explicit Message Example

On manual mode, you can read SDO from the M580 CPU master DTM screen.

To read the `Manufacturer Device Name` of a CANopen slave device follow these steps:

Step	Action
1	From the **DTM Browser**, right-click the DTM master.
2	Select **Device menu → Additional functions → EtherNet/IP Explicit Message**.
3	In the **EtherNet/IP Explicit Message** configuration window, enter or select the following information in these fields: • **IP Address**: The IP address of the BMECXM module • **Class**: 101 (this is the decimal value of the object 65 hex for the `READ_SDO` command.) • **Instance**: 1 • **Name**: Select **Custom Service** then enter to be able to write the Service Number. • **Number**: 65 (this is the decimal value of the service 41 hex for the `READ_SDO` command.) 
4	In the **Data(hex)** field, type the `READ_SDO` command For example, `00 06 08 10 00 0B 00`: **00** The request comes from the PLC **06** This is the node ID of the CANopen slave device (target of the request). **08 10** To read the index 1008 hex which corresponds to the Manufacturer Device Name object. **00** This is the subindex 00 hex of the object. **0B 00** This is the length of the data to read.

Step	Action
5	Click **Send to Device**. **NOTE:** ● In the **Response(hex)** area, data is sent to the configuration tool by the target in the hexadecimal format. ● In the **Status** area, messages indicate whether the explicit message has succeeded or not.
6	Click **Close**.

NOTE: For detailed information on available SDO commands, refer to appendix CANopen SDO commands *(see page 168)*.

Embedded Web Pages

Overview

The BMECXM modules support a set of web pages.

Embedded web pages provide tools to diagnose the basic functionality of the CANopen module through a web browser. These pages display real-time diagnostic data for both the BMECXM module and the CANopen slaves.

NOTE: You have to enable the access control of the PC that is web-connected in the **Security** tab of the BMECXM module *(see page 107)*.

Access

An HTTP server transmits web pages for monitoring and diagnosing the BMECXM module. The server provides easy access to the BMECXM module from standard Internet browsers.

To access the home page, follow these steps:

Step	Action
1	Open an Internet browser: ● Google chrome: version 11 or later ● Mozilla Firefox: version 4 or later ● Internet Explorer: version 8 or later ● Safari: version 5.1.7 or later
2	In the address bar, enter the IP address of the BMECXM module.
3	Press **Enter**.

You can also access web pages through the **Web : Main IP** tab from the CANopen master module screen *(see page 88)*.

NOTE: Web pages are automatically updated every 5 seconds.

Menus

From the **Home** tab, you can access these menus:

Menu	Description
CXM Info/Status	Displays static and status information for the BMECXM module.
EIP Interface	Displays CIP-related diagnostics for the BMECXM module.
IO Connections	Displays diagnostics information on I/O connections between the scanner and the BMECXM module.
CAN Diagnostics	Displays the CAN diagnostics for the BMECXM module.

Menu	Description
CANopen Diagnostics	Displays the CANopen diagnostics.
Slave Details	Displays the list and the status of programmed devices.

NOTE: When clicking the **Reset Counters** button, which is present on some menu pages, all counters are set to 0.

CXM Info/Status Menu

Click **CXM Info/Status** to access this information:

Parameter	Description
LED Displayed	Contains LED indicators. The diagnostics information associated with the LED activity is described in LED Indicators *(see page 130)*. **NOTE:** The **CPU Rate Available** is the percentage of CPU time available.
Name	Gives the device name.
Version	Describes the software and hardware versions that are running on the CANopen module, and the configuration supported.
State	Gives information on: • **CXMOpState**: Module state operating mode ○ 0: `INITIALIZATION` ○ 1: `UNCONFIGURED` ○ 2: `CONFIGURED` ○ 3: `CONNECTED STOP` ○ 4: `CONNECTED RUN` ○ 5: `FALLBACK` • **CXMFBState**: Status of the CANopen fieldbus ○ 0: `IDLE` ○ 1: `NO-CONF` ○ 2: `BUS OFF` ○ 3: `STOPPED` ○ 4: `PRE-OPERATIONAL` ○ 5: `OPERATIONAL` ○ 6: `CLEAR` • **CXMRedundState**: Reserved

Parameter	Description
Ethernet	Gives information on: ● **Ethernet Status** ❍ Bit 0: Linkup/down for Ethernet port 1 ❍ Bit 4: EtherNet/IP RPI in progress ❍ Bit 5: Redundancy status/backup path available ❍ Bit 6: Redundant owner available ❍ Bit 7: Global service status ● **Ethernet Service** ❍ Bit 0: Reserved ❍ Bit 1: SNTP ❍ Bit 2: Reserved for port 502 ❍ Bit 3: FDR ❍ Bit 4...7: Reserved
IP Address	Indicates the IP address, subnet mask, default gateway, and MAC address.

EIP Interface Menu

Click **EIP Interface** to access this information:

Parameter	Description
EIP Interface	Displays the protocol supported and diagnostic information on: ● CIP and current CIP ● CIP errors detected ● Counter ● Error detected counters ● Message counter ● Priority rate

IO Connections Menu

Click **IO Connections** to access this information:

Parameter	Description
IO Connections	• **Configured CXM Watchdog**: the timeout of input reception to switch in FALLBACK state • Production and consumption connection ID • Production and consumption RPI • Production and consumption API • Production and consumption connection parameters
Explicit Messaging	• **Number of explicit connected**: maximum instance number of the object • Originator connection ID and originator IP • **Msg Send Counter**: incremented each time a class 3 CIP message is sent on the connection. • **Msg Receive Counter**: incremented each time a class 3 CIP message is received on the connection. **NOTE:** When clicking the **Next EM** button, it displays the next explicit message.
Field Bus Info	Displays operating modes: • **CXMFBMaxScan**: maximum period in the scanning time of field devices (in ms) • **CXMFBLastScan**: last period in the scanning time of field devices (in ms) • **CXMFBMinScan**: minimum period in the scanning time of field devices (in ms) • **CXMFBBandwidth**: percentage of the fieldbus cycle that is consumed to manage the data exchange • **Pending Acyclic request**: number of received explicit requests that are not already processed.

CAN Diagnostics Menu

Click **CAN Diagnostics** to access this information:

Parameter	Description
Tx	Indicates the number of: • Bytes transmitted • Frames transmitted per second
Rx	Indicates the number of: • Bytes received • Frames received per second
Counter CAN	Gives information on: • **Overrun**: reception buffer overrun counter. It displays the minimum number of frames lost. • **Errors**: CAN transmission or reception error detected counter • **Bus OFF**: CAN controller bus off status counter • **Baudrate**: rate of transmission (in Kbits/s) • **Bus load**: minimum, current, and maximum network load

CANopen Diagnostics Menu

Click **CANopen Diagnostics** to access this information:

Parameter	Description
SYNC ID	Identification number for the synchronization object
SYNC Period	Transmission period of the synchronization object
CXMFbHealth	Gives the network manager status for diagnosing fieldbus: • 0: Idle • 1: Fieldbus error detected • 2: Device fault detected • 3: Device error detected • 4: Device fault detected and error detected
Number of Equipments	Amount of equipments in the configuration.
Total Input Bytes	Number of bytes mapped as input.
Total Output Bytes	Number of bytes mapped as output.
Error Emcy_10xx	Generic detected error count: Number of received emergency messages with code 10xx hex.
Error Emcy_50xx	Device hardware detected error count: Number of received emergency messages with code 50xx hex.
Error Emcy_60xx	Device software detected error count: Number of received emergency messages with code 60xx hex.
Error Emcy_81xx	Communication detected error count: Number of received emergency messages with code 81xx hex.
Error Emcy_82xx	Protocol detected error count: Number of received emergency messages with code 82xx hex.
Error Emcy_90xx	External detected error count: Number of received emergency messages with code 90xx hex.
Error Emcy_FFxx	Specific to the device: Number of received emergency messages with code FFxx hex.

Slave Details Menu

Click **Slave Details** to access the CANopen slave states information. Each device is symbolized by a rectangle colored according to its status.

Color	CANopen Slave Status
White	CANopen node not used (or device not configured)
Green with ✓	`OPERATIONAL`
Orange	`PRE-OPERATIONAL`
Red	`ERR` (Device configured with fault)
Brown	`FAULT` (Device Inoperative)
Blue	`DISABLE` (Device configured but disabled)
Yellow	`STOPPED`
Grey	Webserver is offline

The letter **E** signals an emergency and/or SDO error for the slave device. Click the device to access the detailed information:

Parameter	Description
Device Index	Give the index of the device number among the list of programmed CANopen slave devices.
Emergency Messages Counter	Displays the emergency messages counter for the device.
Emergency	The equipment has emitted an emergency message signaling an event. For more information, refer to Emergency Objects *(see page 151)*. The table gives the 4 most recent messages with code, description and the time when the event occurs.
Event History	Gives the list of error detected during SDO transfer. For more information, refer to CANopen SDO command *(see page 168)*.
NOTE: When clicking the **Reset** button, the **Emergency Messages Counter** is set to 0 and the lists (**Emergency** and **Event History**) are cleared. In addition the letter **E** is removed from the CANopen slave device symbolization.	

Emergency Objects

Overview

Emergency objects (EMCY) of CANopen communication have been defined for diagnostic applications. EMCY objects can be accessed explicitly from the application using READ_SDO.

The COB-ID of these objects contain the identity of the node of the device that produced the emergency message. The COB-ID of EMCY objects are constructed in the following manner:

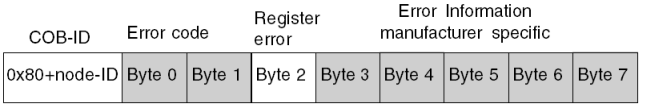
$$COB\text{-}ID_{EMCY} = 0x80 + \text{node identity}$$

Structure

The data field of an EMCY object is composed of 8 bytes containing:

- Emergency detected error code (2 bytes),
- The detected error register (1 byte),
- The factory-specific error information (5 bytes).

This illustration shows the structure of an EMCY object:



You can consult the 4 last emergency messages received in chronological order in the **Device list → Event History** menu of the webpages *(see page 150)*.

NOTE: Regarding Safety considerations, “Emergency objects” and “Fatal error” are mentioned in this manual in conformance with the definition from the DS301 document of the CiA (CAN in Automation).

The contents of the detected error code and error register are specified by CiA.

Detected Error Code 00xx

This table describes the content of detected error code 00xx:

Detected error code (hex)	Description
00xx	Error reset to zero or no error

Detected Error Code 10xx

This table describes the content of detected error code 10xx:

Detected error code (hex)	Description
10xx	Generic error

Detected Error Code 2xxx

This table describes the content of detected error code 2xxx:

Detected error code (hex)	Description
20xx	Current
21xx	Current, input side of the device
22xx	Internal current to the device
23xx	Current, output side of the device

Detected Error Code 3xxx

This table describes the content of detected error code 3xxx:

Detected error code (hex)	Description
30xx	Voltage
31xx	Principal voltage
32xx	Internal voltage to the device
33xx	Output voltage

Detected Error Code 4xxx

This table describes the content of detected error code 4xxx:

Detected error code (hex)	Description
40xx	Temperature
41xx	Ambient temperature
42xx	Device temperature

Detected Error Code 50xx

This table describes the content of detected error code 50xx:

Detected error code (hex)	Description
50xx	Device hardware

Detected Error Code 6xxx

This table describes the content of detected error code 6xxx:

Detected error code (hex)	Description
60xx	Device software
61xx	Internal software
62xx	User software
63xx	Data set

Detected Error Code 70xx

This table describes the content of detected error code 70xx:

Detected error code (hex)	Description
70xx	Additional modules

Detected Error Code 8xxx

This table describes the content of detected error code 8xxx:

Detected error code (hex)	Description
80xx	Monitoring
81xx	Communication
8110	CAN overflow (objects lost)
8120	CAN in passive error mode
8130	Life Guard error or Heartbeat error
8140	Recovered from bus
8150	Collision during COB-ID transmission
82xx	Protocol error
8210	PDO not processed due to length error
8220	PDO length exceeded

Detected Error Code 90xx

This table describes the content of detected error code 90xx:

Detected error code (hex)	Description
90xx	External error

Detected Error Code Fxxx

This table describes the content of detected error code Fxxx:

Detected error code (hex)	Description
F0xx	Additional functions
FFxx	Specific to the device

Chapter 9
Firmware Upgrade

Firmware Update

Overview

You can update the firmware for the BMECXM module by downloading a new firmware version with Unity Loader. The minimum version for Unity Loader is V11.0.

The firmware download can be performed by connecting to the Ethernet network.

Refer to the Unity Loader manual for a description of the download procedure (see *Unity Loader, a SoCollaborative Software, User Manual*).

Password

A firmware password is set in Unity Pro and sent to each BMECXM firmware. To access Unity Pro password in the **Project Browser**, right-click **Project → Properties of Project → Protection**.

To perform the update, the password set in Unity Loader has to match the password set in Unity Pro.

Preparation

Before performing the firmware update:

- Check that the FTP service *(see page 107)* is enabled.
- Stop the PLC.
- Open Unity Loader on your PC. (**Start → Programs → Schneider Electric → Unity Loader**)

⚠ WARNING
UNKNOWN OPERATIONAL STATE OF EQUIPMENT
Evaluate operational state of equipment before stopping the PLC.
Failure to follow these instructions can result in death, serious injury, or equipment damage.

NOTE: If you do not stop the PLC before trying to transfer firmware, you are informed by Unity Loader that the PLC must be stopped. After confirming this message, Unity Loader stops the PLC automatically.

Appendices

Overview

These appendices contain information that should be useful for programming the application.

What Is in This Appendix?

The appendix contains the following chapters:

Chapter	Chapter Name	Page
A	CANopen Master Local Object Dictionary Entry	159
B	CANopen Commands	167
C	CIP Objects	175

Appendix A
CANopen Master Local Object Dictionary Entry

Subject of This Chapter

This chapter contains the local object dictionary entry for CANopen Master.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
Object Dictionary Entries According to Profile DS301	160
Object Dictionary Entries According to Profile DS302	163
BMECXM Manufacturer Specific Object Dictionary Entries	165

Object Dictionary Entries According to Profile DS301

Object Dictionary Entries

The table below presents the object dictionary entries according to profile DS301:

Index (hex)	Object Name	Subindex (hex)	Description	Data Type	Comments
1000	Device type	00	Type of device	Unsigned32	000F 0191 hex
1001	Error register	00	Bit 0 indicates a generic error.	Unsigned8	–
1005	COB-ID SYNC	00	Define COB-ID of the synchronisation object (SYNC)	Unsigned32	–
1006	Communication Cycle Period	00		Unsigned32	–
1007	Synchronous window length	00		Unsigned32	–
1008	Manufacturer device name	00		Visible string(15)	BME CXM 0100
1009	Manufacturer hardware version	00		Visible string(15)	Current hardware revision from V1.2.0.1
100A	Manufacturer software version	00		Visible string(15)	Current software revision from V1.0
1012	COB-ID time stamp message	–	Define COB-ID of the time stamp object (TIME)	Unsigned32	–
1016	Consumer Heartbeat Time	00	Largest Subindex supported Number of entries: 64	Unsigned8	40 hex
		01...40	The consumer heartbeat time defines the expected heartbeat cycle time and has to be higher than the corresponding producer heartbeat time (multiple of 1 ms).	Unsigned32	Node ID + Heartbeat Time: ● Bits 31-24: reserved ● Bits 23-16: Node ID ● Bits 15-0: Heartbeat Time
1017	Producer Heartbeat Time	00	The producer heartbeat time defines the cycle time of the heartbeat (multiple of 1 ms).	Unsigned16	–

Index (hex)	Object Name	Subindex (hex)	Description	Data Type	Comments
1018	Identity object	00	Number of entries	Unsigned8	04 hex
		01	Vendor ID	Unsigned32	0600 005A hex
		02	Product code	Unsigned32	081C xxxx hex
		03	Revision number	Unsigned32	0001 xxxx hex
		04	Serial number	Unsigned32	–
1020	Verify configuration	00	Number of entries	Unsigned8	02 hex
		01	Configuration date	Unsigned32	–
		02	Configuration time	Unsigned32	–
102A	NMT inhibit time	00		Unsigned16	–
1200	Server SDO Parameter	00	Number of entries	Unsigned8	02 hex
		01	COB-ID client -> server (Rx)	Unsigned32	600 hex + Node-ID
		02	COB-ID server -> client (Tx)	Unsigned32	580 hex + Node-ID
1280 ... 1282	Client SDO Parameter 1...3	00	Number of entries	Unsigned8	–
		01	COB-ID client -> Server (Rx)	Unsigned32	
		02	COB-ID server -> Client (Tx)	Unsigned32	
		03	Node-ID of the SDO server	Unsigned8	
1400 ... 14FF	Receive PDO parameter 1...256	00	Largest subindex supported	Unsigned8	–
		01	COB-ID used by PDO	Unsigned32	
		02	Transmission type	Unsigned8	
		03	–	Unsigned16	
		04	–	Unsigned8	
		05	Event timer	Unsigned16	

Index (hex)	Object Name	Subindex (hex)	Description	Data Type	Comments
1600 ... 16FF	Receive PDO mapping 1...256	00	Number of mapped application objects in PDO	Unsigned8	Depends on PDO mapping of the application
		01	PDO mapping for the first application object to be mapped	Unsigned32	Index (16 bit) | subindex (8 bit) | length (8 bit)
		02	PDO mapping for the second application object	Unsigned32	–
		.....	–	–	–
		08	PDO mapping for the eighth application object	Unsigned32	–
1800 ... 18FF	Transmit PDO parameter 1...256	00	Largest subindex supported	Unsigned8	–
		01	COB-ID used by PDO	Unsigned32	
		02	Transmission type	Unsigned8	
		03	Inhibit time	Unsigned16	
		04	Reserved	Unsigned8	
		05	Event timer	Unsigned16	
1A00 ... 1AFF	Transmit PDO mapping 1...256	0	Number of mapped application objects in PDO	Unsigned8	Depends on PDO mapping of the application
		1	PDO mapping for the first application object to be mapped	Unsigned32	Index (16 bit) | subindex (8 bit) | length (8 bit)
		2	PDO mapping for the second application object	Unsigned32	–
		.....	–	–	–
		8	PDO mapping for the eighth application object	Unsigned32	–

Object Dictionary Entries According to Profile DS302

Object Dictionary Entries

The table below presents the object dictionary entries according to profile DS302.

Index (Hex)	Subindex	Description	Object type	Data type
1F22	–	Concise DCF	ARRAY	–
	0	Number of entries	VAR	Unsigned8
	1	Device with Node-ID 1	VAR	DOMAIN
	...	–	–	–
	127	Device with Node-ID 127	–	DOMAIN
1F26	–	Expected configuration date	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F27	–	Expected configuration time	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F80	–	NMT startup	VAR	Unsigned32
1F81	...	Slave assignment	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F82	...	Request NMT	ARRAY	–
	0	Number of entries		Unsigned8
	1	Request NMT for Node-ID 1		Unsigned8
	...	–		–
	128	Request NMT for all nodes		Unsigned8

Index (Hex)	Subindex	Description	Object type	Data type
1F84	...	Device type identification	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F85	...	Vendor identification	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F86	...	Product code	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F87	...	Revision number	ARRAY	–
	0	Number of entries		Unsigned8
	1	Device with Node-ID 1		Unsigned32
	...	–		–
	127	Device with Node-ID 127		Unsigned32
1F8A	–	Restore configuration	ARRAY	–
	0	Number of entries		Unsigned8
	1	Restore for Node-ID 1		Unsigned8
	...	–		–
	64	Restore for Node-ID 64		Unsigned8

BMECXM Manufacturer Specific Object Dictionary Entries

Boot Slave Control Reset

The table below presents the object entry 4210:

Index (Hex)	Subindex	Description	Object type	Data type
4210	–	Boot slave control reset	ARRAY	–
	0	Number of entries		Unsigned8
	1	Reset for Node-ID 1(1)		Unsigned8
	...	–		–
	64	Reset for Node-ID 64(1)		Unsigned8

(1) Data = 0: No reset
Data = 1: Reset all parameters (by default)
Data = 2: Reset communication parameters only (1000 hex-1FFF hex)
Data >2: Not used

Boot Slave Control Start

The table below presents the object entry 4211:

Index (Hex)	Subindex	Description	Object type	Data type
4211	–	Boot slave control start	ARRAY	–
	0	Number of entries		Unsigned8
	1	Start for Node-ID 1(1)		Unsigned8
	...	–		–
	64	Start for Node-ID 64(1)		Unsigned8

(1) Data = 0: No start
Data = 1: Start all parameters (by default)
Data >1: Not used

Force Download

The table below presents the object entry 4212:

Index (Hex)	Subindex	Description	Object type	Data type
4212	–	Force download	ARRAY	–
	0	Number of entries		Unsigned8
	1	Type of force download for node 1(1)		Unsigned8
	...	–		–
	64	Type of force download for node 64(1)		Unsigned8
(1) Data = 0: No forcing (by default) Data = 1: Force download of communication parameters Data = 2: Force download of application parameters Data >2: Not used				

Global SDO Timeout

The table below presents the object entry 5FF0:

Index (Hex)	Subindex	Description	Object type	Data type
5FF0	–	Global SDO timeout	VAR	Unsigned16

Slave Specific SDO Timeout

The table below presents the object entry 5FF1:

Index (Hex)	Subindex	Description	Object type	Data type
5FF1	–	Slave specific SDO timeout	ARRAY	–
	0	Number of entries		Unsigned8
	1	Timeout for Node-ID 1		Unsigned16
	...	–		–
	64	Timeout for Node-ID 64		Unsigned16

Appendix B
CANopen Commands

Subject of This Chapter

This chapter defines CANopen commands that are specific to BMECXM modules.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
CANopen SDO Commands	168
CANopen SDO Abort Code	170
CANopen Start Command	172
CANopen Slave Enabling Command	173

CANopen SDO Commands

Overview

`WRITE_SDO` and `READ_SDO` objects are used to send NMT commands. The SDO abort code is used when the SDO command is not successful.

`WRITE_SDO` Command

This table shows the header of the `WRITE_SDO` command:

Value (hex)	CIP generic message parameter
65	Class ID
40	Service Number
1	Instance
x	Length

This table shows the command data and the response data for the `WRITE_SDO`:

Value	Size	Parameter
Command data		
0: PLC 1...4: DTM	SINT	Connect ID
[1...127]	SINT	Node ID
User defined	INT	Index
User defined	SINT	Subindex
[1...255]	INT	Length
User defined	SINT[...]	Data
Positive response data		
[1...127]	SINT	Node ID
User defined	INT	Index
User defined	SINT	Subindex
0	INT	Status
Negative response data		
[1...127]	SINT	Node ID
User defined	INT	Index
User defined	SINT	Subindex
≠0 *(see Unity Pro, Communication, Block Library)*	SINT	Status
See table *(see page 170)*	SINT[4]	SDO abort code

READ_SDO Command

This table shows the header of the READ_SDO command:

Value (hex)	CIP generic message parameter
65	Class ID
41	Service Number
1	Instance
x	Length

This table shows the command data and the response data for the READ_SDO:

Value	Size	Parameter
Command data		
0: PLC 1...4: DTM	SINT	Connect ID
[1...127]	SINT	Node ID
User defined	INT	Index
User defined	SINT	Subindex
[1...255]	INT	Length
Positive response data		
[1...127]	SINT	Node ID
User defined	INT	Index
User defined	SINT	Subindex
0	INT	Status
User defined	INT	Length
Value requested	SINT[...]	Object value
Negative response data		
0x2B	SINT	Node ID
User defined	INT	Index
User defined	SINT	Subindex
≠0 *(see Unity Pro, Communication, Block Library)*	SINT	Status
See table *(see page 170)*	SINT[4]	SDO abort code

CANopen SDO Abort Code

Overview

The SDO abort code is used when the SDO command is not successful.

SDO Abort Code

SDO Abort Code Value (hex)	Parameter
0503 0000	Toggle bit not alternated.
0504 0000	SDO protocol timed out.
0504 0001	Client/server command specifier not valid or unknown.
0504 0002	Invalid block size (block mode only).
0504 0003	Invalid sequence number (block mode only).
0504 0004	CRC error (block mode only).
0504 0005	Out of memory.
0601 0000	Unsupported access to an object.
0601 0001	Attempt to read a write only object.
0601 0002	Attempt to write a read only object.
0602 0002	Object does not exist in the object dictionary.
0604 0041	Object cannot be mapped to the PDO.
0604 0042	The number and length of the objects to be mapped would exceed PDO length.
0604 0043	General parameter incompatibility reason.
0604 0047	General internal incompatibility in the device.
0606 0000	Access failed due to an hardware error
0607 0010	Data type does not match, length of service parameter does not match
0607 0012	Data type does not match, length of service parameter too high
0607 0013	Data type does not match, length of service parameter too low
0609 0011	Sub-index does not exist.
0609 0030	Value range of parameter exceeded (only for write access).
0609 0031	Value of parameter written too high.
0609 0032	Value of parameter written too low.
0609 0036	Maximum value is less than minimum value.
0800 0000	General error
0800 0020	Data cannot be transferred or stored to the application.
0800 0021	Data cannot be transferred or stored to the application because of local control.

SDO Abort Code Value (hex)	Parameter
0800 0022	Data cannot be transferred or stored to the application because of the present device state.
0800 0023	Object dictionary dynamic generation fails or no object dictionary is present (for example, object dictionary is generated from file and generation fails because of an file error).

CANopen Start Command

Overview

The `EM_Start` command is used to synchronize the start of the BMECXM module.

NOTE: This command is valid in manual mode only. For more information, refer to starting mode *(see page 106)*.

`EM_Start` Command

This table shows the header of the `EM_Start` command:

Value (hex)	CIP generic message parameter
66	Class ID
40	Service Number
1	Instance
x	Length

This table shows the command data and the response data for the `EM_Start`:

Value	Size	Parameter
Command data		
–	–	Not applicable
Response data		
[0...1]	SINT	Status: ● 0: accepted ● 1: refused

CANopen Slave Enabling Command

Overview

The `Slave Enable / Disable` command is used to disable a device which is configured or enable a device which as been already disabled by calling this function before.

`Slave Enable / Disable` Command

This table shows the header of the `Slave Enable / Disable` command:

Value (hex)	CIP generic message parameter
67	Class ID
40	Service Number
1	Instance
x	Length

This table shows the command data for the `Slave Enable / Disable`:

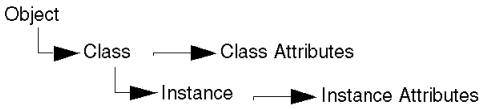
Value	Size	Parameter
Command data		
0...2	SINT	For node ID 1: ● 0: Do not change state ● 1: Enable slave ● 2: Disable slave
...	...	For node n
[0...2]	SINT	For node ID 126: ● 0: Do not change state ● 1: Enable slave ● 2: Disable slave

Appendix C
CIP Objects

Subject of This Chapter

Modicon M580 applications use CIP within a producer/consumer model to provide communication services in an industrial environment. The M580 CPU can access CIP data and services located in connected devices.

CIP object data and content are exposed-and accessed- hierarchically in the following nested levels:



NOTE:
You can use explicit messaging to access these items:

- Access a collection of instance attributes by including only the class and instance values for the object in the explicit message.
- Access a single attribute by adding a specific attribute value to the explicit message with the class and instance values for the object.

This chapter describes the available CIP objects you can use to diagnose the BMECXM module.

What Is in This Chapter?

This chapter contains the following topics:

Topic	Page
`DIAG_FXM_Diagnostic` Object	176
`DIAG_CXM` Object	181
EIP Interface Diagnostic Object	183
I/O Connection Diagnostics Object	187
EtherNet/IP Explicit Connection Diagnostic Object	190

DIAG_FXM_Diagnostic Object

Overview

Basic diagnostic of the CANopen X80 master module can be done via explicit message using the `DIAG_FXM_Diagnostic` object.

The diagnostic object presents the instances, attributes, and services described below.

Class ID

301 hex

Instance IDs

The diagnostic object presents two instance values:

- 0: class
- 1: instance

Attributes

The diagnostic object presents the following attributes.

Instance ID = 0 (class attributes):

Attribute ID (hex)	Type	Description
01	WORD	Version high
02	WORD	Version low
03	WORD	Number of instance

Instance ID = 1 (instance attributes):

Attribute ID (hex)	Parameter	Type	Bit	Description
01	**MAC Address**	DWORD		Ethernet MAC address of the module **NOTE:** only the 4 Least Significant Byte (LSB), for completing the MAC address add 00-80 on the Most Significant Byte (MSB).)
02	PBA version	DWORD		4 bytes for major, minor, intermediate, and release (reserved)
03	Micro FW1 Version	WORD		2 bytes for major, and minor
04	Micro FW2 Version	WORD		2 bytes for major, and minor
05	Firmware Version	DWORD		4 bytes for major, minor, intermediate, and release (reserved)

Attribute ID (hex)	Parameter	Type	Bit	Description
06	Configuration Supported	DWORD		2 bytes for major, and minor
07	Device Name(16)	DWORD		Device name of the module
08	IP address	DWORD		Current Ethernet IPV4 address (format is xxx.xxx.xxx.xxx)
09	Subnet Mask	DWORD		Current Ethernet IPV4 subnet (format is xxx.xxx.xxx.xxx)
0A	Default Gateway	DWORD		Current Ethernet IPV4 Gateway address (format is xxx.xxx.xxx.xxx)
0B	CPU Rate Available	DUINT		Percentage (%) of CPU time available
0C	FxmOpState	BYTE		**0:** `INITIALIZATION` **1:** `UNCONFIGURED` **2:** `CONFIGURED` **3:** `CONNECTED RUN` **4:** `CONNECTED STOP` **5:** `FALLBACK`
0D	FxmRedundState	BYTE		Reserved
0E	FxmDisplay	WORD		2 bits (high, low) per LED: **RUN:** bits (1, 0) **ERR:** bits (3, 2) **I/O:** bits (5, 4) **BS:** bits (7, 6) **CAN RUN:** bits (9, 8) **CAN COM:** bits (11, 10) **CAN ERR:** bits (13, 12) **Off** bit high = 0 and bit low = 0 **Green** bit high = 0 and bit low = 1 **Red** bit high = 1 and bit low = 0 **Yellow** bit high = 1 and bit low = 1

Attribute ID (hex)	Parameter	Type	Bit	Description
0F	Ethernet Status	BYTE		Main Ethernet status:
			0	**PORT1_LINK**: ● 0: Link down for Ethernet port 1 ● 1: Link up for Ethernet port 1
			4	**RPI_CHANGE**: ● 0: EtherNet/IP RPI not in progress ● 1: EtherNet/IP RPI in progress
			5	**REDUNDANCY_STATUS**: ● 0: Backup path not available ● 1: Backup path available
			6	**REDUNDANCY_OWNER**: ● 0: Redundant owner not present ● 1: Redundant owner present
			7	**GLOBAL_STATUS**: ● 0: One or more services not operating normally ● 1: All services operating normally
10	Ethernet Services	BYTE		Detailed Ethernet status:
			1	**SNTP_SERVICE**: ● 0: Service not operating normally ● 1: Service operating normally or disabled
			3	**SNMP_SERVICE**: ● 0: Service not operating normally ● 1: Service operating normally or disabled
			4	**FDR_SERVICE**: ● 0: Unable to download PRM file ● 1: Service operating normally or disabled
			5	**FIRMWARE_UPGRADE** ● 0: Firmware upgrade unauthorized ● 1: Service operating normally
			6	**WEB_PAGE** ● 0: Webpage not available ● 1: Service operating normally or disabled
			7	**EVENT_LOG_STATUS** ● 0: Service not operating normally ● 1: Service operating normally or disabled
11	Syslog_Status	BYTE		
			0	Set to 1 if the syslog client does not receive the acknowledgment of the TCP messages from the syslog server.

Attribute ID (hex)	Parameter	Type	Bit	Description
12	Syslog_Buffer_Free	DUINT		Free place in % of the events buffer.
13	Syslog_Lost_Events	DUINT		Number of events lost since the last restart.
14	FxmFBState	BYTE		Fieldbus operating mode: **0** = IDLE **1** = NO-CONF **2** = BUS OFF **3** = STOPPED **4** = PRE-OPERATIONAL **5** = OPERATIONAL **6** = CLEAR
15	FxmFBHealth	BYTE		Status information of the network manager for diagnosing fieldbus: **0** = Idle **1** = Fieldbus error detected **2** = Device fault detected **3** = Device error detected **4** = Device fault detected and error detected
16	SlavesProgList	BOOL [128]		Slave list for programmed state. 1 bit per slave device: ● 0: Programmed (in configuration file) ● 1: Not expected (not configured or disabled)
17	SlavesLiveList	BOOL [128]		Slave list for responding state. 1 bit per slave device: ● 0: Responding ● 1: Not responding or disabled
18	SlavesDiagList	BOOL [128]		Slave list for error state. 1 bit per slave device: ● 0: Detected error or fault on expected slave ● 1: No reported error
19	SlavesWaitList	BOOL [128]		Slave list that indicates if the device is waiting for explicit order to operate or not. 1 bit per slave device: ● 0: Slave device is waiting for explicit message to operate ● 1: No action required
1A	FxmFBMaxScan	UDINT		Maximal time period of field devices scanning (by 100 µs resolution).
1B	FxmFBLastScan	UDINT		Last time period of field devices scanning (by 100 µs resolution).
1C	FxmFBMinScan	UDINT		Minimal time period of field devices scanning (by 100 µs resolution).

Attribute ID (hex)	Parameter	Type	Bit	Description
1D	Pending Acyclic request	UINT		Number of pending explicit requests.
1F	FxmFB Bandwidth	UINT		Current Fieldbus bus load in %

Service Supported

The DIAG_FXM_Diagnostic object performs the following service:

Service ID (hex)	Service Name	Class	Instance
01	`Get_Attributes_All`	X	X

DIAG_CXM Object

Overview

Diagnostic of the CANopen bus activity can be done via explicit message using the CXM_DIAG object.

The diagnostic object presents the instances, attributes, and services described below.

Class ID

302 hex

Instance IDs

The diagnostic object presents two instance values:

- 0: class
- 1: instance

Attributes

The diagnostic object presents the following attributes.

Instance ID = 0 (class attributes):

Attribute ID (hex)	Type	Description
01	WORD	Version high
02	WORD	Version low
03	WORD	Number of instance

Instance ID = 1 (instance attributes):

Attribute ID (hex)	Type	Description
01	DWORD	Total number of bytes received.
02	DWORD	Number of frames received since the beginning.
03	DWORD	Total number of bytes Transmitted.
04	DWORD	Number of Frames Transmitted since the beginning.
05	DWORD	Reception buffer overrun counter: minimum number of frames lost
06	DWORD	CAN transmission or reception detected error counter. (include all detected error described in error flag of the CAN2.0B protocol)
07	DWORD	Minimum bus load in %
08	DWORD	Current bus load in %
09	DWORD	Maximum bus load in %

Attribute ID (hex)	Type	Description
0A	DWORD	Rate of transmission (Kbits/s).
0B	DWORD	CAN controller Bus Off status counter.
0C	DWORD	0 is no bus off 1 is Bus off
0D	DWORD	Identification number for the SYNC synchronization object.
0E	DWORD	Period Sync ID object.
0F	DWORD	Current number of found error frames in % for last 10000 exchanged frames
10	DWORD	Maximum number of found error frames in %
11	DWORD	Minimum number of error found frames in %
12	DWORD	Generic detected error count: Number of received emergency messages with code 10xx hex
13	DWORD	Device hardware detected error count: Number of received emergency messages with code 50xx hex
14	DWORD	Device software detected error count: Number of received emergency messages with code 60xx hex
15	DWORD	Communication detected error count: Number of received emergency messages with code 81xx hex
16	DWORD	Protocol detected error count: Number of received emergency messages with code 82xx hex
17	DWORD	External detected error count: Number of received emergency messages with code 90xx hex
18	DWORD	Device-specific: Number of received emergency messages with code FFxx hex
19	DWORD	Maximum number of TPDOs to transmit during one cycle
1A	DWORD	Highest used Node ID
1B	DWORD	Number of used RxPDOs
1C	DWORD	Number of used TxPDOs
1D	DWORD	Total number of variables in the input process image.
1F	DWORD	Total number of variables in the output process image.

Service Supported

The CXM_DIAG object performs the following services upon the listed object types:

Service ID (hex)	Service Name	Class	Instance
01	`Get_Attributes_All`	X	X
05	`RESET`	–	X

EIP Interface Diagnostic Object

Overview

The diagnostic object presents the instances, attributes, and services described below.

Class ID

350 hex

Instance IDs

The diagnostic object presents two instance values:

- 0: class
- 1: instance

Attributes

The diagnostic object presents the following attributes.

Instance ID = 0 (class attributes):

Attribute ID (hex)	Type	Description
01	WORD	Version high
02	WORD	Version low
03	WORD	Number of instance

Instance ID = 1 (instance attributes):

Attribute ID (hex)	Parameter	Type	Description
01	Protocol Supported	UINT	Protocol supported
02	Connection DIAG	Structure of:	
	Max CIP IO Cnx Opened	UINT	Maximum number of CIP IO connections opened
	Current CIP IO Cnx	UINT	Number of CIP IO connections currently opened
	Max CIP Explicit Cnx Opened	UINT	Maximum number of CIP explicit connections opened
	Current CIP Explicit Cnx	UINT	Number of CIP explicit connections currently opened
	CIP Cnx Explicit opening Errors	UINT	Incremented at each attempt to open a CIP connection that fails
	CIP Cnx Timout Errors	UINT	Incremented when a CIP connection is timed out
	Max EIP TCP Cnx Opened	UINT	Maximum number of TCP connection opened and used for EIP communication
	Current EIP TCP Cnx Opened	UINT	Number of TCP connections currently opened and used for EIP communication
03	IO Messaging DIAG	Structure of:	
	IO Prod Counter	UDINT	Incremented each time a Class 0/1 CIP message is sent
	IO consumption Counter	UDINT	Incremented each time a Class 0/1 CIP message is received
	IO prod send Errors Counter	UINT	Incremented each Time a Class 0/1 message is not sent
	IO consumption Receive Errors Counter	UINT	Incremented each time a consumption is received with an error
04	Explicit Messaging DIAG	Structure of:	
	Class3 Msg Send counter	UDINT	Incremented each time a Class 3 CIP message is sent
	Class3 Msg Rec counter	UDINT	Incremented each time a Class 3 CIP message is received
	UCMM Msg Send counter	UDINT	Incremented each time an UCMM message is sent
	UCMM Msg Receive counter	UDINT	Incremented each time an UCMM message is received

Attribute ID (hex)	Parameter	Type	Description
05	COM Capacity	Structure of:	
	Capacity Max CIP Cnx	UINT	Max supported CIP connections
	Capacity Max TCP Cnx	UINT	Max supported TCP connections
	Capacity Max Urgent priority rate	UINT	Max CIP transport class 0/1 urgent priority messages packets/s
	Capacity Max Scheduled priority rate	UINT	Max CIP transport class 0/1 scheduled priority messages packets/s
	Capacity Max High priority rate	UINT	Max CIP transport class 0/1 high priority messages packets/s
	Capacity Max Low priority rate	UINT	Max CIP transport class 0/1 low priority messages packets/s
	Capacity Max Explicit rate	UINT	Max CIP transport class 2/3 or other EIP messages packets/s
06	Bandwidth Diag	Structure of:	
	Current sending Urgent priority rate	UINT	CIP transport class 0/1 urgent priority messages packets/s sent
	Current recept Urgent priority rate	UINT	CIP transport class 0/1 urgent priority messages packets/s received
	Current sending Scheduled priority rate	UINT	CIP transport class 0/1 scheduled priority messages packets/s sent
	Current recept Scheduled priority rate	UINT	CIP transport class 0/1 scheduled priority messages packets/s received
	Current sending High priority rate	UINT	CIP transport class 0/1 high priority messages packets/s sent
	Current recept High priority rate	UINT	CIP transport class 0/1 high priority messages packets/s received
	Current sending Low priority rate	UINT	CIP transport class 0/1 low priority messages packets/s sent
	Current recept Low priority rate	UINT	CIP transport class 0/1 Low priority messages packets/s received
	Current sending Explicit rate	UINT	CIP transport class 2/3 or other EIP messages packets sent
	Current reception Explicit rate	UINT	CIP transport class 2/3 or other EIP messages packets received

Service Supported

The object performs the following services upon the listed object types:

Service ID (hex)	Service Name	Class	Instance
01	Get_Attributes_All	X	X
05	RESET	–	X

I/O Connection Diagnostics Object

Overview

The diagnostic object presents the instances, attributes, and services described below.

Class ID

352 hex

Instance IDs

The diagnostic object presents two instance values:

- 0: class
- 1...256: instances

Attributes

The diagnostic object presents the following attributes.

Instance ID = 0 (class attributes):

Attribute ID (hex)	Type	Description
01	UINT	Revision
02	UINT	Maximum instance

Instance ID = 1 to 256 (instance attributes):

Attribute ID (hex)	Parameter	Type	Description
01	IO connections	Structure of:	
	IO product counter	UDINT	Incremented at each production
	IO consumption counter	UDINT	Incremented at each consumption
	IO product send error	UINT	Incremented each time a production is not sent
	IO Consumption Receive error	UINT	Incremented each time a consumption is received with an error
	CIP Connection TimeOut errors	UINT	Incremented when a connection is timed out
	CIP Connection Opening errors	UINT	Incremented at each attempt to open a connection that fails
	CIP Connection State	UINT	State of the CIP IO connection
	CIP Last error General status	UINT	"General Status" of the last error detected on the connection
	CIP Last error extended status	UINT	"Extended Status" of the last error detected on the connection
	Input Com Status	UINT	Communication Status of the Inputs
	Output comm status	UINT	Communication Status of the Outputs
02	Connection Diag	Structure of:	
	Production Connection ID	UDINT	Connection ID for Production
	Consumption Connection ID	UDINT	Connection ID for Consumption
	Production RPI (conf)	UDINT	RPI for production
	Production API {current)	UDINT	API for production
	Consumption RPI (conf)	UDINT	RPI for consumption
	Production API {current)	UDINT	API for consumption
	Production Connection parameters	UDINT	Connection parameters for production
	Consumption Connection parameters	UDINT	Connection parameters for consumption
	Local IP	UDINT	Description from TI 82 , CIP diag
	Local UDP port	UINT	Description from TI 82 , CIP diag
	Remote IP	UDINT	Description from TI 82 , CIP diag
	Remote UDP port	UINT	Description from TI 82 , CIP diag
	Production Multicast IP	UDINT	Multicast IP used for production
	Consumption Multicast IP	UDINT	Multicast IP used for consumption
	Protocol supported	UINT	Protocol(s) supported on the connection

Service Supported

The object performs the following services upon the listed object types:

Service ID (hex)	Service Name	Class	Instance
01	Get_Attributes_All	X	X
0E	Get_Attribute_Single	–	X
4C	Get_and_Clear	–	X
X supported – Not supported			

EtherNet/IP Explicit Connection Diagnostic Object

Overview

The EtherNet/IP explicit diagnostic object presents the instances, attributes, and services described below.

Class ID

353 hex

Instance IDs

The diagnostic object presents two instance values:

- 0: class
- 1...N: instance (N = maximum concurrent number of explicit connections)

Attributes

The diagnostic object presents the following attributes.

Instance ID = 0 (class attributes):

Attribute ID (hex)	Type	Description
01	UINT	Revision
02	UINT	Maximum instance

Instance ID = 1 (instance attributes):

Attribute ID (hex)	Parameter	Type	Description
01	Originator connection ID	UDINT	O to T Connection ID
02	Originator IP	UINT	–
03	Originator TCP port	UDINT	–
04	Target Connection ID	UDINT	T to O Connection ID
05	Target IP	UDINT	–
06	Target TCP port	UDINT	–
07	Msg Send Counter	UDINT	Incremented each time a Class 3 CIP message is sent on the connection.
08	Msg Receive Counter	UDINT	Incremented each time a Class 3 CIP message is received on the connection.

Service Supported

The object performs the following services upon the listed object types:

Service ID (hex)	Service Name	Class	Instance
01	Get_Attributes_All	X	X

Glossary

A

ARRAY

Table containing elements of a single type. This is the syntax: `ARRAY [<limits>] OF <Type>`. For example:

- `ARRAY [1..2] OF BOOL` is a one-dimensional table with two elements of type `BOOL`.
- `ARRAY [1..10, 1..20] OF INT` is a two-dimensional table with 10x20 elements of type `INT`.

asset management

A software application that can configure, monitor, and manage devices used as part of an industrial automation system..

AUX

(*auxiliary*) Optional, periodic processor task that is run through its programming software. The AUX task is used to execute a part of the application requiring a low priority. This task is executed only if the MAST and FAST tasks have nothing to execute. The AUX task has two sections:

- IN: Inputs are copied to the IN section before execution of the AUX task.
- OUT: Outputs are copied to the OUT section after execution of the AUX task.

B

BOOL

(*boolean type*) This is the basic data type used in computing. A `BOOL` variable can have either of these values: 0 (`FALSE`) or 1 (`TRUE`). A bit extracted from a word is of type `BOOL`, for example: `%MW10.4`.

C

CAN

(*Controller area network*) Fieldbus originally developed for automobile applications and now used in many sectors.

CiA

(*CAN in Automation*) International organization of users and manufacturers of CAN devices.

CIP™

(*common industrial protocol*) A comprehensive suite of messages and services for the collection of manufacturing automation applications (control, safety, synchronization, motion, configuration, and information). CIP allows you to integrate these manufacturing applications with enterprise-level Ethernet networks and the Internet. CIP is the core protocol of EtherNet/IP.

class 3 connection

A CIP transport class 3 connection used for explicit messaging between EtherNet/IP devices.

COB-ID

(*communication object identifier*) Unique identifier of a COB on a CANopen network. The identifier determines the priority of a COB.

CPU

(*central processing unit*) Brain of an industrial manufacturing process, also known as the processor or controller. It automates a process as opposed to relay control systems. CPUs are computers suited to survive the harsh conditions of the industrial environment.

D

device DDT (DDDT)

(*Device derived data type*) Predefined by the manufacturer and not editable. It contains the I/O language elements of an I/O module.

DHCP

(*dynamic host configuration protocol*) An extension of the BOOTP communications protocol that provides for the automatic assignment of IP addressing settings, including IP address, subnet mask, gateway IP address, and DNS server names. DHCP does not require the maintenance of a table identifying each network device. The client identifies itself to the DHCP server using either its MAC address, or a uniquely assigned device identifier. The DHCP service utilizes UDP ports 67 and 68.

DIO network

A network containing distributed equipment, in which I/O scanning is performed by a CPU with DIO scanner service on the local rack. DIO network traffic is delivered after RIO traffic, which takes priority in an RIO network.

DTM

(*Device type manager*) Device driver running on the host PC. It provides a unified structure for accessing device parameters, configuring and operating the devices, and troubleshooting devices. DTMs can range from a simple graphical user interface (GUI) for setting device parameters to a highly sophisticated application capable of performing complex real-time calculations for diagnosis and maintenance purposes. In the context of a DTM, a device can be a communications module or a remote device on the network.

Refer to FDT.

E

EDS

(*electronic data sheet*) Simple text file that describes the configuration capabilities of a device. EDS files are generated and maintained by the manufacturer of the device.

EMCY

(*emergency*) A trigger event, generated by an internal error/fault. This object is transmitted with each new error, since error codes are independent mechanisms.

Ethernet

A 10 Mb/s, 100 Mb/s, or 1 Gb/s, CSMA/CD, frame-based LAN that can run over copper twisted pair or fiber optic cable, or wireless. The IEEE standard 802.3 defines the rules for configuring a wired Ethernet network; the IEEE standard 802.11 defines the rules for configuring a wireless Ethernet network. Common forms include 10BASE-T, 100BASE-TX, and 1000BASE-T, which can utilize category 5e copper twisted-pair cables and RJ45 modular connectors.

EtherNet/IP™

A network communication protocol for industrial automation applications. It combines the standard Internet transmission protocols of TCP/IP and UDP with the application layer common industrial protocol (CIP) to support both high-speed data exchange and industrial control. EtherNet/IP employs electronic data sheets (EDS) to classify each network device and its functionality.

explicit messaging

TCP/IP-based messaging for Modbus TCP and EtherNet/IP. It is used for point-to-point, client/server messages that include both data, typically unscheduled information between a client and a server, and routing information. In EtherNet/IP, explicit messaging is considered class 3 type messaging, and can be connection-based or connectionless.

F

FAST

An event-triggered (FAST) task is an optional, periodic processor task that identifies high priority, multiple scan requests, which is run through its programming software. A FAST task can schedule selected I/O modules to have their logic solved more than once per scan. The FAST task has two sections:

- IN: Inputs are copied to the IN section before execution of the FAST task.
- OUT: Outputs are copied to the OUT section after execution of the FAST task.

FDR

(*fast device replacement*) A service that uses configuration software to replace an inoperable product.

FDT

(*Field device tool*) The technology that harmonizes communication between field devices and the system host.

FTP

(*file transfer protocol*) A protocol that copies a file from one host to another over a TCP/IP-based network, such as the Internet. FTP uses a client/server architecture as well as separate control and data connections between the client and server.

G

gateway

A gateway device interconnects two different networks, sometimes through different network protocols. When it connects networks based on different protocols, a gateway converts a datagram from one protocol stack into the other. When used to connect two IP-based networks, a gateway (also called a router) has two separate IP addresses, one on each network.

H

harsh environment

Resistance to hydrocarbons, industrial oils, detergents, and solder chips. Relative humidity up to 100%, saline atmosphere, significant temperature variations, operating temperature between -10 °C and + 70 °C, or in mobile installations.

health bit

Variable that indicates the communication state of the channels.

HMI

(*Human machine interface*) System that allows interaction between a human and a machine.

HTTP

(*hypertext transfer protocol*) A networking protocol for distributed and collaborative information systems. HTTP is the basis of data communication for the Web.

I

implicit messaging

UDP/IP-based class 1 connected messaging for EtherNet/IP. Implicit messaging maintains an open connection for the scheduled transfer of control data between a producer and consumer. Because an open connection is maintained, each message contains data primarily, without the overhead of object information, plus a connection identifier.

IP address

The 32-bit identifier, consisting of both a network address and a host address assigned to a device connected to a TCP/IP network.

L

local rack

An M580 rack containing the CPU and a power supply. A local rack consists of one or two racks: the main rack and the extended rack, which belongs to the same family as the main rack. The extended rack is optional.

M

MAC address

(*media access control address*) A unique 48-bit number associated with a specific piece of hardware. The MAC address is programmed into each network card or device when it is manufactured.

mapping

Transformation of data consigned in a special and different format.

MAST

(*master*) Deterministic processor task that is run through its programming software. The MAST task schedules the RIO module logic to be solved in every I/O scan. The MAST task has two sections:

- IN: Inputs are copied to the IN section before execution of the MAST task.
- OUT: Outputs are copied to the OUT section after execution of the MAST task.

N

NIM

(*Network interface module*) A NIM resides in the first position on an STB island (leftmost on the physical setup). The NIM provides the interface between the I/O modules and the fieldbus master. It is the only module on the island that is fieldbus-dependent, a different NIM is available for each fieldbus.

NMT

(*Network management*) CANopen protocols that provide services for network initialization, detected error control, and device status control.

NTP

(*Network time protocol*) Protocol for synchronizing computer system clocks. The protocol uses a jitter buffer to resist the effects of variable latency.

O

O->T

(*originator to target*) Refer to originator and target.

originator

In EtherNet/IP, a device is considered the originator when it initiates a CIP connection for implicit or explicit messaging communications or when it initiates a message request for unconnected explicit messaging.

P

PDO

(*process data object*) An unconfirmed broadcast message sent from a producer device to a consumer device in a CAN-based network. The transmit PDO from the producer device has a specific identifier that corresponds to the receive PDO of the consumer devices.

R

RIO network

An Ethernet-based network that contains three types of RIO de vices: a local rack, an RIO drop, and a ConneXium extended dual-ring switch (DRS). Distributed equipment may also participate in a RIO network via connection to DRSs.

RPDO

(*received process data object*) Refer to PDO.

RPI

(requested packet interval) The time period between cyclic data transmissions requested by the scanner. EtherNet/IP devices publish data at the rate specified by the RPI assigned to them by the scanner, and they receive message requests from the scanner at each RPI.

S

SDO

(*service data object*) A message used by the fieldbus master to access (read/write) the object directories of network nodes in CAN-based networks. SDO types include service SDOs (SSDOs) and client SDOs (CSDOs).

SNMP

(*simple network management protocol*) Protocol used in network management systems to monitor network-attached devices. The protocol is part of the Internet protocol suite (IP) as defined by the Internet engineering task force (IETF). It consists of network management guidelines, including an application layer protocol, a database schema, and a set of data objects.

SNTP

(*simple network time protocol*) Refer to NTP.

string

A variable that is a series of ASCII characters.

subnet mask

The 32-bit value used to hide (or mask) the network portion of the IP address and reveal the host address of a device on a network using the IP protocol.

T

T->O

(*target to originator*) Refer to originator and target.

task

A group of sections and subroutines, executed cyclically or periodically for the MAST task and periodically for the FAST task. A task possesses a level of priority and is linked to inputs and outputs of the controller. These I/O are refreshed in relation to the task. A controller can have several tasks.

TPDO

(*transmit process data object*) Refer to PDO.

trap

A trap is an event directed by an SNMP agent that indicates one of these events:

- A change has occurred in the status of an agent.
- An unauthorized SNMP manager device has attempted to get data from (or change data on) an SNMP agent.

V

variable

Memory unit that is addressed and modified by a program.

Index

A

authorized devices
  cyber security, *108*
AUX task, *104*

B

backplane, *30*
BMECXM0100
  description, *18*
bootup tab, *80*
bus start, *43*

C

CANopen
  bus parameters, *93*
  connector, *20*
  device configuration, *69*
  device import, *69*
CIP object
  301 hex, *176*
  302 hex, *181*
  350 hex, *183*
  352 hex, *187*
  353 hex, *190*
communication - instructions
  READ_SDO, *122*
  WRITE_SDO, *125*
communication profile, *25*
configuration tab, *75*
configuring the devices
  STB, *85*
  Tesys U, *85*
configuring the servodrives
  ATV, *85*
  Lexium 05, *85*
CXM expert diagnosis tab, *141*
CXM status tab, *140*
cyber security, *27*
  authorized devices, *108*

D

device disconnection, *44*
DTM
  application area, *102*
  description, *100*
  general layout, *99*
  module DTM, *140*
  navigation area, *100*, *101*
  status bar, *103*

E

error control
  heartbeat, *79*
  node guarding, *79*
  tab, *79*
Ethernet configuration tab, *89*
event timer, *77*

F

fallback, *106*
FAST task, *104*

H

hold up time, *106*

I

inhibit time, *77*

L

limits, *27*
  CXM, *40*
  M580, *40*

M

MAST task, *104*, *105*
motion function bloc, *27*

N

NMT (network management), *79*

O

object dictionary, *159*
object dictionary tab, *83*

P

PDO multi-mapping, *78*
PDO tab, *77*

R

READ_SDO, *121*, *122*
  example, *128*
replacing
  CANopen module, *33*
  CANopen slave, *34*
request packet interval, *105*
restrictions, *27*
RPI
  performance, *43*
  values, *105*

S

SDO
  performance, *43*
SDO timeout, *121*
slave live list tab, *141*
starting mode, *106*
system architecture, *25*

T

task
  AUX, *104*
  characteristics, *104*
  cycle time, *43*
  FAST, *104*
  MAST, *104*, *105*
transmission type, *77*

W

web pages
  diagnostics, *89*, *145*
WRITE_SDO, *121*, *125*
  example, *128*